



Ferramentas de IA para UX & UI
Autoria: Álvaro Camillo Neto

Introdução

A engenharia de software está passando por uma das maiores transformações da sua história. Durante décadas, o desenvolvimento de aplicações esteve centrado principalmente na escrita manual de código, na construção artesanal de componentes e na automação tradicional baseada em regras rígidas e determinísticas. Com o avanço dos modelos generativos e dos agentes autônomos, esse cenário começou a mudar de forma acelerada.

Hoje, inteligência artificial deixou de ser apenas um tema de pesquisa ou uma funcionalidade isolada de produto. Ela passou a fazer parte do próprio fluxo operacional das equipes de tecnologia. A IA participa do refinamento de requisitos, da arquitetura, da geração de código, da criação de interfaces, da automação de testes, da revisão técnica e até da execução de tarefas completas de desenvolvimento.

Mas existe um ponto extremamente importante: utilizar IA profissionalmente não significa substituir engenharia de software tradicional. Pelo contrário. Quanto mais avançadas ficam as ferramentas, mais importante se torna a base sólida de arquitetura, qualidade, validação, versionamento, especificação e pensamento crítico.

Foi exatamente com esse objetivo que esta formação foi construída.

Ao longo desta jornada, o foco não esteve apenas em aprender ferramentas específicas, mas em compreender os fundamentos do desenvolvimento AI-Native. Em vez de tratar inteligência artificial como um simples “gerador de código”, exploramos como integrá-la de maneira estratégica dentro de fluxos reais de engenharia de software moderna.

No primeiro módulo, começamos pelos fundamentos da engenharia de IA aplicada ao desenvolvimento. Exploramos como ferramentas generativas podem apoiar equipes desde o refinamento de requisitos até o detalhamento técnico de tarefas. Trabalhamos conceitos como edge cases, UX Writing, comunicação técnica, estruturação de prompts e arquitetura orientada por contexto. O objetivo foi entender que IA não serve apenas para programar, mas também para ampliar produtividade e clareza em todas as etapas do ciclo de desenvolvimento.

Capítulos do Módulo 1

- Refinamento de Requisitos e Edge Cases
- Refinamento de UX Writing e Comunicação Técnica
- Preparação e Sanitização de Datasets
- Laboratório: Structured Prompts no AI Studio

- Prompt as Code & Integração

No segundo módulo, avançamos para desenvolvimento assistido por agentes autônomos. Configuramos ambientes modernos utilizando Angular, MCPs e ferramentas agênticas como AntigraVity. Trabalhamos design systems, acessibilidade, integração visual, criação de componentes reutilizáveis e geração de interfaces utilizando ferramentas como Stitch e Figma. Aqui começamos a entender como agentes podem atuar como copilotos técnicos dentro do fluxo de desenvolvimento front-end moderno.

Capítulos do Módulo 2

- Setup Angular e MCP
- Criação de Tokens, Estilização
- Geração de Componentes Acessíveis
- Criando Interfaces Utilizando o Stitch
- Criando Interfaces a partir do Figma
- Corrigindo a Interface utilizando IA

No terceiro módulo, evoluímos para cenários enterprise. Estruturamos monorepos com Nx, configuramos bibliotecas compartilhadas, trabalhamos Spec-Driven Development com OpenSpec e aprendemos a orquestrar múltiplos agentes em paralelo utilizando Git WorkTree. Também exploramos desenvolvimento assíncrono em nuvem utilizando Google Jules e práticas modernas de integração contínua, code review e melhoria contínua. Esse módulo mostrou como IA pode ser integrada em fluxos reais de equipes distribuídas e arquiteturas corporativas.

Capítulos do Módulo 3

- A Fundação Enterprise (Nx Monorepo, MCP e Execução)
- Spec-Driven Development com OpenSpec
- Orquestração Paralela (Isolamento Físico e OpenSpec Apply)
- Code Review e Melhoria Contínua
- Escala na Nuvem Orquestração e Cadastro de Eventos com Google Jules

No quarto módulo, o foco passou a ser qualidade e automação de testes. Trabalhamos arquitetura de QA utilizando Nx, Cypress e Playwright. Exploramos geração automatizada de testes com OpenSpec, automação semântica com self-healing e agentes autônomos operando browsers através de Playwright MCP. O objetivo foi mostrar como IA está transformando profundamente a forma como pensamos automação funcional, regressão e qualidade de software.

Capítulos do Módulo 4

- Arquitetura de QA no Nx Cypress Tradicional e Geração com OpenSpec
- O Fim da Fragilidade Asserções Semânticas com Cypress

- O Agente de QA Autônomo com Playwright MCP

Por fim, no último módulo, demos um passo ainda mais importante: a IA deixou de ser apenas uma ferramenta para desenvolvimento e passou a fazer parte da própria arquitetura da aplicação. Criamos o BragBot utilizando Angular SSR, GenKit e Gemini, integrando LLMs de forma estruturada através de flows, schemas, validação com Zod e integração full-stack. Esse módulo mostrou como frameworks modernos começam a tratar inteligência artificial como um componente arquitetural real, exigindo preocupações com contratos, observabilidade, segurança e desacoplamento.

Capítulos do Módulo 5

- O que é o GenKit e Configuração Inicial do Projeto
- Design System UNIPDS e Interface Mockada
- O Cérebro do GenKit: Flows, Zod e Google AI
- O Micro-BFF, Integração Full-Stack
- O Engenheiro AI-Native (Revisão da Jornada e Encerramento)

Ao final desta formação, o objetivo não é apenas conhecer ferramentas específicas. O verdadeiro objetivo é compreender uma nova forma de construir software. Uma forma onde desenvolvimento deixa de ser exclusivamente operacional e passa a ser cada vez mais orientado por contexto, arquitetura, especificação e colaboração entre humanos e sistemas inteligentes.

O mercado continuará mudando rapidamente. Novos modelos surgirão. Ferramentas serão substituídas. Frameworks evoluirão constantemente. Mas os fundamentos trabalhados aqui continuarão relevantes: engenharia de contexto, especificação estruturada, validação, observabilidade, automação, arquitetura e pensamento crítico.

Porque o futuro da engenharia de software provavelmente não será nem completamente manual nem completamente automatizado. O futuro será colaborativo. E os profissionais mais valiosos serão justamente aqueles capazes de integrar engenharia clássica e inteligência artificial dentro de fluxos modernos, produtivos e sustentáveis de desenvolvimento.

Módulo 01 - Engenharia de Requisitos e Design de Sistemas com IA

Capítulo 1 - Refinamento de Requisitos e Edge Cases

Quando começamos a desenvolver um sistema, principalmente aplicações que possuem impacto direto na experiência do usuário, existe uma tendência muito forte de iniciar imediatamente pela implementação técnica. O desenvolvedor recebe um requisito, abre a IDE, cria pastas, define componentes, organiza endpoints e começa a construir a solução. Isso acontece porque, naturalmente, nós fomos treinados durante muitos anos para pensar primeiro na execução técnica. Porém, à medida que os sistemas se tornam mais complexos e os usuários mais exigentes, fica cada vez mais evidente que a qualidade de uma aplicação não depende apenas do código. Ela depende da qualidade do entendimento do problema.

Antes de pensar em interface, componentes gráficos ou arquitetura, precisamos entender profundamente a jornada do usuário. Precisamos compreender o que ele deseja fazer, quais obstáculos podem surgir durante o processo e como o sistema deve reagir em diferentes situações. E isso não envolve apenas o chamado caminho feliz, quando tudo funciona perfeitamente. Na prática, a maior parte da experiência do usuário é construída justamente nos momentos em que algo não sai como esperado.

É nesse contexto que a inteligência artificial começa a se tornar uma ferramenta extremamente poderosa para engenharia de requisitos, UX e design de sistemas. O objetivo aqui não é substituir analistas, product owners, desenvolvedores ou designers. Pelo contrário. A IA funciona como um mecanismo de expansão da nossa capacidade analítica. Ela nos ajuda a explorar cenários, identificar lacunas e enxergar situações que muitas vezes passariam despercebidas durante o refinamento.

Nesta aula, quero mostrar exatamente como utilizar IA para melhorar o refinamento de requisitos e aumentar a qualidade da análise antes mesmo de começarmos a implementar qualquer funcionalidade.

Refinamento de requisitos além do caminho feliz

Quando um requisito chega ao time de desenvolvimento, normalmente ele vem em um formato simplificado. O Product Owner costuma descrever o objetivo principal da funcionalidade e as regras mais visíveis do negócio. Em muitos casos, isso é suficiente para iniciar uma conversa, mas raramente é suficiente para iniciar uma implementação de qualidade.

Imagine um requisito simples para um fluxo de PIX agendado. O usuário deve selecionar um contato, informar um valor, escolher uma data e confirmar a operação. Caso tudo ocorra corretamente, o sistema apresenta um comprovante. Além disso, existem algumas regras básicas, como limite diário de transferência, proibição de agendamento para o mesmo dia e possibilidade de cancelamento.

À primeira vista, parece um requisito completo. Porém, quando analisamos esse fluxo com mais profundidade, começamos a perceber quantas perguntas ainda não foram respondidas. O sistema valida saldo no momento do agendamento ou apenas no momento

da execução? O valor agendado já consome imediatamente o limite diário do usuário? O que acontece se a chave PIX for removida depois do agendamento? Como o sistema deve se comportar em feriados? Existe alguma limitação de horário para cancelamento? Como a interface reage quando há falha de comunicação?

Essas perguntas normalmente não aparecem no requisito inicial. E isso não significa que o Product Owner fez um trabalho ruim. Significa apenas que requisitos brutos raramente contemplam todas as possibilidades necessárias para construir uma experiência realmente consistente.

É justamente aqui que a inteligência artificial se torna extremamente útil.

IA como ferramenta de expansão analítica

Uma das aplicações mais interessantes da IA no contexto de desenvolvimento de software é utilizá-la como suporte para exploração de cenários. Em vez de enxergar o modelo apenas como uma ferramenta de geração de código, começamos a utilizá-lo como um mecanismo de reflexão assistida.

Quando enviamos um requisito para a IA e pedimos que ela identifique edge cases, ambiguidades, riscos operacionais e estados de interface, estamos basicamente ampliando nossa própria capacidade de análise. A IA passa a funcionar como uma segunda camada de pensamento crítico.

O mais importante nesse processo é entender que a qualidade da resposta depende diretamente da qualidade do contexto fornecido. Não basta simplesmente jogar um requisito no chat e esperar uma análise sofisticada. Precisamos orientar corretamente o comportamento do modelo.

Por isso, antes mesmo de enviar o requisito, definimos o papel que queremos que a IA desempenhe. Podemos pedir que ela atue como arquiteto de software sênior, especialista em UX, analista de requisitos ou engenheiro de qualidade. Essa definição muda completamente o tipo de resposta gerada.

Quando configuramos corretamente o contexto, a IA começa a analisar o requisito sob uma perspectiva muito mais estratégica. Ela passa a identificar caminhos alternativos, estados de erro, conflitos de negócio e cenários ocultos que normalmente só seriam descobertos durante homologação ou até mesmo em produção.

A importância da engenharia de prompts

Outro ponto extremamente importante é a engenharia de prompts. Muitas pessoas utilizam IA de forma superficial porque não estruturam corretamente suas instruções. Modelos de linguagem respondem muito melhor quando recebem direcionamentos claros.

Um prompt eficiente normalmente possui quatro elementos principais: papel, objetivo, regras e formato de saída.

Primeiro definimos o papel. Queremos que o modelo pense como um arquiteto? Como um analista? Como um especialista em UX? Depois definimos o objetivo. Queremos analisar requisitos? Encontrar edge cases? Mapear estados de interface? Em seguida estabelecemos regras específicas. Podemos pedir para identificar caminhos felizes e infelizes, listar estados de carregamento, detectar conflitos de negócio e estruturar cenários de falha. Por fim, definimos o formato da resposta.

Esse nível de organização melhora drasticamente a qualidade da análise gerada pela IA.

Quando o modelo entende exatamente qual problema precisa resolver e qual perspectiva deve adotar, a utilidade prática da resposta cresce de forma impressionante.

Edge cases e cenários ocultos

Uma das maiores vantagens da IA no refinamento é sua capacidade de sugerir edge cases. Edge cases são situações extremas, menos frequentes ou pouco óbvias que podem impactar o comportamento do sistema.

Em sistemas financeiros, esses cenários são especialmente críticos.

No fluxo de PIX agendado, por exemplo, um caso interessante é a virada de data. Imagine um usuário iniciando o processo às 23h59. Durante a confirmação, o horário muda para meia-noite. O sistema ainda considera aquele PIX como transferência para o mesmo dia? O agendamento continua válido? A interface precisa atualizar automaticamente?

Outro exemplo envolve saldo insuficiente. O requisito original pode não deixar claro quando a validação ocorre. Se o usuário agenda uma transferência hoje para execução amanhã, o sistema reserva aquele saldo imediatamente ou apenas no momento da execução? Caso o dinheiro não esteja disponível no dia programado, o usuário será avisado? O agendamento será cancelado automaticamente?

Também existem questões relacionadas à própria chave PIX. O que acontece se a chave for removida depois do agendamento? O sistema deve validar novamente antes da execução? Como o usuário será notificado em caso de falha?

Perceba que esses cenários dificilmente aparecem em um requisito bruto. Eles surgem durante o aprofundamento analítico. E é exatamente aí que a IA pode acelerar enormemente o trabalho do time.

Estados de interface e experiência do usuário

Outro aspecto frequentemente negligenciado durante o refinamento é o mapeamento dos estados da interface.

Muitos desenvolvedores pensam apenas na tela funcionando perfeitamente. Porém, interfaces reais possuem diversos estados intermediários. E grande parte da percepção de qualidade do usuário vem justamente desses detalhes.

Existem os estados de carregamento, por exemplo. Quando o sistema está validando uma chave PIX, buscando contatos ou processando uma confirmação, o usuário precisa receber feedback visual adequado. Caso contrário, ele sente que a aplicação travou.

Também existem estados vazios. O que acontece quando não há contatos cadastrados? Quando não existem agendamentos ativos? Quando nenhuma busca retorna resultados? Uma boa experiência depende de como esses estados são tratados.

Além disso, existem os estados de erro. Saldo insuficiente, limite excedido, falha de comunicação, chave inválida, timeout de processamento e indisponibilidade do serviço bancário são situações comuns em sistemas financeiros. A forma como o sistema comunica esses problemas impacta diretamente a confiança do usuário.

Quando utilizamos IA para mapear esses estados, conseguimos construir interfaces muito mais maduras e previsíveis.

IA como ferramenta de brainstorming técnico

Existe uma característica muito interessante na utilização da IA para refinamento: ela funciona extremamente bem como ferramenta de brainstorming técnico.

Nem sempre a IA traz respostas perfeitas. Muitas vezes ela sugere cenários improváveis ou interpretações discutíveis. Porém, isso não reduz seu valor. Na prática, o grande benefício está em provocar novas perguntas.

Durante uma reunião de refinamento, a IA pode levantar pontos que estimulem discussões importantes entre desenvolvimento, produto, UX e qualidade. Isso enriquece muito a análise.

Em vez de iniciar a conversa do zero, o time passa a discutir em cima de uma lista já expandida de possibilidades. Isso acelera o refinamento e melhora significativamente a qualidade das decisões.

Relação entre refinamento e desenvolvimento assistido por IA

Outro ponto importante é que requisitos melhores produzem resultados melhores em ferramentas de geração de código.

Muitas equipes começam a utilizar IA para desenvolvimento sem perceber que a qualidade do código gerado depende diretamente da qualidade do requisito fornecido. Se o requisito for superficial, o código gerado também será superficial.

Quanto mais refinado for o contexto, mais precisas serão as respostas da IA.

Isso significa que um bom refinamento melhora não apenas a análise de negócio, mas também:

- geração de código
- criação de testes
- documentação
- modelagem de APIs
- definição de contratos
- automações de QA

A IA depende profundamente da clareza do contexto recebido.

O papel humano continua essencial

Apesar de toda evolução dos modelos de linguagem, existe algo que continua sendo exclusivamente humano: a capacidade de entender contexto organizacional e comportamento real de usuários.

A IA não conhece estratégia de negócio, limitações políticas da empresa, cultura organizacional ou nuances emocionais da experiência do usuário. Ela apenas processa padrões estatísticos baseados no contexto fornecido.

Por isso, ela não substitui analistas, arquitetos ou desenvolvedores.

Ela amplia a produtividade desses profissionais.

Os melhores resultados aparecem quando combinamos pensamento crítico humano com capacidade analítica da inteligência artificial.

Capítulo 2 - Design de Fluxos Lógicos e Diagramação

Depois que refinamos um requisito e começamos a enxergar os edge cases, os estados de interface e os possíveis problemas da jornada do usuário, surge uma segunda necessidade extremamente importante dentro do processo de desenvolvimento: documentar essa jornada de maneira clara, visual e profissional.

Esse é um ponto que muitos times negligenciam. Em várias empresas, o refinamento acontece apenas verbalmente durante reuniões, e depois toda aquela discussão acaba se perdendo em mensagens, comentários de ticket ou até mesmo na memória das pessoas envolvidas. O problema disso é que a equipe inteira começa a trabalhar baseada em interpretações diferentes do mesmo fluxo.

O desenvolvedor entende de uma forma, o QA entende de outra, o Product Owner imagina outra experiência, e o arquiteto cria uma solução pensando em outro comportamento completamente diferente. Quando isso acontece, o retrabalho é inevitável.

Por isso, documentar fluxos corretamente continua sendo uma das práticas mais importantes dentro da engenharia de software moderna.

E aqui entra uma combinação extremamente poderosa: inteligência artificial mais diagramas gerados por código.

Durante muito tempo, criar diagramas era uma atividade cansativa. Precisávamos abrir ferramentas visuais, arrastar caixas, conectar elementos manualmente, ajustar alinhamento, reposicionar textos e manter tudo sincronizado conforme o sistema evoluía. Além de consumir tempo, esse processo normalmente fazia com que os diagramas ficassem desatualizados rapidamente.

Hoje, utilizando ferramentas como Mermaid junto com modelos de IA, conseguimos transformar documentação técnica em algo muito mais rápido, iterativo e integrado ao fluxo real de desenvolvimento.

O Mermaid é uma ferramenta open source extremamente popular que permite criar diagramas utilizando texto. Em vez de desenhar manualmente, escrevemos uma estrutura simples descrevendo os elementos e conexões do fluxo. A partir disso, o Mermaid renderiza automaticamente o diagrama visual.

Essa abordagem muda completamente a maneira como documentamos sistemas.

Primeiro porque o diagrama passa a ser código. Isso significa que ele pode ser versionado junto com o projeto, armazenado no Git, revisado em pull requests e evoluído da mesma maneira que evoluímos nossa aplicação.

Segundo porque conseguimos utilizar IA para gerar esses diagramas automaticamente com base nos requisitos refinados.

Esse é o ponto mais interessante do processo.

Depois de realizar toda análise de edge cases e refinamento da funcionalidade de PIX agendado, agora podemos pedir para a IA transformar aquela análise em um fluxo visual estruturado.

Perceba como isso cria uma cadeia muito poderosa.

Primeiro utilizamos IA para expandir o requisito. Depois utilizamos novamente IA para transformar o refinamento em documentação visual.

Isso reduz drasticamente o tempo gasto em atividades operacionais e aumenta muito a qualidade das discussões técnicas.

O mais importante aqui é entender que não estamos apenas desenhando telas ou fluxos bonitos. Estamos criando representações visuais da jornada do usuário e dos estados do sistema.

Quando pensamos dessa forma, o diagrama deixa de ser apenas documentação e passa a funcionar como ferramenta de alinhamento técnico.

No caso do fluxo de PIX agendado, por exemplo, podemos representar visualmente desde o carregamento inicial da tela até todos os caminhos alternativos possíveis.

O usuário abre a tela. O sistema carrega contatos frequentes. Caso não existam contatos, a interface precisa apresentar um empty state. Se o usuário selecionar um contato, informar um valor inválido ou ultrapassar o limite diário, novos estados de erro precisam aparecer.

Depois disso, ainda temos processos relacionados à autenticação, biometria, comunicação com APIs, timeout, indisponibilidade de serviço e validação de saldo.

Quando tudo isso é representado visualmente, a equipe inteira passa a enxergar o fluxo de maneira muito mais clara.

Esse tipo de documentação ajuda diretamente diversas áreas.

O time de QA consegue criar casos de teste com muito mais precisão. O arquiteto consegue pensar melhor nos componentes necessários. O desenvolvedor entende os estados da interface antes de começar a implementação. O Product Owner consegue validar se a jornada realmente faz sentido para o usuário.

Perceba que o diagrama deixa de ser apenas algo técnico e passa a ser uma representação compartilhada da experiência do produto.

Outro ponto extremamente importante é que o Mermaid possui uma sintaxe relativamente simples. Isso facilita muito a integração com IA.

Os modelos atuais conseguem gerar diagramas Mermaid com bastante qualidade porque a estrutura textual é objetiva e previsível. Podemos definir exatamente o tipo de diagrama que queremos, a orientação visual, os tipos de elementos utilizados e até regras de estilização.

Esse detalhe é importante porque não basta simplesmente pedir para a IA gerar qualquer fluxo. Quanto melhor estruturarmos o prompt, melhor será a qualidade da documentação gerada.

Quando definimos que queremos um flowchart orientado de cima para baixo, cobrindo caminhos felizes e infelizes, representando estados de loading, erros e validações, o modelo consegue produzir uma documentação muito mais próxima da realidade do sistema.

Além disso, conseguimos orientar visualmente o comportamento do diagrama.

Podemos definir que ações do usuário sejam representadas por retângulos, decisões de negócio por losangos e estados críticos com cores específicas. Isso transforma o diagrama em algo muito mais legível.

Esse tipo de refinamento visual parece simples, mas melhora muito a comunicação entre áreas técnicas e não técnicas.

Outro ponto extremamente interessante do Mermaid é a integração com ferramentas modernas do ecossistema de desenvolvimento.

Hoje, plataformas como GitHub, Visual Studio Code, Jira e diversas outras ferramentas já possuem suporte nativo ou plugins para renderização automática de diagramas Mermaid.

Isso significa que o diagrama pode existir diretamente dentro da documentação técnica do projeto sem depender de arquivos externos ou imagens exportadas manualmente.

Esse detalhe resolve um problema histórico da documentação de software: desatualização.

Diagramas tradicionais normalmente envelhecem muito rápido porque qualquer alteração exige edição manual. Como isso dá trabalho, o time deixa de atualizar a documentação.

Com Mermaid, o processo fica muito mais simples.

Se uma nova regra de negócio surgir, basta alterar algumas linhas do fluxo e o diagrama é atualizado automaticamente.

Isso cria um processo muito mais iterativo.

Por exemplo, durante uma reunião de refinamento, o time pode identificar um novo cenário de autenticação negada. Em vez de abrir uma ferramenta gráfica e redesenhar tudo, basta adicionar uma nova condição no fluxo textual.

O diagrama é renderizado imediatamente já refletindo a nova regra.

Essa velocidade muda completamente a dinâmica das discussões técnicas.

A documentação deixa de ser um processo pesado e passa a ser uma ferramenta viva dentro do desenvolvimento.

Outro conceito importante aqui é entender que esses diagramas representam jornadas e não necessariamente arquitetura.

Isso é fundamental.

Muitas pessoas confundem fluxo funcional com arquitetura de sistema. O objetivo inicial aqui é mapear experiência do usuário, decisões de negócio e estados da aplicação.

Claro que posteriormente esses fluxos podem servir como base para arquitetura. Afinal, quando identificamos todos os estados da jornada, começamos naturalmente a enxergar quais serviços, componentes e integrações serão necessários.

Mas o foco principal nesse momento ainda é entendimento do comportamento da aplicação.

Isso também ajuda muito no desenvolvimento orientado a UX.

Quando o time visualiza todos os estados possíveis da interface antes de implementar, a qualidade da experiência cresce consideravelmente.

O desenvolvedor deixa de pensar apenas na tela principal e passa a considerar:

- estados vazios
- estados de carregamento
- estados de falha
- estados de autenticação
- fluxos alternativos
- interrupções
- recuperação de erro

Isso produz aplicações muito mais robustas e previsíveis.

Outro benefício extremamente importante é a possibilidade de utilizar IA como acelerador contínuo de refinamento.

O fluxo não termina quando o diagrama é gerado.

Na prática, o time pode revisar o resultado, identificar novas situações e voltar para a IA refinando ainda mais o contexto.

Esse ciclo iterativo é extremamente poderoso.

A IA gera um primeiro fluxo. O time discute. Novas regras aparecem. O diagrama evolui. Novos cenários são descobertos. O refinamento amadurece.

Isso reduz drasticamente ambiguidades antes da implementação.

E existe um detalhe muito importante aqui: a IA não está tomando decisões sozinha.

Ela está funcionando como copiloto.

Esse conceito é essencial.

Muita gente cria expectativas irreais achando que a IA substituirá completamente análise técnica e arquitetura. Na prática, o melhor resultado aparece quando utilizamos IA como mecanismo de aceleração de trabalho humano especializado.

Ela reduz tarefas repetitivas, acelera brainstorming, ajuda no refinamento e melhora documentação. Porém, a validação continua sendo responsabilidade do time.

O papel humano continua central no processo.

A experiência do arquiteto, do desenvolvedor, do QA e do Product Owner ainda é indispensável para validar se aquele fluxo realmente representa o comportamento esperado do produto.

A diferença é que agora conseguimos explorar possibilidades com muito mais velocidade.

Em vez de gastar horas desenhando manualmente diagramas básicos, podemos focar nossa energia intelectual nas decisões realmente importantes do sistema.

Esse é um dos maiores impactos da IA dentro da engenharia moderna: liberar profissionais qualificados das tarefas operacionais para que eles possam dedicar mais tempo à análise estratégica.

Capítulo 3 - Refinamento de UX Writing e Comunicação Técnica

Quando pensamos em desenvolvimento front-end, normalmente a atenção do time fica concentrada em componentes, layouts, APIs, arquitetura e regras de negócio. Porém, existe uma parte extremamente importante da experiência do usuário que muitas vezes recebe menos atenção do que deveria: os textos que aparecem na interface.

Toda aplicação conversa com o usuário o tempo inteiro.

Ela conversa quando confirma uma operação, quando apresenta um erro, quando informa um carregamento, quando orienta um próximo passo ou quando precisa explicar um problema. E essa comunicação tem impacto direto na percepção de qualidade do sistema.

É justamente nesse ponto que entra o conceito de UX Writing, também chamado em muitos contextos de microcopy.

UX Writing não significa apenas escrever mensagens bonitas para a tela. Na prática, estamos falando de projetar comunicação dentro da experiência do usuário. Cada frase exibida pelo sistema precisa possuir intenção, clareza e contexto.

Uma mensagem mal construída pode gerar insegurança, frustração e até abandono de fluxo. Já uma comunicação clara e orientativa reduz atrito, melhora entendimento e aumenta a confiança do usuário na aplicação.

O ponto importante aqui é perceber que escrever mensagens para interface não é simplesmente inventar frases no momento da implementação.

Esse trabalho normalmente envolve alinhamento entre produto, UX, negócio e desenvolvimento.

Existe tom de voz da empresa, padrão de comunicação, regras de consistência, termos específicos do domínio e até diretrizes emocionais dependendo do tipo de sistema.

Em aplicações financeiras, por exemplo, o cuidado precisa ser ainda maior. O usuário precisa sentir segurança durante toda a navegação. Mensagens agressivas, ambíguas ou excessivamente técnicas podem gerar desconfiança.

Por isso, sistemas modernos tratam UX Writing como parte estratégica do produto.

E novamente a inteligência artificial aparece como uma ferramenta extremamente poderosa para acelerar esse processo.

Assim como utilizamos IA para refinar requisitos e gerar diagramas, também podemos utilizá-la para estruturar mensagens da aplicação de maneira padronizada e profissional.

Mas aqui existe um detalhe importante: não queremos apenas gerar texto. Queremos transformar comunicação em ativo técnico reutilizável dentro do projeto.

Esse é um ponto muito relevante.

Em aplicações front-end modernas, normalmente evitamos escrever mensagens diretamente no código. Não é uma boa prática espalhar textos hardcoded dentro dos componentes da interface.

Existem vários motivos para isso.

Primeiro porque mensagens mudam constantemente durante evolução do produto. Segundo porque aplicações frequentemente precisam suportar múltiplos idiomas. Terceiro porque centralizar textos melhora manutenção e padronização.

Por isso, o mais comum é armazenar mensagens em arquivos estruturados, normalmente JSON, e utilizar bibliotecas de internacionalização para carregar esses textos dinamicamente.

Independentemente do framework utilizado, o conceito é praticamente o mesmo.

No Angular, React, Vue ou qualquer outro ecossistema moderno, geralmente utilizamos bibliotecas específicas para tradução e gerenciamento de mensagens.

O mais importante aqui não é a tecnologia escolhida, mas o padrão arquitetural adotado.

Quando centralizamos mensagens em arquivos estruturados, ganhamos diversas vantagens:

- facilidade de manutenção
- suporte multilíngue
- padronização textual
- desacoplamento da interface
- reutilização de mensagens
- consistência de comunicação

Além disso, essa abordagem facilita muito integração com IA.

Em vez de pedir apenas frases soltas, podemos solicitar diretamente estruturas prontas para utilização no projeto.

Esse detalhe muda completamente o nível de produtividade.

Ao definir corretamente o contexto da IA, conseguimos gerar automaticamente mensagens organizadas já no formato esperado pela biblioteca utilizada pela aplicação.

E aqui novamente entra a importância do System Prompt.

Na aula anterior utilizamos um contexto voltado para análise de requisitos e arquitetura. Agora trocamos completamente o papel do modelo.

Isso é extremamente importante quando trabalhamos com IA generativa.

O mesmo modelo pode atuar como arquiteto, analista, UX Writer, QA ou desenvolvedor dependendo do contexto fornecido.

Nesse caso, configuramos o comportamento da IA para atuar especificamente como especialista em UX Writing.

O objetivo passa a ser padronizar mensagens garantindo clareza, empatia e orientação ao usuário.

Esse ponto é muito importante porque comunicação de interface possui princípios próprios.

Uma boa mensagem não culpa o usuário.

Esse detalhe parece simples, mas faz enorme diferença na experiência.

Mensagens como “você digitou errado”, “erro do usuário” ou “dados inválidos” criam uma sensação negativa e muitas vezes agressiva.

Em vez disso, sistemas maduros utilizam linguagem mais acolhedora e orientativa.

Por exemplo:

“Não foi possível concluir a operação.”
“Verifique os dados informados.”
“O formato informado não foi reconhecido.”

Perceba como a comunicação muda completamente.

A responsabilidade deixa de ser atribuída diretamente ao usuário e a mensagem passa a funcionar como orientação.

Outro princípio importante é objetividade.

Boas mensagens reduzem ambiguidade e indicam claramente qual próximo passo o usuário deve executar.

Sempre que possível, o sistema deve evitar becos sem saída.

Se ocorreu um erro, o usuário precisa entender:

- o que aconteceu
- por que aconteceu
- o que ele pode fazer agora

Esse terceiro ponto é extremamente importante.

Mensagens eficazes normalmente orientam ação.

“Tente novamente.”
“Verifique sua conexão.”
“Ajuste o valor informado.”
“Escolha uma data futura.”

Esse tipo de orientação reduz ansiedade e melhora muito a experiência da interface.

Outro aspecto importante do UX Writing é padronização semântica.

Sistemas maduros evitam utilizar múltiplos termos diferentes para representar a mesma coisa.

Por exemplo, se a aplicação utiliza “transferência”, ela deve evitar alternar entre “envio”, “remessa” e “operação”.

O mesmo vale para conceitos como:

- agendamento
- chave PIX
- autenticação
- comprovante
- saldo disponível

Essa consistência melhora aprendizado e reduz confusão cognitiva.

E aqui novamente percebemos como IA pode ajudar.

Ao definir essas diretrizes no contexto do modelo, conseguimos fazer com que todas as mensagens geradas sigam o mesmo padrão linguístico.

Isso cria uma comunicação muito mais consistente em toda aplicação.

Outro ponto extremamente interessante é aproveitar a janela de contexto do modelo.

Modelos modernos conseguem manter grande quantidade de informação dentro da mesma conversa. Isso permite reutilizar todo refinamento realizado anteriormente.

Na prática, podemos utilizar o mesmo histórico do requisito refinado, dos edge cases identificados e dos diagramas gerados para agora produzir mensagens específicas para cada cenário.

Isso torna o processo muito mais inteligente.

A IA já conhece:

- os fluxos
- os estados
- os erros
- as regras
- os cenários de sucesso
- os comportamentos alternativos

Com isso, ela consegue gerar mensagens muito mais alinhadas ao contexto real da funcionalidade.

Esse é um dos pontos mais poderosos da IA generativa aplicada ao desenvolvimento: reaproveitamento contextual.

Quanto mais contexto consistente fornecemos ao modelo, melhores tendem a ser os resultados.

No caso do PIX agendado, por exemplo, conseguimos gerar mensagens específicas para:

- sucesso de agendamento
- saldo insuficiente
- limite excedido
- data inválida
- erro de autenticação
- timeout de serviço
- falha de comunicação

- cancelamento realizado
- chave PIX inválida

E além do texto em si, podemos estruturar a resposta já em formato técnico reutilizável.

Isso significa gerar objetos JSON contendo:

- código identificador
- título
- mensagem principal
- ação sugerida

Essa abordagem é extremamente interessante porque aproxima IA diretamente do fluxo de desenvolvimento.

O resultado deixa de ser apenas texto e passa a ser parte do próprio projeto.

Na prática, a IA começa a produzir artefatos técnicos utilizáveis.

Outro ponto importante é perceber que UX Writing também faz parte do Design System da empresa.

Muitas organizações possuem não apenas bibliotecas de componentes visuais, mas também guias de comunicação.

Esses guias definem:

- tom de voz
- terminologia oficial
- linguagem institucional
- regras de acessibilidade textual
- padrões emocionais
- estilo de comunicação

Empresas maiores costumam tratar isso de maneira extremamente séria porque consistência textual impacta diretamente percepção de marca.

Quando configuramos essas diretrizes dentro do contexto da IA, conseguimos gerar mensagens alinhadas ao padrão institucional da organização.

Isso reduz inconsistências entre equipes e acelera produção de interfaces.

Outro aspecto extremamente importante é a relação entre UX Writing e acessibilidade.

Mensagens claras ajudam não apenas experiência geral, mas também usuários com diferentes níveis de familiaridade tecnológica.

Sistemas excessivamente técnicos criam barreiras desnecessárias.

Uma boa comunicação reduz carga cognitiva e facilita entendimento da interface.

E isso é especialmente relevante em aplicações financeiras, sistemas públicos e plataformas utilizadas por usuários não técnicos.

Quanto mais simples, objetiva e humana for a comunicação, melhor tende a ser a experiência.

Existe também um ganho enorme em produtividade.

Tradicionalmente, equipes gastam muito tempo discutindo textos pequenos dentro da interface.

Com IA, conseguimos acelerar geração inicial dessas mensagens mantendo padronização e qualidade.

Claro que revisão humana continua essencial.

A IA pode sugerir textos excelentes, mas a validação final ainda precisa considerar contexto do produto, identidade da empresa e expectativas dos usuários.

Mais uma vez, a IA funciona como copiloto e não como substituto completo da análise humana.

O papel do profissional continua sendo fundamental.

O UX Writer, o Product Designer, o desenvolvedor e o Product Owner continuam responsáveis pela qualidade final da experiência.

A diferença é que agora conseguimos acelerar muito etapas operacionais e dedicar mais tempo às decisões realmente importantes da interface.

E esse talvez seja um dos maiores impactos da IA no desenvolvimento moderno.

Ela não elimina conhecimento especializado.

Ela amplifica produtividade de profissionais especializados.

Quando combinamos refinamento de requisitos, documentação de jornada e UX Writing assistido por IA, começamos a construir um fluxo extremamente poderoso de desenvolvimento orientado à experiência do usuário.

O sistema deixa de ser apenas um conjunto de telas e APIs e passa a funcionar como uma experiência cuidadosamente planejada em todos os níveis: comportamento, comunicação, validação, feedback e orientação do usuário.

Capítulo 4 - Preparação e Sanitização de Datasets

Quando trabalhamos com desenvolvimento de software em produção, existe uma mudança importante de mentalidade que acontece com times mais maduros. No começo, normalmente o foco da equipe está apenas em implementar funcionalidades solicitadas pelos stakeholders, Product Owners ou áreas de negócio. Porém, à medida que o sistema cresce e começa a receber usuários reais, percebemos que as informações mais valiosas nem sempre vêm das reuniões internas da empresa.

Muitas vezes, os maiores insights sobre problemas da aplicação aparecem diretamente no comportamento e no feedback dos usuários.

Hoje temos várias fontes de dados extremamente ricas para isso. Sistemas de suporte, plataformas de atendimento, reviews de lojas mobile, comentários em redes sociais, pesquisas de satisfação e tickets de atendimento se transformam em uma fonte contínua de aprendizado sobre o produto.

Esses dados mostram coisas que frequentemente não apareceram durante desenvolvimento, homologação ou testes internos.

Usuários reais utilizam o sistema de formas imprevisíveis. Eles encontram edge cases, falhas de UX, problemas de navegação, lentidão, inconsistências e comportamentos inesperados que o time nem imaginava durante o projeto.

Por isso, aprender a utilizar esses dados corretamente se tornou uma habilidade extremamente importante dentro da engenharia moderna de software.

Mas existe um problema.

Esses datasets normalmente chegam completamente desorganizados.

Quando olhamos dados brutos vindos de reviews ou plataformas de suporte, encontramos:

- mensagens incompletas
- textos mal escritos
- xingamentos
- reclamações sem contexto
- spam
- testes internos
- respostas automáticas
- ruído operacional
- informações irrelevantes
- e pior ainda, dados sensíveis

Esse último ponto é extremamente crítico.

É muito comum usuários colocarem informações pessoais dentro de comentários e tickets sem perceber os riscos envolvidos.

Eles enviam:

- CPF
- telefone
- e-mail
- endereço
- número de conta
- dados bancários
- informações pessoais de terceiros

Do ponto de vista de engenharia e compliance, isso representa um risco enorme.

Se esses dados forem processados inadequadamente, armazenados sem necessidade ou utilizados em pipelines de análise sem proteção adequada, a empresa pode criar problemas sérios relacionados à privacidade e segurança da informação.

É justamente nesse contexto que entra a importância da sanitização de datasets.

Antes de utilizar feedbacks de usuários para análise de produto, geração de tickets, melhoria de UX ou treinamento de modelos internos, precisamos limpar e preparar esses dados corretamente.

E novamente a inteligência artificial aparece como uma ferramenta extremamente poderosa para acelerar esse processo.

Tradicionalmente, sanitização de dados exigia pipelines complexos, regras manuais, expressões regulares extensas e muito processamento específico para diferentes formatos de informação.

Hoje, utilizando LLMs, conseguimos adicionar uma camada semântica extremamente inteligente nesse processo.

Esse é um detalhe muito importante.

Ferramentas tradicionais conseguem detectar padrões relativamente previsíveis, como CPF, telefone ou e-mail. Porém, modelos de linguagem conseguem interpretar contexto.

Isso significa que eles conseguem diferenciar:

- reclamações relevantes
- ruídos operacionais
- mensagens automatizadas

- spam
- conteúdo fora do domínio da aplicação
- testes internos
- comentários sem valor técnico

Essa capacidade muda completamente a qualidade da preparação dos dados.

Mas novamente existe um ponto fundamental: contexto.

A IA não sabe automaticamente o que queremos remover, preservar ou transformar.

Precisamos orientar corretamente o comportamento do modelo.

E aqui reaparece a importância do System Prompt.

Antes de enviar o dataset, definimos qual papel a IA deve assumir.

Nesse caso, o modelo passa a atuar como engenheiro de dados sênior e especialista em compliance e LGPD.

Esse detalhe é importante porque muda a forma como ele interpreta os dados recebidos.

Não queremos apenas limpar texto.

Queremos sanitizar informação mantendo valor analítico.

Essa diferença é fundamental.

Se removermos informação demais, perdemos contexto útil para análise de produto. Se removermos informação de menos, corremos riscos de privacidade e compliance.

Por isso o equilíbrio entre anonimização e preservação contextual é tão importante.

Na prática, começamos definindo regras claras para o modelo.

A primeira regra normalmente envolve remoção de informações pessoais identificáveis, conhecidas como PII.

PII significa qualquer dado que permita identificar direta ou indiretamente um indivíduo.

Isso inclui:

- CPF
- telefone
- e-mail
- endereço
- nomes completos
- dados bancários

- documentos

O objetivo aqui não é apagar completamente o feedback, mas substituir essas informações por marcadores neutros.

Dessa forma preservamos a estrutura da mensagem sem expor dados sensíveis.

Por exemplo, imagine um usuário relatando:

“Fui fazer um PIX e o aplicativo travou. Meu CPF é...”

Do ponto de vista técnico, o CPF não agrega nenhum valor para análise do bug. O que importa é entender o comportamento do sistema durante a operação.

Nesse caso, a IA remove a informação sensível mantendo intacto o contexto do problema.

Esse detalhe é extremamente importante porque preserva valor analítico do dataset.

Outro ponto relevante é remoção de ruído.

Datasets reais possuem enorme quantidade de informações inúteis para análise de produto.

É muito comum encontrar:

- mensagens de teste
- automações
- bots
- spam
- reclamações fora do escopo
- comentários sem relação com software

Por exemplo, um usuário pode reclamar do atendimento presencial da agência dentro do review do aplicativo mobile.

Embora isso seja relevante para empresa em algum contexto, não necessariamente ajuda o time de engenharia a melhorar o produto digital.

Da mesma forma, mensagens automatizadas ou testes internos não agregam valor para análise funcional.

A IA consegue identificar esse tipo de ruído semanticamente.

Isso representa uma vantagem enorme em comparação com filtros tradicionais puramente sintáticos.

Outro aspecto extremamente importante é preservação do contexto técnico.

Esse talvez seja um dos maiores desafios da sanitização.

Queremos remover ruído sem destruir informação útil.

Por exemplo, se um usuário disser:

“O aplicativo trava quando tento agendar um PIX no dia 31 usando iPhone.”

Esse comentário possui várias informações extremamente valiosas:

- ação executada
- contexto funcional
- edge case de calendário
- possível bug de validação
- dispositivo utilizado

Esses detalhes precisam ser preservados.

Da mesma forma, comentários sobre UX também possuem enorme valor analítico.

Muitas vezes usuários não descrevem bugs técnicos, mas relatam dificuldades de navegação, excesso de cliques, problemas de descoberta de funcionalidade ou frustração na experiência.

Tudo isso ajuda times de produto e engenharia a evoluírem a aplicação.

Quando utilizamos IA corretamente, conseguimos separar ruído de sinal útil com muito mais eficiência.

E existe outro ganho extremamente importante: estruturação dos dados.

Não queremos apenas texto limpo.

Queremos um dataset pronto para processamento posterior.

Por isso normalmente orientamos a IA a devolver os resultados em formato estruturado, como JSON.

Esse detalhe transforma completamente o pipeline de análise.

Quando os dados já saem estruturados, conseguimos alimentar facilmente:

- sistemas de analytics
- dashboards
- ferramentas de BI
- pipelines de IA
- geração automática de tickets
- classificação de bugs

- agrupamento de problemas
- análise de sentimento

Além disso, podemos anonimizar identificadores de usuário mantendo rastreabilidade estatística sem expor identidade real.

Esse ponto é muito importante dentro de LGPD e boas práticas de segurança.

Na prática, conseguimos analisar comportamento coletivo sem necessidade de armazenar dados pessoais reais.

Outro aspecto extremamente interessante é perceber como a IA está funcionando aqui como camada intermediária de preparação semântica.

Ela não substitui completamente engenharia de dados tradicional.

Isso é muito importante entender.

Para datasets grandes, pipelines complexos e processamento massivo, ainda continuamos precisando de:

- engenharia de dados
- governança
- validação
- observabilidade
- compliance
- segurança
- arquitetura de dados

A IA entra como acelerador inteligente dentro desse fluxo.

Ela reduz trabalho operacional e melhora qualidade da preparação inicial dos dados.

Esse talvez seja um dos maiores padrões que começam a aparecer ao longo de toda engenharia moderna assistida por IA.

A inteligência artificial não elimina especializações técnicas.

Ela potencializa produtividade de especialistas.

O engenheiro de dados continua sendo fundamental. O analista continua sendo fundamental. O arquiteto continua sendo fundamental.

A diferença é que agora conseguimos automatizar tarefas extremamente demoradas utilizando capacidade semântica dos modelos.

Outro ponto importante é que datasets sanitizados criam base para novas automações.

Depois de limpar os feedbacks, podemos começar a utilizar IA para:

- identificar padrões recorrentes
- agrupar bugs semelhantes
- gerar tickets automaticamente
- sugerir melhorias de UX
- detectar regressões
- priorizar problemas
- classificar severidade
- analisar satisfação

Perceba como a sanitização funciona como primeira etapa de uma cadeia muito maior.

Sem dados limpos, todas as análises posteriores perdem qualidade.

Esse conceito é extremamente importante em qualquer projeto envolvendo IA ou analytics.

A qualidade da saída depende diretamente da qualidade da entrada.

Se o dataset estiver contaminado com ruído, inconsistência ou dados irrelevantes, os resultados analíticos também serão ruins.

Por isso preparação de dados continua sendo uma das etapas mais importantes de qualquer pipeline moderno.

E talvez o ponto mais interessante de tudo isso seja perceber que estamos aproximando engenharia de software, UX, analytics e IA dentro do mesmo fluxo de trabalho.

O feedback do usuário deixa de ser apenas reclamação operacional e passa a funcionar como matéria-prima para evolução contínua do produto.

Isso cria um ciclo extremamente poderoso:

- usuários utilizam o sistema
- feedbacks são coletados
- dados são sanitizados
- IA ajuda na análise
- melhorias são identificadas
- novos requisitos surgem
- produto evolui

Esse tipo de abordagem transforma completamente maturidade do desenvolvimento orientado ao usuário.

Capítulo 5 - Laboratório: Structured Prompts no AI Studio

À medida que sistemas entram em produção e começam a receber usuários reais, uma das maiores dificuldades dos times técnicos deixa de ser apenas implementar funcionalidades novas. O grande desafio passa a ser interpretar corretamente o volume enorme de feedbacks gerados pelos próprios usuários da aplicação.

Todo produto em produção gera sinais constantemente.

Esses sinais aparecem em reviews de loja, tickets de suporte, pesquisas de satisfação, comentários internos, métricas de comportamento e relatos enviados pelos próprios clientes. Em teoria, isso é excelente, porque significa que temos uma fonte contínua de informação sobre problemas reais do sistema.

Na prática, porém, isso rapidamente se transforma em caos operacional.

Os times começam a receber centenas ou milhares de reclamações diferentes. Algumas são duplicadas. Outras são vagas. Muitas descrevem sintomas sem explicar o problema real. Existem feedbacks emocionais, mensagens incompletas, relatos técnicos, problemas de UX e solicitações de novas funcionalidades tudo misturado no mesmo dataset.

E normalmente surge a mesma pergunta:

“Por onde começamos?”

Esse é um dos pontos mais difíceis dentro da gestão de produto moderna.

Não basta apenas ler feedbacks manualmente. Precisamos transformar dados brutos em backlog acionável.

Ou seja, precisamos converter reclamações difusas em informação estruturada que ajude engenharia, UX e produto a tomarem decisões.

É justamente aqui que structured prompts e LLMs começam a gerar um impacto extremamente relevante.

Nas aulas anteriores, utilizamos IA para:

- refinar requisitos
- mapear edge cases
- gerar documentação visual
- criar UX Writing
- sanitizar datasets

Agora damos mais um passo importante: transformar feedback sanitizado em backlog técnico priorizado.

Esse processo é extremamente interessante porque conecta diretamente experiência do usuário com evolução contínua do produto.

O feedback deixa de ser apenas comentário perdido em plataforma de suporte e passa a alimentar o pipeline real de engenharia.

Mas para isso funcionar corretamente, novamente precisamos estruturar muito bem o contexto da IA.

E aqui reaparece um dos conceitos mais importantes de todo uso profissional de LLMs: System Prompt estruturado.

Existe uma diferença enorme entre perguntar algo de maneira genérica para um modelo e fornecer um contexto profissional completo.

Quando apenas jogamos informações no chat sem direção clara, normalmente recebemos respostas superficiais ou inconsistentes.

Por outro lado, quando estruturamos:

- papel
- objetivo
- regras
- formato de saída

a qualidade do resultado muda completamente.

Esse padrão começa a aparecer repetidamente em todos os fluxos modernos envolvendo IA generativa.

O modelo precisa entender exatamente:

- quem ele deve representar
- qual problema precisa resolver
- quais critérios deve utilizar
- como deve organizar a saída

Nesse laboratório, o modelo passa a atuar como Tech Lead e Product Manager de um aplicativo financeiro.

Esse detalhe é extremamente importante.

Quando damos esse papel ao modelo, ele começa a interpretar os feedbacks sob perspectiva de produto e engenharia simultaneamente.

Ele deixa de apenas resumir comentários e passa a pensar em:

- impacto técnico

- criticidade
- experiência do usuário
- priorização
- backlog
- melhoria contínua

Esse tipo de direcionamento transforma completamente utilidade prática da IA.

O objetivo agora não é limpar dados, mas analisar feedbacks sanitizados e transformá-los em backlog priorizado de engenharia e design.

Perceba como estamos construindo um fluxo progressivo de maturidade.

Primeiro refinamos requisitos.

Depois documentamos jornadas.

Depois estruturamos comunicação.

Depois limpamos datasets.

Agora utilizamos os próprios dados limpos para alimentar evolução do produto.

Isso cria um ciclo extremamente poderoso dentro da engenharia moderna orientada por feedback contínuo.

Outro ponto muito importante aqui é a definição das regras de classificação.

A IA precisa entender como categorizar os problemas.

Nem todo feedback representa o mesmo nível de impacto.

Alguns comentários indicam bugs críticos. Outros representam melhorias de UX. Outros sugerem novas funcionalidades.

Sem classificação adequada, o backlog rapidamente vira uma lista caótica de tarefas sem prioridade clara.

Por isso definimos categorias explícitas como:

- bug crítico
- melhoria de interface
- nova funcionalidade

Além disso, também definimos severidade.

Esse detalhe é extremamente importante para priorização.

Quando um usuário relata crash ou tela branca, por exemplo, estamos diante de um problema de alta severidade.

Tela branca em aplicações front-end é sempre um sinal muito grave.

Mesmo que o erro interno aconteça inevitavelmente em algum cenário extremo, a interface nunca deveria simplesmente quebrar sem tratamento.

Esse é um princípio importante dentro da engenharia front-end moderna.

Erros podem acontecer.

O que não pode acontecer é ausência de tratamento visual para esses erros.

Uma aplicação madura precisa possuir mecanismos de fallback.

O usuário precisa receber:

- feedback visual
- mensagem clara
- estado de erro controlado
- possibilidade de recuperação

Quando a interface simplesmente fica branca, o sistema transmite sensação de falha total e perda de controle.

Isso impacta diretamente confiança do usuário no produto.

E é interessante perceber como a IA começa a identificar esse tipo de problema automaticamente durante análise dos feedbacks.

Outro ponto extremamente poderoso dos structured prompts é a geração de ações propostas.

Esse talvez seja um dos maiores diferenciais do processo.

Não queremos apenas detectar problemas.

Queremos gerar direcionamento técnico inicial para resolução.

Por exemplo, ao detectar tela branca durante transação PIX, a IA pode sugerir:

- investigar logs
- tratar exceções não capturadas
- adicionar error boundaries
- melhorar fallback visual
- revisar timeout da integração

Perceba que o modelo começa a atuar quase como apoio técnico inicial para triagem de backlog.

Isso não substitui arquitetura nem debugging real, mas acelera muito a análise inicial dos problemas.

Outro exemplo interessante envolve problemas de calendário.

Quando usuários relatam falhas relacionadas a datas inexistentes, como agendamento para dia 31 em meses incompatíveis, a IA consegue identificar tanto o problema de UX quanto o problema técnico.

Nesse caso, ela pode sugerir:

- validação no DatePicker
- restrição de datas inválidas
- tratamento de exceção no back-end
- melhoria de validação no front-end

Esse tipo de análise integrada é extremamente valioso porque conecta experiência do usuário com engenharia de software.

Muitas vezes os usuários não descrevem tecnicamente o problema.

Eles apenas relatam sintomas.

A IA ajuda justamente a transformar sintomas em hipóteses estruturadas de melhoria.

Outro aspecto muito importante é agrupamento semântico.

Em datasets reais, diversos usuários frequentemente reclamam do mesmo problema utilizando palavras completamente diferentes.

Uma pessoa diz que o aplicativo travou.

Outra diz que ficou carregando infinitamente.

Outra diz que apareceu tela branca.

Outra relata fechamento inesperado.

Embora os textos sejam diferentes, semanticamente eles podem representar o mesmo bug.

Modelos de linguagem conseguem identificar essas similaridades muito melhor do que filtros tradicionais baseados apenas em palavras-chave.

Isso permite consolidar problemas repetidos em tickets únicos mais organizados.

Esse detalhe melhora enormemente qualidade do backlog.

Em vez de dezenas de tickets duplicados, o time passa a trabalhar em problemas agrupados semanticamente.

Isso facilita:

- priorização

- planejamento
- análise de impacto
- gestão de sprint
- tomada de decisão

Outro ponto extremamente importante é o formato estruturado de saída.

Mais uma vez, não queremos apenas texto solto.

Queremos objetos estruturados que possam ser reutilizados diretamente em pipelines reais.

Quando pedimos saída em JSON contendo:

- ID do ticket
- categoria
- severidade
- resumo
- ação proposta

estamos aproximando IA diretamente do fluxo operacional do produto.

Esse detalhe é muito poderoso.

Na prática, conseguimos integrar esses resultados com:

- Jira
- sistemas internos
- plataformas de backlog
- automações
- dashboards
- ferramentas de analytics

A IA deixa de ser apenas ferramenta de conversa e passa a funcionar como componente operacional dentro da engenharia.

Esse talvez seja um dos pontos mais importantes da maturidade no uso de IA generativa.

O valor real não está apenas em “conversar com o modelo”.

O valor aparece quando transformamos respostas em ativos reutilizáveis dentro do fluxo técnico da empresa.

Outro aspecto extremamente relevante é a janela de contexto.

Modelos modernos conseguem trabalhar com volumes muito grandes de informação simultaneamente.

Isso permite analisar enormes quantidades de feedbacks dentro da mesma sessão.

Quanto maior o contexto analisado, melhor tende a ser percepção de padrões coletivos.

Isso abre possibilidades extremamente interessantes para:

- detecção de regressões
- tendências de UX
- bugs recorrentes
- funcionalidades desejadas
- problemas sazonais
- comportamento de usuários

Perceba como começamos a transformar IA em mecanismo contínuo de inteligência de produto.

E novamente é importante reforçar: isso não substitui Product Managers, Tech Leads ou analistas.

A IA funciona como acelerador de produtividade.

Ela reduz trabalho operacional pesado e permite que profissionais especializados dediquem mais tempo às decisões estratégicas.

O papel humano continua essencial para:

- validação
- priorização final
- entendimento de negócio
- alinhamento organizacional
- análise arquitetural
- tomada de decisão

A diferença é que agora conseguimos transformar grandes volumes de informação bruta em backlog estruturado com muito mais velocidade.

Esse tipo de abordagem muda completamente maturidade do desenvolvimento orientado a produto.

O sistema deixa de evoluir apenas baseado em suposições internas e passa a evoluir continuamente alimentado pelo comportamento real dos usuários.

Capítulo 6 - Prompt as Code & Integração

Nas últimas aulas, nós geramos muito valor para o processo de desenvolvimento. Utilizamos inteligência artificial para refinar requisitos, mapear edge cases, gerar documentação de jornada, criar mensagens de UX Writing, sanitizar feedbacks de usuários e transformar esses dados em backlog priorizado de produto. Cada uma dessas etapas mostrou como uma LLM pode funcionar como acelerador de produtividade para times técnicos, produto, UX e qualidade.

Mas existe um ponto muito importante que diferencia um uso experimental de IA de um uso profissional dentro de engenharia de software. Um profissional sênior não pode tratar tudo isso como algo perdido no histórico de uma conversa.

Se um prompt gera valor, ele precisa ser tratado como ativo do projeto.

E se ele é um ativo do projeto, ele precisa ser organizado, versionado, documentado e compartilhado da mesma forma que qualquer outro artefato técnico importante.

Esse é o conceito central desta aula: Prompt as Code.

Quando falamos em Prompt as Code, estamos dizendo que prompts não devem ser tratados como textos descartáveis. Eles precisam ser gerenciados com a mesma disciplina que aplicamos ao código-fonte, à documentação técnica, aos arquivos de configuração, aos contratos de API e aos scripts de automação.

Isso significa armazenar os prompts em um repositório, organizar diretórios, criar nomes claros, versionar alterações, documentar propósito e permitir que outras pessoas do time consigam reutilizar aquele material.

Esse cuidado é fundamental porque prompts bem construídos carregam conhecimento técnico da equipe.

Um System Prompt que orienta a IA a atuar como analista de requisitos, por exemplo, não é apenas uma instrução solta. Ele representa uma forma estruturada de analisar requisitos dentro daquele contexto de negócio.

Um prompt de UX Writing representa diretrizes de comunicação, tom de voz, termos permitidos e padrões de resposta da interface.

Um prompt de sanitização de dados representa regras de compliance, LGPD, preservação de contexto técnico e remoção de ruído.

Um prompt de Product Manager representa critérios de classificação, severidade, priorização e transformação de feedbacks em backlog.

Ou seja, cada prompt contém decisões técnicas, decisões de produto e padrões operacionais que podem e devem ser reutilizados.

Se esse material fica apenas dentro do histórico da ferramenta de IA, ele se perde com facilidade. Outra pessoa não consegue encontrar. O time não consegue revisar. Não existe rastreabilidade. Não existe versionamento. Não existe governança.

Por isso, a primeira etapa é tirar os prompts da ferramenta e colocá-los no repositório.

Na prática, isso significa copiar os System Prompts criados ao longo das aulas e salvá-los em arquivos separados. Cada arquivo deve ter um nome que indique claramente seu propósito. Um prompt para análise de requisitos deve ficar separado de um prompt para UX Writing. Um prompt para sanitização de dados deve ficar separado de um prompt para geração de backlog.

Essa organização melhora muito a reutilização.

Quando outro desenvolvedor precisar refinar um requisito, ele poderá ir diretamente na pasta de prompts, encontrar o arquivo correto e executá-lo no motor de LLM utilizado pela equipe.

Isso também evita que cada pessoa recrie prompts diferentes para resolver o mesmo problema.

Com o tempo, esse repositório se transforma em uma biblioteca interna de inteligência operacional da equipe.

Outro ponto importante é a estrutura de pastas.

Um projeto minimamente organizado precisa separar claramente o que é dado bruto, o que é prompt, o que é resultado processado e o que é documentação.

Por exemplo, dados originais de feedback podem ficar em uma pasta de data. Prompts podem ficar em uma pasta específica de prompts. Resultados gerados, como backlog, mensagens JSON, dados sanitizados e relatórios, podem ficar em uma pasta de reports ou results. A documentação principal fica no README.

Essa separação parece simples, mas transmite maturidade técnica.

Quando alguém abre o repositório, precisa entender rapidamente o propósito do projeto e o papel de cada arquivo.

Isso vale tanto para projetos internos de empresa quanto para projetos de portfólio no GitHub.

E esse é um ponto que eu considero muito importante.

Quando estamos estudando, criando laboratórios e publicando projetos, o GitHub passa a funcionar como uma vitrine técnica. Recrutadores, tech leads e outros desenvolvedores

podem avaliar não apenas se você sabe usar uma ferramenta, mas se você sabe organizar uma solução de forma profissional.

Um projeto com bons prompts, dados organizados, resultados claros e README bem escrito transmite muito mais maturidade do que um repositório com arquivos soltos.

Por isso, a documentação do repositório também precisa receber atenção.

E aqui novamente podemos utilizar IA.

Depois de organizar os arquivos, podemos gerar um README profissional com apoio de um meta prompt.

Um meta prompt é um prompt utilizado para produzir outro artefato técnico a partir do próprio projeto. Nesse caso, pedimos para a IA analisar a estrutura do repositório e gerar uma documentação completa explicando o objetivo, o pipeline, os arquivos, a forma de uso e as tecnologias envolvidas.

Esse README precisa explicar claramente o que o projeto faz.

Ele deve apresentar a ideia central, mostrar como os prompts são utilizados, descrever a arquitetura do pipeline e orientar outro desenvolvedor a clonar o repositório e executar os mesmos fluxos no motor de LLM de sua preferência.

Também é importante mencionar a stack utilizada. Neste projeto, trabalhamos com conceitos como prompt engineering, Google AI Studio, Gemini, Markdown, JSON, Mermaid.js e sanitização de dados.

O objetivo do README não é apenas enfeitar o repositório. Ele funciona como porta de entrada para qualquer pessoa que queira entender ou reutilizar o projeto.

Outro detalhe importante é o formato da saída quando pedimos para a IA gerar documentação.

Se queremos copiar diretamente o resultado para um arquivo README, precisamos orientar o modelo a retornar apenas Markdown válido. Não queremos que ele escreva frases como “aqui está o seu README” ou “segue abaixo a documentação”. Esse tipo de texto pode atrapalhar quando copiamos diretamente para o arquivo.

Esse cuidado com o formato de saída aparece em várias etapas do trabalho com IA.

Sempre que queremos integrar a resposta da LLM a um fluxo técnico, precisamos ser explícitos sobre o formato esperado.

Isso vale para JSON, Markdown, Mermaid, YAML, documentação, prompts e qualquer outro artefato.

Outro aprendizado importante da aula é o cuidado com o System Prompt ativo.

Durante a demonstração, ao tentar gerar o README, o modelo inicialmente respondeu no formato errado porque ainda estava usando um System Prompt anterior, voltado para análise de backlog. Isso é um erro muito comum quando reutilizamos a mesma conversa ou o mesmo ambiente de IA.

O modelo continua obedecendo ao contexto configurado.

Se o System Prompt ativo diz que ele deve atuar como Product Manager e retornar JSON, ele pode tentar aplicar essa regra mesmo quando a nova tarefa é completamente diferente.

Esse comportamento mostra como System Prompts são poderosos, mas também exigem disciplina.

Antes de mudar de tarefa, precisamos validar se o contexto ativo ainda faz sentido. Caso contrário, devemos limpar o contexto, remover o System Prompt anterior ou criar uma nova conversa.

Esse cuidado evita resultados inconsistentes e reduz perda de tempo.

No uso profissional de IA, contexto é parte da arquitetura.

Não basta saber escrever prompts isolados. Precisamos saber quando reaproveitar contexto, quando trocar contexto e quando limpar completamente a sessão.

Esse é um dos pontos mais importantes para trabalhar com LLMs de forma confiável.

Também é importante entender que a integração com URLs ou repositórios pode exigir configurações específicas dependendo da ferramenta utilizada. No caso do AI Studio, existe a possibilidade de habilitar contexto de URL para que o modelo consiga acessar e analisar um repositório informado.

Em outras ferramentas, pode ser necessário habilitar browsing, conectar uma integração, carregar os arquivos manualmente ou fornecer a árvore do projeto dentro do prompt.

O conceito permanece o mesmo: a IA precisa receber contexto suficiente para gerar uma documentação correta.

E, como sempre, a validação humana continua sendo indispensável.

A IA pode gerar um README excelente, mas ainda precisamos revisar se os nomes dos arquivos estão corretos, se a descrição bate com o projeto real, se o fluxo foi interpretado adequadamente e se não existem informações inventadas.

Esse ponto é essencial.

Trabalhar profissionalmente com IA não significa aceitar cegamente tudo que o modelo retorna. Significa usar a IA para acelerar trabalho técnico e aplicar revisão humana qualificada no resultado.

Quando tratamos prompts como código, começamos a criar uma base reutilizável de produtividade para o time. Cada prompt versionado pode ser melhorado, revisado e padronizado. Cada alteração pode ser discutida. Cada evolução pode ser registrada.

Isso permite criar uma espécie de biblioteca interna de agentes, papéis e tarefas recorrentes.

Com o tempo, a equipe pode ter prompts padronizados para:

refinamento de requisitos, análise de edge cases, revisão de UX Writing, geração de casos de teste, sanitização de dados, classificação de feedbacks, geração de documentação, análise de pull requests, criação de diagramas e preparação de backlog.

Esse tipo de biblioteca gera escala.

Em vez de cada pessoa depender de improviso, o time passa a trabalhar com padrões compartilhados.

E isso é exatamente o que diferencia um uso amador de IA de um uso profissional.

IA não deve ser apenas uma ferramenta individual de produtividade. Ela pode se tornar parte do processo de engenharia da equipe.

Mas, para isso, precisamos organizar, versionar e documentar.

Prompt as Code é justamente essa disciplina aplicada aos artefatos gerados e utilizados com LLMs.

Quando fazemos isso corretamente, transformamos prompts em patrimônio técnico da organização.

Módulo 02 - O Ecossistema AI-Native no Frontend

Capítulo 1 - Setup Angular e MCP

Neste momento do módulo, começamos a mudar um pouco o tipo de trabalho que estamos fazendo. Até aqui, usamos inteligência artificial para apoiar atividades mais ligadas ao refinamento, documentação, comunicação e análise de produto. Refinamos requisitos,

levantamos edge cases, criamos diagramas, estruturamos mensagens de interface, sanitizamos dados e transformamos feedbacks de usuários em backlog.

Agora começamos a entrar no desenvolvimento front-end propriamente dito.

A ideia passa a ser colocar a mão no código e entender como as ferramentas de IA podem ajudar no fluxo real de desenvolvimento de interfaces, componentes e jornadas. Mas, antes de começar a construir telas, precisamos preparar o ambiente corretamente.

Esse ponto é importante porque existe uma diferença grande entre simplesmente pedir para uma IA gerar código e configurar um ambiente de desenvolvimento para que ela consiga trabalhar com mais precisão.

Nesta etapa, vamos usar um projeto Angular como base. Eu vou utilizar Angular porque é um framework bastante consolidado, com uma CLI madura, estrutura organizada e muitos recursos interessantes para construção de aplicações front-end modernas. Mas o conceito que estamos trabalhando aqui não depende exclusivamente do Angular.

O mesmo raciocínio poderia ser aplicado em React, Vue ou qualquer outro framework web.

O mais importante é entender como preparar o ambiente, como configurar a IDE assistida por IA e como garantir que o agente utilize boas práticas atualizadas do framework escolhido.

Nas aulas anteriores, trabalhamos bastante com o Google AI Studio. Agora vamos mudar o instrumental e utilizar uma IDE orientada a IA, chamada Antigravity.

A ideia dessa ferramenta é trabalhar com uma filosofia AI First. Ela lembra bastante o Visual Studio Code, o Cursor e outras IDEs modernas com chat integrado. A diferença é que esse tipo de ambiente permite controlar agentes de codificação, executar tarefas no projeto, analisar arquivos e aplicar alterações diretamente no código.

Esse é um ponto importante do desenvolvimento moderno com IA.

A IA deixa de ser apenas uma ferramenta externa de conversa e passa a atuar dentro do próprio ambiente de desenvolvimento.

Isso muda bastante o fluxo de trabalho.

Em vez de copiar e colar trechos de código entre o chat e o projeto, passamos a interagir com um agente que enxerga o workspace, entende a estrutura dos arquivos, propõe alterações, cria componentes, modifica rotas e executa tarefas.

Mas isso também exige mais responsabilidade.

Quando damos acesso ao projeto para um agente, precisamos saber orientar bem o que ele deve fazer. Precisamos definir escopo, restrições, padrões e objetivos. Caso contrário,

ele pode tomar decisões que não queremos, gerar estilos antes da hora, alterar arquivos demais ou criar uma estrutura diferente da esperada.

Por isso, antes de qualquer implementação, começamos criando o projeto Angular.

O Angular possui uma ferramenta de linha de comando, a Angular CLI, que permite criar projetos, componentes, serviços, rotas e outras estruturas do framework. Mesmo que ao longo do curso a gente não utilize o CLI para tudo, ele continua sendo uma forma muito prática de iniciar o projeto com a estrutura oficial.

No exemplo da aula, criamos um projeto Angular chamado PixApp, utilizando CSS puro e habilitando roteamento. Também deixamos claro que a versão utilizada no momento da gravação era a versão 21, mas esse detalhe pode mudar com o tempo. O ponto principal não é decorar a versão, e sim entender o processo.

Depois que o projeto é criado, abrimos a pasta dentro do Antigravity.

Ao abrir o workspace, percebemos que a interface é muito parecida com o Visual Studio Code. Isso acontece porque o Antigravity funciona como um fork do VS Code. Por isso, muitos plugins e extensões do próprio VS Code também podem ser utilizados nele.

Esse detalhe é interessante porque reduz a curva de aprendizado. Para quem já trabalha com VS Code, a navegação, terminal, árvore de arquivos e extensões são familiares.

Depois de abrir o projeto, subimos a aplicação Angular localmente e verificamos a página inicial padrão gerada pelo framework.

Até aqui, temos apenas um projeto limpo e funcionando.

Mas ainda falta uma parte muito importante: preparar a IA para trabalhar corretamente com Angular.

Esse é um dos pontos mais relevantes da aula.

Modelos de linguagem são treinados em grandes bases de dados, mas essas bases possuem uma janela temporal. Mesmo modelos modernos podem não ter sempre as práticas mais recentes de um framework específico.

E frameworks web evoluem rapidamente.

Angular, React, Vue, Next.js, Spring, Quarkus, Micronaut e praticamente qualquer tecnologia moderna passam por mudanças frequentes. Novas APIs surgem, padrões antigos deixam de ser recomendados, bibliotecas mudam e boas práticas evoluem.

Então, quando usamos IA para gerar código, existe um risco real de o modelo sugerir padrões antigos.

No caso do Angular, por exemplo, isso pode aparecer em uso inadequado de módulos, sintaxe desatualizada, padrões antigos de roteamento ou desconhecimento de recursos mais recentes como standalone components e signals.

Por isso, não basta confiar apenas no conhecimento prévio do modelo.

Precisamos direcionar o agente para a documentação oficial e para as boas práticas atualizadas.

É aqui que entra o MCP.

MCP significa Model Context Protocol. De forma prática, podemos entender MCP como uma forma de conectar agentes de IA a servidores especializados que fornecem ferramentas e contexto atualizado.

No caso da aula, usamos o MCP do Angular.

A equipe do Angular disponibiliza um servidor MCP que permite ao agente acessar documentação, recomendações e boas práticas atualizadas do framework. Isso ajuda muito a reduzir o risco de gerar código baseado em padrões antigos.

Esse conceito é extremamente importante.

Quando configuramos MCP, deixamos de trabalhar apenas com o conhecimento interno do modelo e passamos a dar ao agente uma ponte com uma fonte especializada.

Isso torna o desenvolvimento assistido por IA muito mais confiável.

No fluxo da aula, usamos um comando do próprio Angular para obter a configuração necessária do MCP. Esse comando retorna um JSON que deve ser adicionado à configuração da IDE ou do agente.

No Antigravity, essa configuração é feita na área de agentes, dentro da seção de MCP Servers. A partir do momento em que registramos o servidor MCP do Angular, o agente passa a ter acesso às ferramentas fornecidas por esse servidor.

Na prática, isso significa que, ao criar componentes, rotas ou estruturas Angular, o agente pode consultar a fonte oficial atualizada antes de propor alterações.

Esse é um cuidado que precisamos trazer para qualquer stack.

Se estou trabalhando com Angular, quero usar a documentação atualizada do Angular. Se estou trabalhando com React, quero direcionar para boas práticas modernas do React. Se estou trabalhando com back-end Java, quero orientar o agente para Spring, Quarkus ou qualquer stack específica que esteja sendo usada.

IA em desenvolvimento não é apenas escrever prompt.

É configurar contexto, ferramentas e fontes confiáveis para que o agente trabalhe com consistência.

Depois que o ambiente está configurado, podemos iniciar a primeira interação real com o projeto.

Mas, mesmo aqui, precisamos ser cuidadosos.

O objetivo da primeira tarefa não é criar uma interface bonita ou finalizada. O objetivo é criar o esqueleto funcional da aplicação.

Por isso, o prompt precisa deixar claro o papel do agente, as tarefas esperadas e as restrições.

A estrutura básica continua seguindo o padrão que vimos nas aulas anteriores: papel, objetivo, regras e saída esperada.

No caso da implementação inicial, pedimos para o agente atuar como engenheiro de front-end sênior especialista em Angular 21. Em seguida, definimos tarefas objetivas: limpar o conteúdo padrão gerado pela CLI, criar uma estrutura semântica simples com navegação, menu lateral e área principal, configurar uma rota para PIX, criar um componente standalone para transferência e montar um formulário inicial.

Também definimos uma restrição essencial: não gerar CSS e não aplicar estilo inline.

Esse ponto é muito importante porque a IA tende a tentar “melhorar” a entrega, adicionando estilos, classes e decisões visuais por conta própria. Mas, neste momento, não queremos isso. Queremos apenas HTML puro e cru, porque a estilização será feita depois.

Essa capacidade de restringir escopo é uma das habilidades mais importantes no trabalho com agentes.

Quanto mais claro for o limite da tarefa, melhor será a qualidade da entrega.

Outro detalhe interessante é que o Antigravity trabalha com planejamento. Antes de aplicar as mudanças, o agente pode criar um plano de execução. Ele identifica os arquivos que serão alterados, descreve o que pretende fazer e aguarda aprovação.

Esse fluxo é muito importante porque mantém o desenvolvedor no controle.

A IA propõe, mas o desenvolvedor revisa.

Esse é o modelo ideal de trabalho com agentes de codificação.

Não queremos um processo totalmente automático em que a IA modifica o projeto sem validação. Queremos um copiloto que acelere o trabalho, mas que ainda permita revisão, aprovação e correção humana.

Depois da aprovação, o agente cria os arquivos necessários, configura a rota, monta o componente e ajusta o layout base da aplicação.

Ao final, temos uma primeira versão simples do PixApp: um menu lateral com item PIX, uma rota configurada e um formulário inicial com campos como chave PIX, valor, data de agendamento e botão de confirmação.

Ainda não temos estilização, ainda não temos componentes finais, ainda não temos integração real. Mas temos a base do projeto funcionando.

E isso é exatamente o que queríamos nesta etapa.

O objetivo foi preparar o ambiente, configurar a IDE assistida por IA, conectar o MCP do Angular e realizar a primeira geração controlada de estrutura front-end.

Esse fluxo mostra muito bem como o desenvolvimento com IA precisa ser conduzido.

Primeiro, criamos um projeto limpo. Depois, configuramos o agente. Em seguida, conectamos fontes atualizadas de contexto por meio do MCP. Só então começamos a pedir alterações no código.

Essa ordem é importante porque aumenta consistência e reduz retrabalho.

Quando tentamos pular direto para geração de código sem preparar o ambiente, aumentamos o risco de receber uma solução desatualizada, desorganizada ou desalinhada com o framework.

Quando configuramos tudo corretamente, a IA passa a trabalhar como parte integrada do ambiente de engenharia.

E esse é o ponto central desta aula: antes de usar IA para codar, precisamos preparar corretamente o contexto em que ela vai trabalhar.

Capítulo 2 - Criação de Tokens, Estilização

Na última aula, nós configuramos o projeto Angular, preparamos a IDE, configuramos o Antigravity e conectamos o MCP do time do Angular para garantir uma interação mais assertiva entre o nosso agente e o framework. Esse ponto é importante porque, quando trabalhamos com IA aplicada ao desenvolvimento, não queremos apenas pedir código de

forma genérica. Queremos que o agente trabalhe com contexto, boas práticas e estrutura compatível com a tecnologia que estamos usando.

Agora, o próximo passo é começar a dar identidade visual para a aplicação. Na aula anterior, criamos uma estrutura inicial bem simples, sem estilização. A aplicação tinha navegação, rota e um formulário básico para o fluxo de PIX, mas ainda não tinha uma camada visual definida. E antes de sair aplicando CSS diretamente em cada componente, precisamos parar e pensar em design system.

Quando falamos em tokens de design, não estamos falando dos tokens de LLM que vimos em outros contextos. Aqui o conceito vem de UX e design system. Um token de design é a menor unidade reutilizável do sistema visual. Ele pode representar uma cor, uma fonte, um espaçamento, um raio de borda, uma sombra ou qualquer outro valor fundamental que compõe a identidade visual da aplicação.

Esse conceito é extremamente importante porque evita inconsistência. Em uma aplicação real, principalmente quando várias pessoas ou equipes trabalham no mesmo produto, não podemos permitir que cada tela tenha um tom diferente de azul, um vermelho diferente para erro ou espaçamentos definidos de forma aleatória. Isso cria uma experiência visual fragmentada e passa sensação de falta de maturidade no produto.

Por isso, antes de estilizar componentes individualmente, criamos uma base centralizada de tokens. No nosso caso, vamos utilizar variáveis nativas de CSS dentro do arquivo global de estilos do Angular. A ideia é transformar o briefing de marca em valores reutilizáveis que poderão ser consumidos por todos os componentes da aplicação.

O briefing que recebemos simula uma demanda comum de produto. A nova marca do PixApp precisa transmitir confiança e modernidade. A cor principal será um azul noturno profundo, a cor de destaque será um verde neon vibrante, os erros usarão vermelho clássico, o modo claro terá fundo quase branco e o modo escuro terá fundo chumbo. A fonte principal para títulos será Montserrat, enquanto os textos corridos usarão Inter. Também definimos que os espaçamentos devem seguir múltiplos de quatro, como oito, dezesseis, vinte e quatro e trinta e dois pixels.

Um time menos experiente poderia pegar esse briefing e sair aplicando valores diretamente em cada componente. Isso até funciona no começo, mas rapidamente se transforma em um problema. Se amanhã a marca mudar, se o verde virar roxo ou se a fonte for substituída, será necessário procurar cada ocorrência espalhada no projeto. Esse tipo de acoplamento visual dificulta manutenção e aumenta o risco de inconsistência.

A abordagem correta é criar uma camada central de tokens. Assim, os componentes não precisam conhecer valores absolutos. Eles consomem variáveis semânticas. Esse detalhe é muito importante. Em vez de nomear uma variável pelo nome da cor, como “azul” ou “verde”, é melhor usar nomes ligados à função daquela cor no sistema. Por exemplo, cor

primária, cor de destaque, cor de erro, cor de fundo e cor de texto. Dessa forma, se a cor primária mudar de azul para outro tom no futuro, o significado da variável continua correto.

Aqui entramos novamente no uso profissional de prompts. Em vez de escrever o comando diretamente no chat, criamos um arquivo de prompt dentro do próprio projeto. Essa é uma prática muito interessante porque transforma o prompt em parte da base técnica. O arquivo pode ser versionado, revisado e reutilizado posteriormente. Esse fluxo se conecta diretamente ao conceito de Prompt as Code que trabalhamos anteriormente.

Nesse prompt, definimos que o agente deve atuar como um design system engineer sênior. O objetivo é converter um briefing textual de marca em uma base estruturada de tokens de design usando variáveis nativas de CSS puro. Também definimos diretrizes claras: usar nomenclatura semântica, criar escala de tipografia, estruturar variáveis dentro do seletor raiz e preparar o sistema para suportar troca de tema.

Esse formato de prompt segue a mesma estrutura que viemos utilizando no módulo inteiro: papel, objetivo, regras e formato de saída. Essa consistência é importante porque reduz ambiguidade. Quanto mais claro for o escopo, menor a chance de o agente gerar algo desalinhado com a intenção do projeto.

Depois disso, usamos o agente para ler tanto o briefing quanto o prompt de geração de tokens e escrever o resultado no arquivo global de estilos da aplicação. O ponto principal não é decorar a ferramenta, mas entender a técnica. Eu posso estar usando Antigravity, Angular e Gemini, mas o mesmo raciocínio pode ser aplicado em Cursor, Windsurf, Copilot ou qualquer IDE moderna com agentes de IA.

O resultado é uma base global de estilos contendo variáveis para cores, tipografia, espaçamento, tema claro e tema escuro. A partir desse momento, a aplicação passa a ter uma identidade visual centralizada.

Outro detalhe importante é que podemos adaptar essa estratégia para outras tecnologias. Se o projeto usar Tailwind, CSS Modules, Styled Components ou qualquer outra abordagem, o prompt deve ser ajustado para gerar a estrutura compatível com aquela stack. O conceito continua o mesmo: o design system precisa estar centralizado e os componentes precisam consumir tokens, não valores soltos.

Depois de criar os tokens, partimos para a aplicação desses tokens na estrutura visual da aplicação. Aqui o prompt precisa ser ainda mais restritivo. Pedimos ao agente para estilizar a navegação principal e o formulário de transferência PIX, mas proibimos explicitamente o uso de cores hexadecimais absolutas. O agente deve consumir obrigatoriamente as variáveis criadas no arquivo global usando a função de variáveis do CSS.

Essa restrição é essencial. Modelos de IA tendem a tentar resolver visualmente a tarefa da forma mais direta possível. Se não delimitarmos bem, eles podem gerar cores fixas, estilos inline ou valores duplicados. Isso quebra justamente a ideia de design system. Por isso, precisamos reforçar que a estilização deve usar os tokens já criados.

Também aproveitamos para pedir uma pequena reorganização do componente. No exemplo, o agente havia deixado o template dentro do próprio arquivo do componente, algo comum em componentes pequenos e em alguns estilos de desenvolvimento. Eu prefiro separar o template em seu próprio arquivo HTML, principalmente para manter clareza e organização quando a tela começa a crescer. Essa é uma preferência arquitetural que também precisa ser comunicada ao agente.

Esse ponto mostra uma prática importante no uso de IA para desenvolvimento: o agente precisa aprender as preferências do projeto. Não basta pedir uma tarefa funcional. Precisamos orientar estrutura, estilo de código, separação de responsabilidades e padrões que o time deseja seguir.

O Antigravity então cria uma lista de tarefas antes de aplicar as mudanças. Ele identifica que precisa atualizar a rota principal, extrair o template para um arquivo HTML, ajustar a referência do componente, criar ou atualizar os estilos e aplicar os tokens no layout. Esse planejamento é útil porque permite revisar a intenção do agente antes da execução.

Depois que as alterações são aplicadas, revisamos os arquivos gerados. O componente passa a utilizar template externo, os estilos ficam separados e a aplicação começa a consumir os tokens globais. A navegação recebe identidade visual, o formulário passa a ter aparência mais profissional e o layout começa a refletir o briefing original da marca.

O ponto mais importante é que a estilização não está presa a valores absolutos. Ela está conectada aos tokens. Isso significa que, se amanhã quisermos mudar a cor de destaque de verde para laranja, roxo ou vermelho, fazemos a alteração em um único lugar. Todos os componentes que consomem aquele token refletem automaticamente a mudança.

Essa é a grande vantagem de trabalhar com tokens de design.

Eles permitem consistência, manutenção e evolução visual do produto com muito mais segurança.

Também vale destacar que essa prática reduz desalinhamento entre design e desenvolvimento. Quando o design system está representado tecnicamente como tokens, fica mais fácil garantir que aquilo que foi definido por produto e UX seja realmente implementado na aplicação.

Em projetos maiores, esses tokens podem inclusive vir de ferramentas de design, como Figma, e serem sincronizados com o código. Mas mesmo em um projeto simples, já conseguimos aplicar o mesmo princípio: valores visuais importantes precisam ser centralizados, nomeados semanticamente e reutilizados de forma consistente.

Essa aula mostra bem como a IA pode ajudar não apenas a escrever código, mas também a estruturar boas práticas de front-end. O agente não está apenas criando CSS. Ele está ajudando a transformar um briefing de marca em uma base reutilizável de design system,

aplicar essa base nos componentes e manter a aplicação preparada para evoluir visualmente sem retrabalho.

Capítulo 3 - Geração de Componentes Acessíveis

À medida que começamos a estruturar um front-end mais profissional, existe uma tendência muito comum entre desenvolvedores menos experientes de focar apenas em layout, estilização e funcionamento visual da interface. A aplicação fica bonita, os componentes aparecem corretamente na tela, os formulários funcionam e os fluxos parecem completos. Porém, existe uma camada extremamente importante da experiência do usuário que muitas vezes acaba sendo ignorada durante o desenvolvimento: acessibilidade.

E esse é um ponto que precisa ser tratado com muita seriedade.

Quando falamos em acessibilidade, não estamos falando apenas de adicionar algumas propriedades no HTML ou seguir uma checklist superficial. Acessibilidade envolve experiência humana, inclusão digital, usabilidade, navegação, semântica e até aspectos psicológicos da interação entre pessoas e sistemas.

Uma aplicação acessível é uma aplicação preparada para ser utilizada pelo maior número possível de pessoas, independentemente de limitações visuais, motoras, cognitivas ou tecnológicas.

Isso significa pensar em:

- navegação por teclado
- leitores de tela
- ordem semântica do conteúdo
- clareza textual
- contraste visual
- gerenciamento de foco
- feedback de interface
- comportamento previsível

Esse tema é muito maior do que apenas desenvolvimento front-end.

Mas, do ponto de vista técnico, existem várias práticas que ajudam a tornar os componentes muito mais compatíveis com tecnologias assistivas.

E é exatamente isso que começamos a trabalhar nesta etapa.

Na aula anterior, nós criamos nossos design tokens e estruturamos uma base visual consistente para a aplicação. Agora começamos a evoluir o front-end criando uma pequena biblioteca de componentes reutilizáveis.

Mas antes de sair criando botões, cards, inputs e formulários, precisamos garantir que esses componentes já nasçam preparados para acessibilidade.

Esse ponto é muito importante.

Depois que um projeto cresce, corrigir acessibilidade costuma ser muito mais caro do que construir corretamente desde o início.

Por isso, o ideal é incorporar boas práticas de acessibilidade já no nascimento dos componentes.

No exemplo da aula, escolhemos criar um componente modal de erro.

Esse é um ótimo exemplo porque modais possuem diversos desafios relacionados à acessibilidade.

Precisamos garantir:

- leitura correta por leitores de tela
- gerenciamento adequado de foco
- navegação por teclado
- fechamento via ESC
- semântica adequada
- isolamento visual e lógico da interface

Além disso, queremos manter integração com os design tokens criados anteriormente.

Mais uma vez utilizamos a mesma abordagem que trabalhamos ao longo do módulo inteiro: prompts estruturados.

E aqui aparece um conceito extremamente interessante para times técnicos: Prompt Garden.

Um Prompt Garden é basicamente uma coleção organizada de prompts reutilizáveis da equipe.

Em vez de cada desenvolvedor criar prompts isolados e improvisados, o time passa a manter uma biblioteca versionada de prompts especializados para diferentes tarefas.

Por exemplo:

- geração de componentes acessíveis
- criação de design tokens

- refinamento de requisitos
- geração de backlog
- documentação técnica
- sanitização de dados
- UX Writing

Isso transforma prompts em patrimônio técnico compartilhado da equipe.

Essa prática se conecta diretamente com o conceito de Prompt as Code que trabalhamos anteriormente.

Na prática, criamos um novo arquivo dentro da pasta de prompts do projeto especificamente voltado para geração de componentes acessíveis.

E aqui novamente seguimos a mesma estrutura que já virou padrão no nosso fluxo:

- papel
- objetivo
- regras
- formato de saída

Esse padrão é extremamente importante porque reduz ambiguidade.

Quanto mais claro o contexto fornecido ao agente, melhor tende a ser a qualidade da implementação.

Nesse caso, definimos que o agente deve atuar como engenheiro de front-end sênior especialista em acessibilidade e nas diretrizes da W3C WCAG.

Esse detalhe é importante.

A W3C é uma das principais organizações responsáveis pela definição de padrões da web. Dentro dela, existem recomendações específicas de acessibilidade chamadas WCAG, que orientam boas práticas para construção de interfaces acessíveis.

Quando direcionamos o agente para essas diretrizes, aumentamos muito a chance de receber uma implementação alinhada com padrões reais da web moderna.

Depois definimos o objetivo: gerar um componente Angular standalone seguindo boas práticas corporativas e consumindo exclusivamente os design tokens já existentes.

Isso também é importante porque não queremos que o agente crie estilos isolados ou ignore nosso design system.

Queremos que acessibilidade e consistência visual caminhem juntas.

Em seguida entram as regras técnicas.

E aqui aparecem alguns dos conceitos mais importantes da aula: propriedades ARIA.

Muitas pessoas iniciantes em front-end olham atributos ARIA e pensam que eles não fazem diferença porque visualmente nada muda na tela.

Mas esse pensamento é extremamente perigoso.

As propriedades ARIA existem justamente para enriquecer semanticamente a interface para tecnologias assistivas.

Leitores de tela, navegadores adaptativos e outras ferramentas de acessibilidade utilizam essas propriedades para interpretar corretamente o comportamento dos componentes.

Por exemplo, quando informamos que um componente é um modal, estamos dizendo semanticamente para o sistema que aquele elemento não representa uma navegação de página inteira, mas sim uma camada sobreposta dentro da mesma tela.

Isso altera completamente a forma como leitores de tela interpretam o foco e anunciam conteúdo para o usuário.

Outros atributos ajudam a descrever:

- títulos
- relações semânticas
- elementos ocultos
- estados interativos
- hierarquia de navegação

Tudo isso melhora enormemente experiência de usuários que dependem de tecnologias assistivas.

E aqui existe um detalhe importante: acessibilidade não beneficia apenas pessoas com deficiência visual total.

Ela melhora experiência de navegação para diversos perfis de usuário.

Navegação por teclado, por exemplo, é extremamente útil em várias situações.

Usuários avançados utilizam teclado para ganhar produtividade. Pessoas com limitações motoras dependem disso. Tecnologias assistivas também utilizam fluxo de foco e navegação semântica para operar corretamente.

Por isso, outro requisito importante do componente foi suporte completo à navegação por teclado.

O modal precisa permitir fechamento usando a tecla ESC.

Esse detalhe parece pequeno, mas é extremamente importante.

Imagine um usuário utilizando apenas teclado ou um leitor de tela preso dentro de um modal sem mecanismo fácil de saída. Isso cria um problema grave de usabilidade.

Além disso, também precisamos pensar em gerenciamento de foco.

Quando um modal abre, o foco precisa ser direcionado corretamente para o componente. Quando ele fecha, o foco idealmente deve retornar ao elemento anterior.

Esse tipo de detalhe melhora muito experiência de acessibilidade.

Outro ponto importante é que o prompt também reforça uma restrição já utilizada anteriormente: proibição de valores absolutos de estilo.

O componente deve consumir exclusivamente os design tokens já definidos.

Esse detalhe mostra como diferentes partes da arquitetura front-end começam a se integrar:

- design system
- acessibilidade
- prompts estruturados
- componentes reutilizáveis
- IA assistida por contexto

Tudo começa a funcionar como um fluxo unificado.

Depois de criar o prompt, iniciamos uma nova conversa com o agente.

Esse ponto também é importante.

Quando trabalhamos com IDEs assistidas por IA, criar conversas separadas por contexto ajuda muito na organização do raciocínio do agente.

Misturar muitos assuntos diferentes dentro do mesmo histórico pode gerar confusão contextual.

Por isso, separar conversas por domínio costuma melhorar qualidade das respostas.

Na solicitação do componente, pedimos explicitamente:

- um modal standalone
- inputs de título e mensagem
- fechamento por botão
- fechamento via ESC
- consumo exclusivo dos tokens do style.css

Perceba novamente o nível de especificidade.

Esse é um dos maiores aprendizados de todo trabalho profissional com LLMs.

Quanto mais específico for o prompt, menor a chance de comportamento inesperado.

Antigamente isso era ainda mais crítico porque modelos alucinavam muito mais. Hoje os agentes evoluíram bastante, principalmente quando combinados com MCP e documentação atualizada, mas ainda assim especificidade continua sendo uma das melhores práticas mais importantes.

Outro aspecto extremamente interessante do Antigravity é o sistema de tarefas.

Antes de aplicar alterações, o agente monta uma espécie de plano de execução.

Ele identifica:

- arquivos que serão criados
- componentes necessários
- estilos envolvidos
- dependências
- modificações planejadas

Isso é muito importante porque mantém o desenvolvedor no controle.

Mais uma vez, o modelo ideal não é automação cega.

Queremos um copiloto técnico que proponha mudanças, mas permita revisão humana antes da execução.

Durante a implementação apareceu um detalhe interessante relacionado ao Angular Language Service.

O agente marcou alguns falsos positivos de erro durante a criação do listener da tecla ESC.

Isso é importante porque mostra uma realidade do desenvolvimento assistido por IA: nem sempre os avisos da IDE representam erros reais de execução.

Ferramentas de análise estática, plugins e language servers também possuem limitações.

Por isso, precisamos validar o comportamento real da aplicação antes de assumir que algo está quebrado.

Esse ponto reforça novamente a importância da revisão humana.

Depois que o componente ficou pronto, integramos o modal ao fluxo principal da aplicação.

Criamos então uma regra simples de negócio apenas para demonstração: se o valor do PIX ultrapassar cinco mil reais, o modal de erro deve aparecer.

Esse exemplo conecta várias coisas que trabalhamos anteriormente:

- regra de negócio
- UX Writing
- acessibilidade
- design tokens
- componentes reutilizáveis
- Angular moderno
- controle de estado

Também utilizamos a nova sintaxe condicional do Angular para controlar a exibição do modal.

Esse detalhe mostra novamente o valor do MCP.

Como o agente está conectado às boas práticas mais recentes do Angular, ele consegue utilizar recursos modernos do framework em vez de padrões antigos.

Ao final, o comportamento ficou exatamente como esperado:

- o modal aparece corretamente
- a mensagem é exibida
- o fechamento funciona no botão
- o ESC fecha o modal
- os tokens de design são respeitados
- os atributos ARIA estão presentes
- leitores de tela conseguem interpretar corretamente o componente

Visualmente, muitas dessas propriedades não mudam nada para usuários comuns.

Mas semanticamente fazem enorme diferença para tecnologias assistivas.

E esse talvez seja um dos pontos mais importantes da acessibilidade moderna.

Boa acessibilidade muitas vezes é invisível visualmente, mas transforma completamente a experiência para quem depende dela.

Esse capítulo mostra muito bem como IA pode ajudar não apenas na velocidade de desenvolvimento, mas também na incorporação de boas práticas técnicas que muitas vezes acabam esquecidas em projetos tradicionais.

Quando configuramos corretamente contexto, prompts e ferramentas, conseguimos utilizar agentes para acelerar desenvolvimento mantendo qualidade arquitetural, consistência visual e preocupação real com acessibilidade.

Capítulo 4 - Criando Interfaces Utilizando o Stitch

Até este ponto do módulo, nós utilizamos inteligência artificial para acelerar praticamente todas as etapas do nosso fluxo de desenvolvimento. Refinamos requisitos, levantamos edge cases, criamos diagramas, estruturamos UX Writing, sanitizamos datasets, geramos backlog técnico, organizamos prompts como código, configuramos nosso ambiente Angular, criamos design tokens e começamos a construir componentes acessíveis utilizando agentes integrados ao framework.

Agora começamos a explorar uma nova etapa muito importante dentro do fluxo moderno de desenvolvimento front-end: prototipação visual assistida por IA.

E aqui entra uma ferramenta extremamente interessante chamada Stitch.

O Stitch é uma ferramenta do Google voltada especificamente para geração de interfaces gráficas utilizando linguagem natural. A ideia é muito parecida com o que já vimos no AI Studio: eu escrevo um prompt descrevendo o que quero, e a IA gera o resultado. A grande diferença é que o Stitch é especializado em UX e interface visual.

Esse detalhe muda completamente o tipo de saída produzida.

Enquanto no AI Studio trabalhávamos mais com texto, documentação, prompts e análise semântica, no Stitch o foco é gerar layouts, estruturas visuais e protótipos de interface.

É muito importante entender isso corretamente.

O Stitch não substitui diretamente o front-end final da aplicação. Ele funciona muito mais como uma ferramenta de ideação, prototipação e aceleração visual.

Pense nele como uma camada intermediária entre UX e implementação.

E esse é um dos grandes pontos da aula: aprender como combinar múltiplas ferramentas de IA dentro do mesmo fluxo de engenharia.

Outra observação importante é que o conceito vale muito mais do que a ferramenta específica.

Hoje estamos utilizando o Stitch porque ele representa bem essa categoria de soluções. Mas existem várias ferramentas semelhantes no mercado, como Lovable e muitas outras que provavelmente continuarão surgindo rapidamente.

A tecnologia muda muito rápido.

Por isso, o mais importante não é decorar nome de ferramenta. O mais importante é entender a técnica e o fluxo de trabalho.

A técnica aqui é utilizar IA para gerar rapidamente protótipos de interface que depois serão integrados ao projeto real.

No nosso caso, decidimos criar a tela de comprovante do PIX.

Até agora tínhamos criado o formulário de transferência e os componentes de validação. Agora queremos criar uma experiência visual para o comprovante de sucesso da operação.

E aqui começamos novamente com aquilo que virou praticamente uma regra em todo uso profissional de IA: o prompt.

Mesmo em ferramentas especializadas, a qualidade do resultado continua extremamente dependente da clareza do contexto fornecido.

No Stitch, descrevemos uma tela desktop para comprovante PIX de sucesso. Definimos que o design deveria ser moderno, clean e minimalista, pensado para monitores largos. Também especificamos elementos importantes da interface:

- card centralizado
- sombra suave
- ícone de sucesso
- valor em destaque
- nome do recebedor
- data e hora
- botão de retorno

Além disso, reaproveitamos os conceitos visuais já definidos anteriormente no nosso design system:

- azul escuro
- verde neon
- identidade visual consistente

Esse detalhe é muito importante.

Mesmo quando usamos ferramentas externas de prototipação, precisamos manter alinhamento visual com o sistema real da aplicação.

O Stitch então gera automaticamente uma interface visual baseada na descrição fornecida.

E aqui existe um detalhe extremamente interessante: o resultado não é apenas uma imagem.

O Stitch gera estrutura real de interface.

Isso significa HTML, CSS e componentes visuais utilizáveis como base para implementação.

Esse é um ponto muito poderoso do fluxo moderno de IA aplicada ao front-end.

Não estamos apenas desenhando mockups estáticos.

Estamos gerando estruturas semi-implementadas que podem servir como ponto de partida técnico.

Depois que o protótipo é gerado, conseguimos:

- visualizar
- testar
- criar variações
- adaptar
- exportar
- transformar em código real

Inclusive, a ferramenta permite exportar para plataformas como Figma e outras integrações.

Mas aqui precisamos entender uma coisa importante: o código gerado pelo Stitch ainda não está integrado ao nosso projeto Angular.

Ele é apenas um protótipo visual.

No exemplo da aula, o Stitch gerou uma página HTML utilizando Tailwind CSS.

Isso é ótimo para prototipação rápida, mas não resolve diretamente nosso problema.

Nosso projeto atual:

- utiliza Angular
- usa standalone components
- possui design tokens próprios
- segue arquitetura específica
- não utiliza Tailwind

Então, precisamos fazer uma etapa de refatoração e adaptação.

E é aqui que a combinação entre ferramentas começa a ficar extremamente poderosa.

Primeiro usamos o Stitch para gerar rapidamente a interface visual.

Depois usamos o Antigravity junto com nosso agente Angular para transformar aquele protótipo em componente real integrado à aplicação.

Esse fluxo mostra muito bem como IA não precisa funcionar isoladamente.

Podemos combinar ferramentas especializadas em diferentes etapas do desenvolvimento.

O Stitch funciona como acelerador visual.

O Antigravity funciona como agente de engenharia integrado ao projeto.

O Angular MCP garante aderência às melhores práticas do framework.

Os design tokens garantem consistência visual.

Tudo começa a trabalhar em conjunto.

Depois de baixar o código gerado pelo Stitch, organizamos o material dentro do projeto criando uma pasta específica para armazenar o modelo exportado.

Esse detalhe é importante porque mantém separação clara entre:

- protótipo bruto
- implementação real
- prompts
- componentes da aplicação

Mais uma vez aparece aqui uma preocupação constante ao longo do módulo inteiro: organização técnica.

IA gera muito material rapidamente. Se não organizarmos corretamente, o projeto vira caos.

Por isso continuamos utilizando:

- pastas específicas
- prompts versionados
- contexto organizado
- separação de responsabilidades

Depois disso, criamos mais um prompt especializado.

Novamente seguimos o mesmo padrão que já virou praticamente uma arquitetura operacional do módulo:

- papel
- objetivo
- diretrizes
- formato de saída

O agente agora assume o papel de engenheiro front-end sênior especialista em Angular e design systems corporativos.

O objetivo deixa de ser gerar uma interface do zero e passa a ser refatorar um HTML bruto exportado do Stitch para um componente Angular moderno.

Esse detalhe é muito importante.

A IA começa a trabalhar não apenas como geradora de código, mas como ferramenta de transformação arquitetural.

Ela pega uma estrutura genérica e adapta para a stack específica do projeto.

As diretrizes reforçam vários pontos importantes:

- criar standalone component
- usar Input Signals
- consumir design tokens
- evitar cores absolutas
- manter layout desktop
- gerar arquivos separados

Perceba como o nível de especificidade continua extremamente alto.

Isso é uma característica constante do desenvolvimento profissional assistido por IA.

Quanto mais importante for consistência arquitetural, mais explícitas precisam ser as regras fornecidas ao agente.

Depois disso, utilizamos o Antigravity para ler tanto o prompt quanto o código HTML exportado pelo Stitch.

E aqui aparece outro conceito importante da ferramenta: artifacts.

O Antigravity cria artefatos intermediários para explicar o raciocínio da tarefa.

Ele monta listas de ações, planejamento de implementação e rastreamento das alterações realizadas.

Isso é extremamente útil porque permite acompanhar o comportamento do agente.

Não estamos trabalhando com uma caixa-preta completamente invisível.

Conseguimos entender:

- o que ele pretende fazer
- quais arquivos serão alterados
- quais decisões estão sendo tomadas

- quais componentes estão sendo criados

Esse fluxo melhora muito rastreabilidade e revisão técnica.

Depois da execução, o agente transforma o HTML exportado em um componente Angular real.

Ele:

- separa HTML
- cria CSS
- adapta ícones
- converte estrutura
- utiliza Input Signals
- aplica standalone component
- integra design tokens

Outro detalhe importante foi a substituição dos ícones utilizados pelo Stitch.

Como o protótipo original usava bibliotecas do Tailwind, adaptamos para Material Symbols, que se encaixavam melhor na stack da aplicação.

Isso mostra outro ponto importante do desenvolvimento assistido por IA: adaptação contextual.

Não queremos apenas copiar código bruto gerado por uma ferramenta.

Queremos integrar esse código ao ecossistema arquitetural real do projeto.

Depois disso, fizemos a integração do componente dentro do fluxo do PIX.

Agora, quando a transferência possui valor válido e menor que cinco mil reais, o formulário desaparece e o comprovante é exibido.

Mais uma vez utilizamos recursos modernos do Angular, como:

- standalone components
- Input Signals
- control flow moderno
- arroba-if
- arroba-else

E novamente o MCP do Angular ajuda muito aqui.

O agente consegue utilizar sintaxes atualizadas porque está conectado às melhores práticas mais recentes do framework.

Ao final, temos um fluxo completo funcionando:

- formulário PIX
- validação
- regra de negócio
- modal de erro
- comprovante de sucesso
- identidade visual consistente
- integração Angular
- acessibilidade
- design tokens

E tudo isso foi construído combinando múltiplas ferramentas de IA em etapas diferentes do processo.

Esse talvez seja um dos pontos mais importantes desta aula.

O futuro do desenvolvimento assistido por IA não está em uma única ferramenta mágica que resolve tudo.

O verdadeiro ganho aparece quando combinamos ferramentas especializadas dentro de um fluxo inteligente de engenharia.

Uma ferramenta gera protótipos.

Outra refatora código.

Outra documenta.

Outra analisa requisitos.

Outra cria backlog.

Outra ajuda no design system.

O profissional moderno passa a atuar muito mais como orquestrador dessas capacidades.

E isso muda bastante o papel do desenvolvedor.

Em vez de gastar toda energia em tarefas operacionais repetitivas, começamos a focar mais em:

- arquitetura
- validação
- contexto
- integração
- experiência
- consistência
- tomada de decisão

A IA acelera execução, mas continua dependendo fortemente de direcionamento técnico humano.

E talvez esse seja o grande padrão que apareceu ao longo de todo o módulo.

Quanto melhor estruturamos contexto, arquitetura e fluxo de trabalho, mais valor conseguimos extrair das ferramentas de IA dentro da engenharia de software moderna.

Capítulo 5 - Criando Interfaces Utilizando o Stitch

Até agora, nós exploramos diferentes formas de utilizar inteligência artificial dentro do fluxo moderno de desenvolvimento front-end. Refinamos requisitos, geramos documentação, criamos UX Writing, estruturamos design systems, utilizamos prompts versionados, criamos componentes acessíveis e até prototipamos interfaces utilizando o Stitch.

Agora chegamos em um cenário extremamente comum dentro de empresas maiores e times mais maduros de produto: o handoff entre UX e desenvolvimento.

No dia a dia corporativo, normalmente existe uma separação clara entre quem pesquisa experiência do usuário e quem implementa a aplicação. O time de UX conduz entrevistas, estudos de usabilidade, pesquisas, testes e validações visuais. Depois disso, eles entregam o resultado para o time de desenvolvimento utilizando ferramentas como Figma.

Esse processo é conhecido como handoff.

O objetivo do handoff é transformar decisões visuais e comportamentais em algo que o desenvolvimento consiga implementar com fidelidade.

Durante muitos anos, esse fluxo dependia totalmente de interpretação manual. O desenvolvedor abria o Figma, analisava espaçamentos, fontes, cores, alinhamentos, tamanhos e começava a transformar aquilo em HTML, CSS e componentes reais.

Esse processo continua existindo, mas a IA muda bastante a forma como ele pode ser executado.

Agora conseguimos utilizar modelos multimodais para analisar imagens diretamente e transformar essas imagens em código integrado à aplicação.

E isso é um ponto extremamente importante desta aula.

Até agora trabalhamos muito com:

- texto para texto
- texto para código
- HTML para componente

Agora começamos a trabalhar com:

- imagem para código

Esse é um salto muito relevante no uso de IA aplicada ao desenvolvimento.

Os modelos multimodais modernos conseguem interpretar layouts visuais, identificar componentes, reconhecer hierarquia de interface, analisar espaçamentos e gerar implementações bastante próximas do design original.

Claro que isso ainda exige validação humana, mas acelera enormemente o fluxo entre UX e front-end.

No exemplo da aula, simulamos exatamente esse cenário corporativo.

Recebemos uma tela criada no Figma representando um extrato PIX. Além da imagem exportada, também recebemos um conjunto de especificações estruturais simulando o Dev Mode do Figma.

Esse detalhe é importante.

O Figma moderno já fornece várias informações úteis para implementação:

- espaçamentos
- dimensões
- alinhamentos
- tipografia
- cores
- hierarquia visual

Esses dados ajudam muito tanto desenvolvedores humanos quanto agentes de IA.

Mas existe um detalhe extremamente importante aqui: nosso projeto já possui um design system próprio.

Nós já criamos:

- design tokens
- tipografia
- cores
- padrões de layout
- estrutura Angular
- standalone components
- regras arquiteturais

Então não queremos simplesmente copiar visualmente o Figma de maneira cega.

Queremos adaptar aquela interface à realidade arquitetural da nossa aplicação.

Esse é um dos pontos mais importantes do desenvolvimento profissional.

O objetivo não é transformar o front-end em uma reprodução literal de imagens.

O objetivo é integrar UX, arquitetura e consistência visual dentro do sistema real.

Por isso, novamente começamos criando um System Prompt especializado.

E aqui aparece mais uma vez o padrão que praticamente se tornou a espinha dorsal de todo o módulo:

- papel
- objetivo
- regras
- formato de saída

O agente assume o papel de engenheiro front-end sênior especialista em Angular 21, acessibilidade e design systems.

Isso é importante porque o modelo precisa entender que não está apenas desenhando HTML genérico.

Ele está implementando um componente corporativo integrado a uma aplicação Angular moderna.

Depois definimos o objetivo: analisar uma imagem de alta fidelidade do Figma e gerar um standalone component capaz de renderizar corretamente aquela lista de transações.

Esse detalhe é muito importante.

O modelo agora precisa interpretar visualmente uma imagem e transformar aquilo em estrutura técnica.

Aqui entra o conceito de multimodalidade.

Durante muito tempo, os modelos trabalhavam principalmente com texto. Hoje, conseguimos fornecer múltiplos tipos de entrada:

- texto
- imagem
- áudio
- vídeo
- documentos

No nosso caso, usamos:

- imagem do layout
- especificações textuais
- contexto arquitetural do projeto

Tudo isso simultaneamente.

Esse fluxo é extremamente poderoso porque aproxima muito o trabalho da IA do fluxo real das empresas.

Na prática, o desenvolvedor recebe exatamente isso no dia a dia:

- imagens do Figma
- especificações
- regras de negócio
- design system
- arquitetura existente

A IA começa a atuar como acelerador desse handoff.

Depois entram as diretrizes técnicas.

Aqui pedimos:

- standalone component
- interface TypeScript para transações
- uso de Signals
- uso do control flow moderno
- consumo exclusivo dos design tokens
- proibição de cores absolutas
- uso de ícones do Material Symbols

Perceba novamente o nível de detalhamento.

Quanto mais queremos preservar consistência arquitetural, mais explícitas precisam ser as restrições do agente.

Outro ponto importante é a proibição de cores hexadecimais absolutas.

Isso aparece repetidamente ao longo do módulo porque é uma prática extremamente importante em design systems maduros.

O agente não deve inventar cores novas.

Ele precisa consumir os tokens já definidos globalmente.

Esse detalhe garante:

- consistência visual

- manutenção simplificada
- troca futura de temas
- alinhamento com UX

Depois de estruturar o prompt, iniciamos uma nova conversa com o Antigravity e fornecemos:

- o prompt
- a imagem exportada do Figma
- os specs textuais

Nesse momento, o agente começa a executar uma tarefa multimodal real.

E aqui percebemos uma diferença interessante de comportamento.

Quando trabalhamos apenas com texto, o processamento tende a ser mais rápido. Agora o modelo precisa interpretar visualmente a imagem antes de transformar aquilo em código.

Esse processo exige mais raciocínio contextual.

O agente analisa:

- composição visual
- hierarquia
- alinhamento
- estrutura de listas
- ícones
- espaçamentos
- padrões semânticos

Depois disso, ele começa a gerar o componente Angular.

Mais uma vez, o Antigravity cria artifacts intermediários mostrando o planejamento da tarefa.

Isso é muito importante porque mantém rastreabilidade do raciocínio do agente.

Ao final da geração, o componente é criado corretamente:

- HTML
- CSS
- TypeScript
- estrutura Angular
- control flow moderno
- integração com design tokens

Outro ponto interessante é que o agente utiliza os recursos modernos do Angular 21, como o novo control flow com `aroba-for`.

Isso novamente acontece porque configuramos corretamente o MCP e orientamos o agente a seguir as boas práticas mais recentes do framework.

Esse detalhe mostra novamente como contexto influencia diretamente qualidade da implementação.

Depois de gerar o componente, ainda precisamos integrá-lo à aplicação.

Criamos então uma nova conversa solicitando:

- criação da rota extrato
- inclusão do item de menu
- uso de `routerLink` oficial
- manutenção do estilo da navegação

Esse fluxo é muito parecido com o desenvolvimento real.

A IA gera componentes isolados, mas ainda precisamos integrá-los corretamente ao fluxo da aplicação.

Durante a integração apareceu um problema interessante relacionado aos ícones do Material Symbols.

O componente estava utilizando corretamente os nomes dos ícones, mas a biblioteca não havia sido importada.

Esse é um ponto extremamente importante no uso profissional de IA.

A IA acelera muito o desenvolvimento, mas ainda precisamos revisar e validar comportamento real da aplicação.

E isso não é um defeito da ferramenta.

Isso faz parte do fluxo normal de engenharia.

Mesmo desenvolvedores humanos esquecem imports, dependências ou detalhes de configuração.

A diferença é que agora conseguimos detectar rapidamente o problema, entender o contexto e corrigir utilizando o próprio agente.

Outro detalhe interessante foi reaproveitar o contexto da conversa anterior.

Em vez de abrir uma conversa completamente nova, voltamos ao contexto do componente porque ele já continha parte do raciocínio necessário para correção.

Esse é um aprendizado importante no trabalho com LLMs.

Contexto possui valor.

Saber quando reaproveitar contexto e quando começar do zero é uma habilidade importante no uso profissional de IA.

Depois da correção, o agente adiciona corretamente a importação da biblioteca de ícones e o extrato passa a funcionar visualmente da maneira esperada.

O resultado final mostra algo muito interessante:

A interface mantém fidelidade ao Figma, mas ao mesmo tempo respeita o design system da aplicação.

Esse equilíbrio é extremamente importante.

Não queremos uma cópia visual desconectada da arquitetura real do projeto.

Queremos adaptação contextual.

E talvez esse seja um dos maiores aprendizados desta aula.

IA não serve apenas para acelerar escrita de código.

Ela pode funcionar como ponte inteligente entre:

- UX
- design
- arquitetura
- desenvolvimento
- design systems
- frameworks modernos

Começamos o módulo trabalhando texto para texto.

Depois texto para código.

Depois HTML para Angular.

Agora imagem para Angular.

Perceba como as possibilidades vão se expandindo.

Esse é exatamente o impacto da multimodalidade no desenvolvimento moderno.

As interfaces entre design, documentação e implementação começam a ficar muito mais fluidas.

E o desenvolvedor passa cada vez mais a atuar como alguém que orquestra contexto, valida qualidade, define arquitetura e integra capacidades diferentes de IA dentro do fluxo de engenharia.

Capítulo 6 - Corrigindo a interface utilizando IA

Ao longo das últimas aulas, nós exploramos diversas maneiras de utilizar inteligência artificial dentro do fluxo moderno de desenvolvimento front-end. Refinamos requisitos, criamos documentação, estruturamos design systems, geramos componentes acessíveis, utilizamos prototipação com Stitch e até transformamos imagens do Figma em componentes Angular reais.

Mas existe um ponto extremamente importante que precisa ficar muito claro: IA acelera desenvolvimento, mas não elimina revisão técnica.

Esse talvez seja um dos maiores erros que algumas pessoas começam a cometer quando entram no universo de desenvolvimento assistido por IA. Elas acreditam que o agente gera código automaticamente e pronto, o trabalho está finalizado.

Na prática, o fluxo profissional funciona de maneira completamente diferente.

A IA funciona como acelerador de produtividade, mas a responsabilidade pela qualidade final continua sendo do desenvolvedor.

E isso inclui:

- revisão visual
- validação arquitetural
- acessibilidade
- responsividade
- testes
- integração
- comportamento da interface

Esse capítulo é extremamente importante justamente porque mostra algo muito próximo da realidade do dia a dia profissional: corrigir problemas gerados durante o próprio desenvolvimento assistido por IA.

Porque bugs continuam existindo.

Inconsistências continuam aparecendo.

Detalhes de UX continuam exigindo refinamento.

E isso não significa fracasso da ferramenta.

Isso faz parte natural da engenharia de software.

No início da aula, nós analisamos o comprovante PIX que havia sido criado anteriormente utilizando o Stitch e integrado ao Angular.

Funcionalmente, ele estava correto.

O valor aparecia, os dados estavam certos, o fluxo funcionava e o componente estava integrado à aplicação.

Porém, visualmente existia um problema grave de acessibilidade.

O texto estava escuro sobre um fundo azul escuro, criando contraste insuficiente e dificultando leitura até mesmo para usuários sem deficiência visual.

Esse tipo de problema é extremamente comum em desenvolvimento front-end.

Muitas vezes a tela parece “bonita” à primeira vista, mas pequenos detalhes de contraste, espaçamento ou hierarquia visual comprometem experiência real do usuário.

E esse é um ótimo exemplo de por que revisão humana continua sendo indispensável.

A IA gerou a estrutura corretamente, mas não percebeu que o contraste estava inadequado dentro daquele contexto visual específico.

Então iniciamos o processo de correção.

E aqui aparece um aprendizado importante: nem toda alteração exige um novo System Prompt complexo.

Nas aulas anteriores, usamos prompts estruturados para:

- geração de componentes
- criação de design systems
- integração Angular
- prototipação
- acessibilidade

Mas aqui estamos fazendo uma correção extremamente pontual.

Nesse tipo de situação, normalmente uma instrução direta e objetiva já é suficiente.

Esse é um ponto importante no uso profissional de IA.

Nem sempre precisamos construir prompts gigantescos.

O nível de estrutura deve ser proporcional à complexidade da tarefa.

No exemplo da aula, simplesmente informamos ao agente:

- existe um problema de acessibilidade
- o texto está escuro
- o fundo é azul noturno
- precisamos melhorar legibilidade
- devemos usar a variável `varColorTextLight`

Perceba como o pedido é extremamente específico.

Quanto mais preciso for o contexto, melhor tende a ser o resultado.

E aqui aparece novamente um padrão importante: reutilização do design system.

Em vez de pedir uma cor qualquer, usamos explicitamente um token já existente na aplicação.

Isso mantém consistência visual e evita criação de soluções isoladas.

O agente então analisa o componente e realiza ajustes específicos no CSS:

- corrige a cor do texto
- ajusta elementos do detalhe visual
- melhora o botão
- aplica contraste correto

Depois da alteração, a interface imediatamente melhora.

Agora o comprovante possui legibilidade adequada e a experiência visual fica muito mais consistente.

Esse exemplo parece simples, mas ele ensina algo extremamente importante sobre desenvolvimento assistido por IA: revisão crítica é obrigatória.

O desenvolvedor continua sendo responsável por validar:

- contraste
- legibilidade
- semântica
- consistência
- UX
- acessibilidade

A IA ajuda muito, mas ela ainda não substitui percepção humana refinada sobre experiência visual.

Outro ponto extremamente importante da aula aparece logo depois: responsividade.

No desktop, a aplicação parecia correta.

Mas quando abrimos o Chrome DevTools e começamos a simular dispositivos móveis, vários problemas aparecem.

Esse fluxo é extremamente comum em desenvolvimento front-end real.

Uma interface pode funcionar perfeitamente em monitores largos e quebrar completamente em telas pequenas.

Por isso, validar responsividade faz parte obrigatória do processo.

E aqui entra outro aprendizado importante: desenvolvimento front-end assistido por IA exige ainda mais revisão visual do que desenvolvimento back-end.

No back-end, conseguimos validar muita coisa com:

- testes unitários
- testes de integração
- contratos
- asserts
- validação de JSON

Já no front-end, vários problemas aparecem apenas visualmente:

- alinhamento
- responsividade
- contraste
- overflow
- espaçamento
- adaptação mobile
- experiência de navegação

Ou seja, mesmo que o código compile corretamente, a experiência ainda pode estar ruim.

Por isso o uso de DevTools continua sendo extremamente importante.

Na aula, identificamos dois problemas principais em mobile.

O primeiro estava no histórico de transações.

A lista horizontal estava espremendo os valores em telas pequenas.

O segundo estava relacionado ao menu lateral.

Em dispositivos mobile, o menu ocupava espaço demais e esmagava a interface principal.

Esses dois problemas são extremamente comuns em aplicações responsivas.

E novamente utilizamos IA para acelerar a correção.

Mais uma vez não criamos um System Prompt complexo porque a tarefa é relativamente objetiva.

Em vez disso, descrevemos exatamente o que queremos:

- abaixo de 600 pixels
- transformar layout em coluna
- ajustar flex-direction
- esconder menu lateral
- criar menu colapsável
- usar Signal
- adicionar botão hambúrguer
- utilizar Material Symbols

Perceba novamente a especificidade.

Esse talvez seja um dos maiores padrões de todo o módulo.

Quanto mais detalhado o contexto, mais controlado tende a ser o comportamento do agente.

O Antigravity então começa a executar as alterações.

E aqui vemos novamente um fluxo muito importante:

Primeiro ele analisa os arquivos relevantes.

Depois identifica:

- app.ts
- app.html
- CSS do histórico

Em seguida aplica as alterações necessárias.

No componente principal, ele cria um Signal chamado isMenuOpen para controlar estado do menu mobile.

Esse detalhe é importante porque estamos utilizando Angular moderno com Signals, mantendo alinhamento arquitetural com o restante da aplicação.

Depois ele cria o botão hambúrguer utilizando Material Symbols.

Além disso, implementa a lógica de abertura e fechamento do menu lateral.

No CSS do histórico, ele utiliza media queries para detectar larguras menores que 600 pixels e altera o comportamento do layout:

- flex-direction muda para coluna
- alinhamento é ajustado
- espaçamentos são reorganizados

Tudo isso melhora drasticamente experiência mobile.

Outro detalhe interessante é que o próprio agente acabou utilizando um número mágico em parte do CSS.

Esse ponto é muito importante porque mostra novamente que a IA não substitui revisão arquitetural.

Mesmo quando o resultado parece correto, ainda precisamos analisar:

- consistência
- aderência ao design system
- reutilização de tokens
- padrões arquiteturais

Ou seja, o agente ajuda muito, mas continua precisando de supervisão humana qualificada.

Depois das alterações, testamos novamente no DevTools.

Agora o menu funciona corretamente em mobile:

- o botão hambúrguer aparece
- o menu abre e fecha
- a interface não fica esmagada
- o histórico reorganiza corretamente os elementos

Esse fluxo mostra algo extremamente importante sobre desenvolvimento front-end moderno.

Construir interface não é apenas fazer a tela “funcionar”.

Precisamos validar:

- acessibilidade
- contraste
- experiência visual
- comportamento mobile
- semântica
- integração

- consistência

E talvez esse seja um dos maiores aprendizados deste capítulo.

A IA não elimina o trabalho crítico do desenvolvedor.

Ela elimina parte do trabalho operacional.

Isso é muito diferente.

O profissional continua responsável por:

- validar UX
- revisar comportamento
- detectar inconsistências
- analisar responsividade
- garantir qualidade

Mas agora conseguimos corrigir problemas muito mais rapidamente utilizando agentes como copilotos técnicos.

Esse é o verdadeiro impacto da IA no desenvolvimento moderno.

Ela acelera implementação, refatoração e correção, permitindo que o desenvolvedor dedique mais energia para análise crítica, experiência do usuário e qualidade arquitetural.

E isso talvez seja o ponto mais importante de todo o módulo até aqui:

IA não substitui engenharia.

Ela potencializa engenharia quando usada com contexto, revisão e pensamento crítico.

Módulo 03 - Orquestração de Agentes Locais e Nuvem

Capítulo 1 - A Fundação Enterprise (Nx Monorepo, MCP e Execução)

Nesta aula, começamos uma nova fase do nosso trabalho com ferramentas agênticas. Até aqui, exploramos bastante o uso de IA para front-end, UX, design system, componentes, prototipação e correção de interfaces. Agora, o objetivo é avançar para um cenário mais completo, aproximando o fluxo de desenvolvimento de uma aplicação enterprise real, com front-end, back-end, compartilhamento de contratos e execução integrada.

A ideia é construir uma plataforma de gerenciamento de Call for Papers para eventos. Esse tipo de sistema é interessante porque envolve várias partes comuns em aplicações corporativas: cadastro de palestrantes, submissão de propostas, regras de negócio, telas administrativas, APIs, contratos compartilhados e integração entre camadas. Por isso, ele se torna um bom laboratório para praticar não apenas geração de código com IA, mas também organização de arquitetura, uso de agentes e boas práticas de desenvolvimento com múltiplos projetos.

Neste projeto, vamos trabalhar com Angular no front-end e NestJS no back-end. É importante não confundir NestJS com Next.js. O NestJS é um framework JavaScript para back-end, bastante usado para criação de APIs estruturadas. Mas, como venho reforçando ao longo do curso, o ponto central aqui não é decorar a ferramenta específica. Poderíamos usar React no front-end, Java no back-end, Python, Spring Boot, Quarkus ou qualquer outra stack. O mais importante é entender o modo de trabalho: como eu organizo o projeto, como eu configuro agentes, como eu conecto documentação atualizada e como eu uso a IA para acelerar desenvolvimento sem abrir mão de controle técnico.

Para esse cenário, escolhemos trabalhar com monorepo. Um monorepo é uma estratégia em que vários projetos ficam dentro do mesmo repositório de código. No nosso caso, teremos pelo menos três partes: a aplicação Angular, a API em NestJS e uma biblioteca compartilhada de tipos. Essa abordagem não é nova. Grandes empresas utilizam monorepos há muitos anos. O Google, por exemplo, é um caso clássico de empresa que mantém uma base gigantesca de código organizada dessa forma, com ferramentas internas próprias para gerenciar escala, dependências e execução.

No ecossistema JavaScript, uma das ferramentas mais conhecidas para trabalhar com monorepos é o Nx. O Nx é open source, possui licença MIT e fornece uma estrutura muito interessante para gerenciar múltiplas aplicações, bibliotecas, dependências, execução, testes e build dentro de um único workspace. Além disso, ele possui recursos avançados, como cache, análise de dependências e integração com Nx Cloud para acelerar pipelines de CI/CD.

O ponto mais importante para nós é que o Nx permite gerenciar front-end e back-end no mesmo repositório sem transformar a aplicação em um monólito. Essa distinção é fundamental. Monorepo é uma estratégia de organização de código e desenvolvimento. Monólito é uma estratégia arquitetural ou de deploy. Posso ter um monorepo com várias aplicações independentes, cada uma empacotada, implantada e escalada separadamente. Também posso ter um monólito em múltiplos repositórios. São conceitos diferentes.

No nosso projeto, o monorepo vai facilitar bastante porque queremos compartilhar contratos entre front-end e back-end. Em uma arquitetura tradicional com repositórios separados, muitas vezes criamos uma estrutura de DTOs ou interfaces no back-end e depois precisamos duplicar esses contratos no front-end. Isso aumenta risco de divergência. Um campo muda de um lado, mas não muda do outro. Um nome é alterado

na API, mas a interface do front-end continua antiga. Esse tipo de inconsistência gera bugs difíceis de rastrear.

Com uma biblioteca compartilhada dentro do monorepo, conseguimos definir contratos comuns em um único lugar. Por exemplo, se criarmos uma interface de Speaker com identificador, nome, e-mail, título da palestra e informação se a pessoa é GDE ou não, tanto o front-end quanto o back-end podem consumir o mesmo contrato. Isso melhora consistência, reduz duplicação e torna a evolução do sistema mais segura.

O primeiro passo é criar o workspace Nx. Esse workspace funciona como a base do repositório. Ele inicializa a estrutura, configura o Git e prepara o ambiente para receber múltiplas aplicações. Depois disso, instalamos os plugins necessários para Angular, NestJS e JavaScript ou TypeScript básico. Esses plugins permitem que o Nx gere aplicações e bibliotecas seguindo as boas práticas de cada ecossistema.

Em seguida, criamos a aplicação Angular. Mesmo que não estejamos chamando diretamente a Angular CLI, o Nx utiliza a infraestrutura do Angular por baixo dos panos. Durante a criação, definimos algumas escolhas importantes, como uso de CSS puro, roteamento habilitado, standalone components e modo strict do TypeScript. Também escolhemos ferramentas de teste modernas, como Vitest para testes unitários e Playwright para testes end-to-end. Essas escolhas deixam o projeto preparado para as próximas etapas, inclusive quando formos falar de automação e validação.

Depois criamos a aplicação NestJS para a API. Aqui também o Nx ajuda bastante, porque ele integra essa aplicação ao workspace e já entende sua relação com o restante do repositório. Durante a configuração, definimos ferramentas como ESLint e Jest, mantendo uma base adequada para qualidade de código e testes.

Por fim, criamos a biblioteca compartilhada de tipos. Essa biblioteca será usada para armazenar interfaces, DTOs e contratos que precisam ser conhecidos tanto pelo front-end quanto pela API. Esse é um dos maiores ganhos práticos do monorepo neste exemplo.

Depois que tudo está criado, conseguimos executar os projetos de forma integrada. Em vez de abrir vários terminais manualmente, subir a API em uma aba, o front-end em outra, conferir porta por porta e gerenciar tudo separadamente, o Nx permite executar múltiplos targets ao mesmo tempo. Assim, conseguimos subir o servidor de desenvolvimento do front-end e da API com um comando coordenado pelo próprio workspace. Isso simplifica muito o dia a dia.

Com o ambiente funcionando, abrimos o projeto no Antigravity. Como já vimos nas aulas anteriores, o Antigravity é apenas uma das ferramentas possíveis. O mesmo conceito pode ser aplicado no Cursor, Copilot, Codex, Claude Code ou outras soluções agênticas. O importante é entender que agora estamos trabalhando com uma base maior, com múltiplos projetos e uma estrutura enterprise.

Nesse momento, configuramos também o MCP do Nx. Esse é um ponto muito importante.

O MCP permite que o agente tenha uma forma mais estruturada de interagir com a ferramenta. Em vez de depender apenas do conhecimento interno da LLM ou tentar adivinhar comandos, o agente passa a contar com um servidor especializado que entende o Nx, seus projetos, targets, bibliotecas e boas práticas. Isso torna o trabalho mais assertivo.

Quanto mais facilitamos o contexto para o agente, menor a chance de erro. Isso também reduz consumo desnecessário de tokens, porque o agente precisa tentar menos caminhos errados. Em um uso profissional, isso importa bastante. IA tem custo, mesmo quando parece gratuita no ambiente de desenvolvimento. Uma configuração bem feita economiza tempo, dinheiro e retrabalho.

Depois de configurar o MCP, fazemos a primeira exploração prática com o agente: criar um DTO de Speaker na biblioteca compartilhada. O prompt deixa claro que o agente deve consultar o MCP do Nx, mapear o projeto, identificar a biblioteca shared types e criar uma interface TypeScript dentro dela. Também deixamos uma restrição explícita: não modificar os aplicativos front-end ou API.

Essa restrição é muito importante. Quando trabalhamos com agentes em bases maiores, precisamos delimitar escopo. Se eu quero alterar apenas uma biblioteca compartilhada, preciso dizer isso claramente. Caso contrário, o agente pode decidir mexer em outros arquivos, criar integrações desnecessárias ou alterar aplicações que não deveriam ser tocadas naquele momento.

O agente então analisa o workspace, identifica os projetos, cria um plano e apresenta a proposta antes de executar. Esse fluxo de planejamento é uma boa prática. Eu consigo revisar antes que o código seja alterado. Se o plano estiver errado, posso ajustar. Se estiver correto, autorizo a execução.

No plano, ele propõe criar a interface Speaker, exportá-la no ponto público da biblioteca e executar validações usando comandos do Nx. Essa etapa mostra como MCP e agente funcionam melhor juntos. O agente não está apenas escrevendo um arquivo solto. Ele entende que a biblioteca precisa exportar o contrato para ser consumido por outras partes do monorepo.

Depois que a alteração é aplicada, revisamos o resultado. A interface contém os campos solicitados e fica disponível para uso futuro no front-end e no back-end. Em seguida, fazemos o commit da alteração.

Esse ponto também é importante. Mesmo com IA, boas práticas tradicionais continuam valendo. Commits pequenos, atômicos e bem descritos continuam sendo fundamentais. A IA pode até ajudar gerando mensagens de commit melhores, mas a disciplina de versionamento continua sendo responsabilidade do desenvolvedor.

Nesta aula, portanto, criamos a fundação enterprise do projeto. Configuramos um monorepo Nx, adicionamos front-end Angular, back-end NestJS, biblioteca compartilhada

de tipos, execução integrada e MCP para melhorar a interação do agente com o workspace. Também criamos o primeiro contrato compartilhado usando IA de forma controlada, revisável e alinhada com a estrutura do projeto.

Capítulo 2 - A Fundação Enterprise (Nx Monorepo, MCP e Execução)

À medida que começamos a trabalhar com projetos maiores e mais complexos utilizando agentes de IA, surge um problema muito importante que não pode ser ignorado: a falsa sensação de que basta escrever uma frase simples para a IA criar um sistema inteiro corretamente.

Esse é um dos erros mais comuns no uso inicial de ferramentas agênticas.

Muita gente entra nesse universo imaginando algo como:

“Crie um sistema de eventos técnicos.”

E espera que a IA produza automaticamente uma solução robusta, consistente, segura, bem arquitetada e alinhada às necessidades reais do produto.

Na prática, isso raramente acontece.

Mesmo com toda evolução das LLMs, quando deixamos espaço demais para interpretação, aumentamos enormemente:

- alucinação
- inconsistência arquitetural
- retrabalho
- desperdício de tokens
- decisões erradas
- geração de requisitos implícitos
- código desalinhado

Esse ponto é extremamente importante.

Quanto mais complexa for a feature, mais perigoso se torna depender apenas de prompts genéricos.

Nas primeiras aulas do módulo, nós já começamos a resolver isso parcialmente utilizando prompts estruturados. Trabalhamos bastante com:

- papel

- regras
- objetivo
- formato de saída

Essa abordagem funciona muito bem para tarefas pequenas e focadas.

Por exemplo:

- gerar um componente
- criar um modal
- produzir UX Writing
- gerar design tokens
- criar um DTO
- refatorar uma tela

Mas quando começamos a falar de features completas, envolvendo:

- front-end
- back-end
- contratos
- regras de negócio
- validações
- testes
- acessibilidade
- arquitetura

o nível de complexidade cresce muito.

E é justamente nesse ponto que entra o conceito de Spec-Driven Development.

Spec-Driven Development significa desenvolvimento guiado por especificações.

A ideia principal é extremamente interessante.

Em vez de pedir diretamente para a IA gerar código, primeiro criamos uma especificação estruturada da feature.

Essa especificação se transforma no artefato central do desenvolvimento.

E isso representa uma mudança importante de mentalidade.

Durante muitos anos, principalmente com a popularização do ágil, muita gente passou a tratar documentação e especificação como algo secundário.

A ideia de “o código é a documentação” ficou muito forte.

Mas agora, com a chegada das LLMs, especificação voltou a ganhar enorme relevância.

Porque agora não estamos mais apenas escrevendo código manualmente.

Estamos orientando agentes.

E agentes precisam de contexto extremamente bem definido.

Quanto mais clara for a especificação, maior tende a ser:

- assertividade
- velocidade
- qualidade
- consistência
- economia de tokens

Isso cria uma dinâmica muito interessante dentro da engenharia moderna.

A especificação deixa de ser apenas documentação humana e passa a funcionar como contrato operacional entre pessoas e agentes de IA.

Outro ponto importante é que hoje já existem ferramentas específicas voltadas para esse fluxo.

A comunidade começou rapidamente a perceber que precisava de padrões para trabalhar melhor com agentes autônomos.

Então começaram a surgir ferramentas como:

- SpecKit
- Kiro
- OpenSpec

Cada uma possui suas particularidades, mas todas trabalham com a mesma ideia central: gerar artefatos estruturados que servirão como base para os agentes implementarem as funcionalidades.

Nesta aula, escolhemos utilizar o OpenSpec.

O OpenSpec é uma ferramenta open source criada pela comunidade e totalmente agnóstica de fornecedor.

Esse detalhe é importante.

Ela não depende de uma empresa específica e pode ser utilizada com diferentes agentes e diferentes IDEs.

Outro ponto extremamente interessante do OpenSpec é que ele não possui “mágica escondida”.

Na prática, ele funciona utilizando um conjunto organizado de arquivos Markdown que atuam como System Prompts especializados.

Isso é um aprendizado muito importante.

Muitas vezes as pessoas imaginam que ferramentas agênticas possuem algum mecanismo secreto extremamente sofisticado.

Mas quando analisamos internamente, percebemos que boa parte delas funciona através de:

- prompts estruturados
- contexto
- arquivos Markdown
- instruções organizadas
- workflows bem definidos

O OpenSpec basicamente instala uma coleção de skills no projeto.

Essas skills são arquivos Markdown contendo instruções detalhadas para o agente.

E aqui aparece novamente um conceito que já vimos anteriormente no módulo: Prompt as Code.

Os prompts agora fazem parte do projeto.

Eles são versionados, organizados e reutilizados como qualquer outro artefato técnico.

No caso do OpenSpec, ele cria arquivos especializados para diferentes tarefas:

- exploração
- proposta
- design
- aplicação
- tasks

Cada skill orienta o agente sobre como agir naquele contexto específico.

Por exemplo, a skill de exploração ensina o agente a mapear a estrutura do projeto.

A skill de proposta ensina como gerar os artefatos da feature.

A skill de aplicação ensina como executar as tasks posteriormente.

Esse modelo é extremamente interessante porque transforma comportamento do agente em algo transparente e auditável.

Podemos abrir os arquivos, ler as instruções e até modificá-las.

Nada está escondido.

Outro detalhe importante é que isso funciona independentemente da ferramenta agêntica utilizada.

No exemplo da aula, utilizamos AntigraVity com Gemini.

Mas poderíamos usar:

- Claude
- Codex
- Cursor
- Copilot
- DeepSeek
- qualquer outra ferramenta compatível

O OpenSpec simplesmente adapta os prompts para o agente escolhido.

Depois da instalação, começamos utilizando uma capability muito interessante: exploração do projeto.

Nesse momento, usamos um comando especial para pedir que o agente analisasse nosso monorepo Nx e gerasse um mapa estrutural da aplicação.

Esse fluxo é extremamente útil no mundo real.

Imagine entrar em uma empresa nova, com um projeto grande, documentação desatualizada e centenas de arquivos.

Tradicionalmente, o desenvolvedor precisaria navegar manualmente pela base até entender arquitetura, aplicações, bibliotecas e dependências.

Agora conseguimos utilizar IA para acelerar enormemente esse processo.

O agente utiliza:

- MCP do Nx
- MCP do Angular
- estrutura do workspace

E gera um resumo da topologia da aplicação.

Inclusive utilizando diagramas ASCII para representar visualmente a arquitetura.

Esse detalhe é muito interessante porque mostra como os agentes conseguem transformar exploração estrutural em documentação legível.

Depois disso, partimos para a criação da especificação real da feature.

A feature escolhida foi um módulo de submissão de palestras para o sistema de eventos.

Ou seja, um fluxo de Call for Papers.

Mais uma vez começamos definindo contexto extremamente detalhado.

No prompt informamos:

- objetivo da feature
- regras arquiteturais
- frameworks utilizados
- uso obrigatório de standalone components
- uso de Signals
- acessibilidade via ARIA
- utilização do NestJS
- compartilhamento do SpeakerDTO
- obrigatoriedade de testes

Perceba como o nível de detalhamento cresce bastante.

E isso é proposital.

Quanto mais importante for a feature, maior precisa ser o refinamento da especificação.

Outro ponto extremamente importante foi deixar claro o que NÃO deveria ser feito.

Isso é uma prática fundamental no trabalho com agentes.

Se não delimitarmos explicitamente certas restrições, o agente pode começar a inventar requisitos adicionais.

Por exemplo:

- criar autenticação sem necessidade
- adicionar upload de arquivos
- criar banco desnecessário
- adicionar regras não solicitadas

Por isso, especificação boa não descreve apenas o que fazer.

Ela também descreve o que evitar.

Depois da geração, o OpenSpec cria três artefatos principais:

- proposal.md
- design.md
- tasks.md

Cada um possui um papel diferente.

O proposal descreve o “o quê” da feature.

Ou seja:

- qual problema resolve
- qual objetivo da mudança
- impacto esperado
- capacidades adicionadas

O design descreve “como” a feature deve funcionar.

Aqui entram:

- decisões arquiteturais
- requisitos funcionais
- requisitos não funcionais
- limitações
- escolhas técnicas

Já o tasks.md representa o plano operacional de implementação.

Ele divide as atividades entre:

- back-end
- front-end
- integração
- testes

Esse detalhe é extremamente importante porque o agente passa a trabalhar em cima de um plano estruturado.

E isso reduz enormemente improvisação durante geração de código.

Outro ponto interessante é perceber como esse fluxo se aproxima bastante de práticas antigas da engenharia de software, mas agora potencializadas por IA.

Continuamos precisando:

- refinamento
- especificação
- arquitetura
- validação
- alinhamento técnico

A diferença é que agora conseguimos acelerar produção desses artefatos utilizando LLMs.

E talvez esse seja um dos maiores aprendizados desta aula.

A IA não elimina engenharia de requisitos.

Ela aumenta ainda mais a importância dela.

Porque agora a especificação não serve apenas para alinhar pessoas.

Ela serve para orientar agentes autônomos.

Outro detalhe extremamente importante é que os artefatos gerados não são definitivos.

Eles devem ser revisados pelo time.

O desenvolvedor continua responsável por:

- validar decisões
- revisar arquitetura
- identificar inconsistências
- complementar regras
- corrigir lacunas

A IA gera uma primeira versão extremamente acelerada da especificação.

Mas a qualidade final continua dependendo de revisão humana qualificada.

Isso reforça novamente um padrão que apareceu durante todo o módulo:

A IA acelera produção.

Mas o pensamento crítico continua humano.

Depois da revisão, os artefatos são versionados dentro do repositório.

Isso também é muito importante.

Agora a especificação passa a fazer parte do histórico do projeto.

No futuro, quando a feature evoluir, o time poderá:

- revisar decisões
- criar novas versões
- refatorar specs
- adicionar alterações incrementais

Isso cria rastreabilidade extremamente valiosa para projetos maiores.

E talvez esse seja o maior ganho do Spec-Driven Development.

Ele transforma geração de código assistida por IA em um processo muito mais disciplinado, previsível e controlado.

Em vez de depender de prompts improvisados gigantescos, passamos a trabalhar com:

- especificações estruturadas
- arquitetura explícita
- tasks organizadas
- decisões documentadas
- contratos claros

Isso melhora:

- qualidade
- previsibilidade
- manutenção
- colaboração
- reutilização
- economia de tokens

E ao mesmo tempo aproxima IA do fluxo profissional real de engenharia de software moderna.

Capítulo 3 - Orquestração Paralela (Isolamento Físico e OpenSpec Apply)

Nas últimas aulas, nós começamos a estruturar um fluxo muito mais próximo da realidade de desenvolvimento enterprise moderno. Criamos um monorepo utilizando Nx, organizamos front-end Angular, back-end NestJS, compartilhamos contratos através de bibliotecas comuns e configuramos MCPs para melhorar o contexto dos agentes.

Depois disso, demos um passo ainda mais importante: adotamos Spec-Driven Development utilizando OpenSpec.

Em vez de pedir diretamente para a IA gerar código de forma improvisada, passamos a trabalhar com:

- proposal
- design
- tasks
- requisitos
- regras arquiteturais

- restrições explícitas

Isso trouxe muito mais previsibilidade para o desenvolvimento.

Agora chegamos em um ponto extremamente importante da engenharia moderna assistida por IA: execução paralela de agentes.

E aqui começamos a entrar em um nível bem mais avançado de fluxo de trabalho.

Até este momento, utilizamos apenas um agente por vez trabalhando na aplicação.

Mas em projetos reais, principalmente projetos maiores, isso rapidamente se torna um gargalo.

Porque no desenvolvimento tradicional já trabalhamos com múltiplas pessoas simultaneamente:

- front-end
- back-end
- QA
- UX
- DevOps

Então a pergunta natural passa a ser:

“Como trabalhamos com múltiplos agentes ao mesmo tempo sem causar colisão no código?”

Esse é exatamente o problema que começamos a resolver nesta aula.

Porque existe um risco muito grande quando vários agentes modificam simultaneamente a mesma base de código.

Sem organização adequada, começamos a ter:

- sobrescrita de arquivos
- conflitos
- inconsistências
- mudanças acidentais
- perda de contexto
- problemas de merge

Então precisamos criar isolamento.

E aqui entra uma funcionalidade extremamente interessante do Git chamada WorkTree.

O Git WorkTree não é uma funcionalidade nova.

Ela existe há bastante tempo.

Mas durante muitos anos foi pouco utilizada pela maioria das equipes.

Agora, com o crescimento do desenvolvimento assistido por IA, ela começou a ganhar enorme relevância.

Porque ela resolve exatamente um dos problemas centrais do trabalho paralelo com agentes.

A ideia do WorkTree é extremamente elegante.

Em vez de duplicar manualmente o repositório inteiro várias vezes, o Git permite criar múltiplos workspaces físicos associados ao mesmo repositório.

Cada workspace pode apontar para uma branch diferente.

Isso significa que conseguimos trabalhar com múltiplas versões do código simultaneamente na mesma máquina.

E tudo continua integrado ao mesmo banco de dados Git.

Esse detalhe é extremamente importante.

Não estamos copiando projetos independentes.

Estamos criando ambientes isolados ligados ao mesmo histórico Git.

Na prática, isso nos permite simular múltiplas equipes trabalhando paralelamente.

No exemplo da aula, criamos dois WorkTrees:

- um para API
- outro para UI

Cada um associado à sua própria branch.

Esse isolamento físico resolve um problema fundamental:

Cada agente enxerga apenas sua própria área de trabalho.

Isso evita que dois agentes tentem modificar os mesmos arquivos simultaneamente.

Esse fluxo é muito mais seguro.

Outro detalhe extremamente importante é que esse isolamento não existe apenas por causa da IA.

Ele também faz muito sentido para desenvolvimento humano.

Imagine dois desenvolvedores trabalhando simultaneamente na mesma máquina ou em ambientes compartilhados.

O WorkTree permite separar contextos de forma extremamente organizada.

Depois de criar os WorkTrees, começamos a utilizar o Agent Manager do Antigravity.

Esse é outro ponto muito importante.

Ferramentas modernas de desenvolvimento assistido por IA estão começando a evoluir de “chat integrado” para verdadeiros orquestradores de agentes.

O Agent Manager basicamente permite abrir múltiplos workspaces simultaneamente e atribuir tarefas independentes para cada agente.

Ou seja:

- cada agente possui seu contexto
- cada agente possui sua branch
- cada agente possui seu workspace físico
- cada agente executa suas próprias tarefas

Isso cria uma dinâmica extremamente parecida com múltiplos membros da equipe trabalhando ao mesmo tempo.

No nosso caso, um agente ficou responsável pelo back-end e outro pelo front-end.

E aqui aparece algo extremamente importante: divisão clara de responsabilidades.

Na especificação criada anteriormente com OpenSpec, já havíamos separado as tasks entre:

- front-end
- back-end
- shared types

Isso agora se torna extremamente útil.

Porque conseguimos aplicar apenas subconjuntos específicos da especificação em cada agente.

No caso da API, utilizamos o comando apply filtrando apenas as tarefas de:

- backend
- shared

Já no front-end aplicamos apenas:

- frontend

Esse detalhe é extremamente importante.

Os agentes não estão improvisando comportamento.

Eles estão executando partes específicas de uma especificação previamente refinada.

Esse é exatamente o poder do Spec-Driven Development combinado com múltiplos agentes.

A especificação se transforma em mecanismo de orquestração.

Outro detalhe extremamente interessante é perceber como o prompt fica muito menor nesta etapa.

Na geração inicial da spec, escrevemos bastante contexto.

Mas agora não precisamos repetir tudo.

Porque:

- requisitos já existem
- arquitetura já existe
- tasks já existem
- restrições já existem

O agente apenas consulta os artefatos da spec e executa o que foi definido.

Isso reduz enormemente repetição e desperdício de tokens.

Outro ponto importante é observar como o Antigravity trabalha com planejamento antes da execução.

Antes de modificar qualquer arquivo, o agente apresenta:

- plano de implementação
- tarefas
- arquivos impactados
- decisões previstas

Esse fluxo é extremamente saudável.

Porque mantém o desenvolvedor no controle.

Mais uma vez aparece um padrão importante de todo o módulo:

Não queremos automação cega.

Queremos copilotos técnicos supervisionados.

O agente propõe.

O desenvolvedor valida.

Depois da aprovação, os agentes começam a trabalhar simultaneamente.

E aqui começamos a ver algo extremamente interessante.

O front-end cria:

- componente Angular
- rota submit-talk
- integração com Signals
- acessibilidade ARIA
- mocks temporários

Enquanto isso, o back-end cria:

- DTOs
- validações
- controllers
- services
- integração com Class Validator

Esse fluxo reproduz exatamente o que acontece em times reais.

Muitas vezes o front-end precisa continuar avançando mesmo sem API pronta.

Então utiliza mocks temporários.

Depois, quando a API estiver pronta, a integração real acontece.

Outro detalhe extremamente importante da aula foi o problema de limite de tokens.

O agente do back-end inicialmente falhou utilizando o modelo Flash por limite de output.

Isso é extremamente relevante no uso profissional de IA.

Nem toda tarefa deve usar o mesmo modelo.

Existe uma estratégia importante de custo e capacidade.

Modelos menores:

- custam menos
- respondem mais rápido
- funcionam bem para tarefas simples

Modelos maiores:

- possuem contexto maior
- possuem melhor raciocínio
- lidam melhor com tarefas complexas

Por isso, uma prática muito comum começa a surgir:

Usar modelos mais fortes para:

- planejamento
- arquitetura
- raciocínio complexo

E modelos mais baratos para:

- execução operacional
- codificação repetitiva
- pequenas tarefas

Esse gerenciamento estratégico de modelos será cada vez mais importante no mercado.

Porque IA não é gratuita em ambientes profissionais.

Quanto maior o uso, maior o custo operacional.

Outro ponto extremamente interessante foi o comportamento de validação automática do Antigravity.

O agente não apenas escreveu código.

Ele também:

- executou testes
- abriu navegador
- verificou rotas
- validou comportamento

Isso mostra como os agentes modernos estão começando a atuar muito além da simples geração de código.

Eles começam a participar também da validação operacional da feature.

Claro que isso ainda exige revisão humana.

Mas acelera enormemente o fluxo.

Outro detalhe muito importante foi perceber que os resultados gerados ainda precisam ser avaliados criticamente.

O agente criou um design visual para a tela, mas como a especificação não detalhava completamente UX e identidade visual, ele tomou decisões próprias.

Isso reforça novamente algo extremamente importante:

A qualidade da saída depende diretamente da qualidade da especificação.

Se quisermos controle fino sobre UX, precisamos incluir:

- Figma
- tokens
- layout
- comportamento visual
- padrões de design

na própria spec.

Quanto menos explícitos formos, mais espaço deixamos para improvisação do agente.

Outro aprendizado importante da aula é perceber como o desenvolvimento começa a se aproximar de uma orquestração de equipes digitais.

Agora temos:

- múltiplos agentes
- múltiplos workspaces
- múltiplas branches
- múltiplos modelos
- múltiplos contextos

E o papel do desenvolvedor começa a mudar bastante.

Ele passa a atuar muito mais como:

- arquiteto
- orquestrador
- revisor
- integrador
- validador

do que apenas digitador de código.

Isso não reduz importância técnica.

Na verdade, aumenta ainda mais a necessidade de entendimento arquitetural.

Porque agora precisamos saber:

- como dividir tarefas
- como especificar corretamente
- como evitar colisões
- como controlar contexto
- como revisar entregas
- como integrar resultados

Ao final da aula, os dois agentes concluíram suas partes:

- front-end implementado
- back-end implementado
- DTOs compartilhados
- validações criadas
- componentes gerados
- testes executados

Tudo isso trabalhando simultaneamente em branches isoladas através de WorkTree.

E talvez esse seja o maior aprendizado deste capítulo.

O futuro do desenvolvimento assistido por IA não será apenas um único agente escrevendo código.

Será orquestração coordenada de múltiplos agentes especializados trabalhando paralelamente sob supervisão humana.

E isso exige:

- isolamento
- versionamento
- especificação
- arquitetura
- coordenação
- revisão

Ou seja, exatamente os fundamentos clássicos da engenharia de software, agora potencializados por IA.

Capítulo 4 - Code Review e Melhoria Contínua

Na última aula, nós demos um passo extremamente importante dentro do desenvolvimento assistido por IA: executamos múltiplos agentes em paralelo utilizando WorkTree, branches isoladas e OpenSpec Apply. Cada agente ficou responsável por uma parte específica da

aplicação e conseguimos desenvolver front-end e back-end simultaneamente sem colisão de código.

Agora chegamos em outro ponto fundamental da engenharia moderna: integração, revisão e melhoria contínua.

E esse capítulo é extremamente importante porque mostra algo que muitas pessoas esquecem quando começam a trabalhar com IA: gerar código não é o fim do processo.

Na verdade, geração automática é apenas o começo.

O verdadeiro trabalho de engenharia continua acontecendo depois da implementação.

Precisamos:

- revisar
- validar
- integrar
- testar
- refatorar
- melhorar
- documentar

Esse ponto é essencial.

Quanto mais IA utilizamos para acelerar implementação, mais importante se torna nossa capacidade de auditoria técnica.

Porque agora a velocidade de geração aumentou drasticamente.

Então, a qualidade da revisão precisa acompanhar essa velocidade.

A primeira coisa que fizemos na aula foi retornar para nossa branch principal de trabalho e criar uma branch específica de integração.

Esse detalhe é muito importante.

Mesmo trabalhando sozinhos no projeto, continuamos seguindo boas práticas clássicas de engenharia de software.

E isso é algo que eu venho reforçando bastante ao longo do módulo:

IA não elimina boas práticas.

Ela aumenta ainda mais a importância delas.

Porque agora o código é produzido muito rapidamente.

Então:

- versionamento
- revisão
- rastreabilidade
- organização
- isolamento
- merge controlado

se tornam ainda mais importantes.

Na prática, criamos uma branch específica chamada Integrate CFP Feature.

O objetivo dela era servir como espaço seguro para juntar tudo que os agentes produziram anteriormente.

Depois disso, realizamos o merge das duas branches:

- front-end
- back-end

E aqui aparece algo muito interessante.

Mesmo que os agentes tenham trabalhado em pastas físicas separadas através do WorkTree, para o Git isso continua sendo apenas branches normais.

Esse detalhe é extremamente elegante.

O WorkTree resolve o isolamento físico da máquina sem alterar o fluxo tradicional de versionamento.

Ou seja:

- continuamos usando merge
- continuamos usando branch
- continuamos usando commit
- continuamos usando revisão

A diferença é que agora conseguimos trabalhar com múltiplos ambientes simultaneamente de forma muito mais organizada.

Depois do merge, subimos a aplicação para fazer validação manual inicial.

Esse fluxo é extremamente importante.

Muita gente começa a confiar excessivamente em agentes e esquece da validação prática.

Mas aplicações reais precisam ser testadas funcionando.

Então executamos:

- front-end
- back-end
- integração
- submissão de formulário

Criamos uma submissão de palestra e validamos o fluxo completo.

E aqui aparece novamente um ponto muito importante sobre IA aplicada à engenharia.

Mesmo quando o código compila corretamente, isso não significa automaticamente que a feature está pronta.

Ainda precisamos validar:

- comportamento
- UX
- integração
- consistência
- semântica
- experiência real

Depois disso, realizamos uma operação extremamente importante dentro do fluxo do OpenSpec: o archive da especificação.

Esse é um conceito muito interessante do desenvolvimento guiado por especificação.

A especificação não morre depois que a feature é implementada.

Ela continua viva dentro do projeto.

O comando archive basicamente transforma aquela spec em baseline histórica da aplicação.

Isso é extremamente poderoso.

Porque futuras especificações passam a considerar automaticamente:

- requisitos anteriores
- arquitetura anterior
- decisões anteriores
- padrões anteriores
- comportamento já implementado

Esse detalhe melhora enormemente continuidade contextual do agente.

Na prática, o OpenSpec começa a construir uma espécie de memória evolutiva do projeto.

Isso resolve um problema histórico da engenharia de software.

Durante muitos anos, requisitos e documentação rapidamente ficavam desatualizados.

O código evoluía, mas a documentação morria.

Então as equipes paravam de confiar nela.

Com IA, isso muda bastante.

Agora conseguimos manter especificação e implementação muito mais sincronizadas.

Porque:

- a spec gera o código
- o código atualiza a spec
- a spec arquivada alimenta futuras evoluções

Isso cria um ciclo extremamente interessante de rastreabilidade viva.

Outro ponto importante é perceber que o archive não substitui Git.

Essa é uma distinção importante.

O Git versiona arquivos.

O OpenSpec versiona contexto semântico da feature.

São coisas complementares.

O Git sabe que arquivos mudaram.

A spec sabe por que mudaram.

Esse detalhe faz enorme diferença em projetos maiores.

Depois disso, começamos uma nova feature: criação do dashboard para visualizar palestras submetidas.

E aqui aparece novamente o fluxo completo do Spec-Driven Development.

Em vez de simplesmente pedir:

“Crie uma tela de dashboard.”

Nós especificamos:

- rota GET no NestJS
- retorno de SpeakerDTO
- componente Angular
- uso de HTTP Client

- uso de Signals
- manutenção da identidade visual
- consistência com os design tokens existentes
- navegação integrada

Perceba novamente o nível de detalhe.

Isso é extremamente importante.

Quanto mais madura fica a engenharia assistida por IA, mais percebemos que especificação de qualidade é uma habilidade central.

Outro detalhe extremamente importante foi reforçar consistência visual.

Na especificação deixamos explícito que o dashboard deveria respeitar rigorosamente:

- design system
- cores
- tokens
- identidade visual existente

Isso é muito importante porque os agentes tendem a improvisar visualmente quando deixamos espaço demais para interpretação.

Então precisamos constantemente orientar:

- o que reutilizar
- quais padrões seguir
- quais restrições respeitar

Outro ponto extremamente interessante foi observar o agente utilizando os MCPs durante geração da spec.

Ele consultou:

- Angular MCP
- Nx MCP

para entender corretamente o workspace antes de gerar implementação.

Isso reforça novamente um padrão muito importante do desenvolvimento moderno com IA:

Quanto melhor configuramos o ambiente do agente, melhor tende a ser a qualidade da saída.

MCPs reduzem:

- tentativa e erro

- desperdício de tokens
- comandos incorretos
- inconsistências arquiteturais

Esse detalhe será cada vez mais importante no futuro.

A configuração do ambiente passará a ser uma habilidade central do desenvolvedor.

Depois da criação da spec, novamente recebemos os três artefatos principais:

- proposal
- design
- tasks

E aqui aparece um ponto extremamente interessante.

O agente percebeu sozinho que seria necessário extrair tokens do formulário original para manter consistência entre as telas.

Esse comportamento aconteceu porque agora existe contexto acumulado:

- specs anteriores arquivadas
- design system existente
- histórico arquitetural

Isso mostra como desenvolvimento guiado por especificação melhora muito continuidade das decisões técnicas.

Depois da aprovação da spec, partimos para o apply.

E aqui utilizamos apenas um agente.

Esse detalhe também é importante.

Múltiplos agentes não são obrigatórios.

Eles são uma estratégia.

Precisamos saber quando vale a pena paralelizar e quando uma única execução é suficiente.

Essa decisão depende de:

- complexidade
- tamanho da feature
- isolamento possível
- dependências
- custo computacional

Outro ponto extremamente interessante da aula foi observar o agente corrigindo seu próprio comportamento durante execução.

Inicialmente ele tentou executar testes usando `npm run test` tradicional.

Mas como o projeto estava organizado com Nx, aquilo não funcionou corretamente.

Então ele analisou o workspace e ajustou automaticamente os comandos.

Esse tipo de comportamento mostra como MCP e contexto estruturado melhoram muito capacidade de autocorreção do agente.

Sem contexto suficiente, ele provavelmente ficaria:

- tentando comandos errados
- desperdiçando tokens
- gerando loops

Quanto melhor o ambiente estiver preparado, mais eficiente o agente se torna.

Outro detalhe extremamente interessante foi o uso do navegador automatizado pelo próprio Antigravity.

O agente:

- abriu a aplicação
- preencheu formulário
- submeteu dados
- navegou até dashboard
- validou interface
- inspecionou DOM
- executou asserts

Esse fluxo aproxima bastante IA de testes funcionais automatizados.

Claro que ainda precisamos validar manualmente muita coisa.

Mas isso reduz enormemente trabalho operacional repetitivo.

Outro ponto importante foi perceber que mesmo após toda automação, continuamos responsáveis pela revisão final.

O agente gera:

- relatório
- resumo
- lista de alterações
- arquivos modificados

Mas quem aprova continua sendo o desenvolvedor.

E isso talvez seja um dos pontos mais importantes de todo o módulo.

O papel do desenvolvedor não desaparece.

Ele evolui.

Agora dedicamos menos energia para:

- código repetitivo
- boilerplate
- tarefas operacionais

E mais energia para:

- arquitetura
- validação
- revisão
- integração
- consistência
- qualidade
- experiência do usuário

Depois disso, realizamos novamente o archive da especificação do dashboard.

Esse detalhe é extremamente importante.

Cada feature agora passa a deixar uma trilha histórica estruturada dentro do projeto.

Isso cria uma documentação viva extremamente valiosa para futuras evoluções.

Por fim, realizamos a limpeza do ambiente.

Removemos:

- WorkTrees
- branches temporárias
- ambientes auxiliares

Esse detalhe também é importante.

Ambientes temporários precisam ser gerenciados corretamente para evitar acúmulo de lixo operacional no projeto.

Ao final da aula, tínhamos:

- front-end integrado

- back-end integrado
- dashboard funcional
- especificações arquivadas
- histórico rastreável
- múltiplos merges concluídos
- agentes validados

Tudo isso utilizando uma combinação de:

- OpenSpec
- MCP
- Nx
- Angular
- NestJS
- WorkTree
- agentes paralelos
- revisão humana

E talvez esse seja o maior aprendizado deste capítulo.

IA não elimina engenharia de software tradicional.

Ela reforça ainda mais a importância de:

- revisão
- arquitetura
- rastreabilidade
- versionamento
- especificação
- integração contínua
- melhoria contínua

Porque agora conseguimos produzir software muito mais rápido.

E justamente por isso precisamos elevar também nossa maturidade de engenharia.

Capítulo 5 - Escala na Nuvem Orquestração e Cadastro de Eventos com Google Jules

Nas últimas aulas, nós trabalhamos com agentes de IA dentro do nosso próprio ambiente de desenvolvimento. Usamos AntigraVity, configuramos MCPs, criamos especificações com OpenSpec, executamos múltiplos agentes em paralelo e isolamos cada contexto com

Git WorkTree. Até esse momento, o agente estava atuando como um copiloto local. Ele trabalhava junto comigo, na minha máquina, usando meu ambiente, meus arquivos e minha sessão ativa.

Agora damos um passo diferente. Em vez de usar apenas agentes acoplados ao meu ambiente local, começamos a trabalhar com agentes independentes, assíncronos, que executam tarefas na nuvem. Esse é um modelo muito importante porque muda a dinâmica do desenvolvimento. Eu deixo de tratar o agente apenas como alguém que está fazendo pair programming comigo e começo a tratá-lo como outro desenvolvedor trabalhando em paralelo.

No fluxo local, se eu desligo minha máquina, o agente para. No fluxo assíncrono, a ferramenta sobe o próprio ambiente, clona o repositório, cria uma branch, executa a tarefa, roda testes e abre um pull request para revisão. Isso aproxima bastante o uso da IA do fluxo real de uma equipe de desenvolvimento.

Existem várias ferramentas nesse mercado. Uma das mais conhecidas é o Devin. Nesta aula, usamos o Google Jules. O importante, como sempre, não é decorar a ferramenta específica, mas entender o fundamento. O conceito é delegar tarefas bem definidas para um agente autônomo, deixar ele trabalhar fora da minha máquina e depois revisar o resultado como eu revisaria o pull request de uma pessoa da equipe.

O Jules se conecta ao GitHub como um GitHub App. Depois de autorizado, ele consegue acessar os repositórios permitidos, criar branches, realizar alterações e abrir pull requests. Ele também pode trabalhar a partir de issues do GitHub ou integrações com outras ferramentas, como Jira. No nosso exemplo, usamos um fluxo direto: selecionamos o repositório da plataforma CFP e criamos uma nova tarefa.

Antes de executar qualquer coisa, existe um ponto importante: configuração do repositório. Como o Jules cria um ambiente efêmero na nuvem, ele precisa saber como preparar o projeto. Em um projeto simples como o nosso, um monorepo Node com Nx, Angular e NestJS, praticamente não precisamos configurar nada. Mas em projetos reais isso muda bastante. Pode ser necessário configurar scripts de setup, instalar dependências específicas, definir variáveis de ambiente e cadastrar secrets.

Esse cuidado é essencial. Eu não devo colocar API keys, senhas ou credenciais diretamente em scripts ou arquivos de configuração. Tudo que for sensível precisa ficar em secrets. O raciocínio é o mesmo de onboarding de uma pessoa nova no time: se eu preciso documentar como subir o ambiente para um desenvolvedor, também preciso documentar isso para o agente.

Na tarefa da aula, pedimos para o Jules implementar a feature de cadastro de local de evento dentro do nosso monorepo Nx. O prompt foi direto, mas ainda assim detalhado. Definimos o papel do agente como desenvolvedora full stack sênior e especificamos várias diretrizes obrigatórias.

Primeiro, reforçamos o design system. O novo formulário deveria manter rigorosamente o padrão visual, cores e design tokens já aplicados no formulário de Call for Papers. Esse ponto é importante porque, quando não delimitamos identidade visual, o agente pode criar uma interface funcional, mas desalinhada do restante do sistema.

Depois, definimos a navegação. O menu deveria conter exatamente três opções: cadastro de eventos, cadastro de palestras e dashboard. Aqui já incluímos uma pequena refatoração de front-end. Em um fluxo ideal de produção, poderíamos separar isso em tarefas menores. Para fins didáticos, deixamos tudo em uma única tarefa para demonstrar o funcionamento do agente assíncrono.

Na arquitetura, pedimos a criação de um novo DTO compartilhado para evento, dentro da biblioteca comum do monorepo. Também pedimos a implementação no back-end NestJS, com controller e service para receber POST e salvar os dados em memória. No front-end Angular, solicitamos a criação do formulário usando Reactive Forms e Signals. Por fim, pedimos que ele criasse uma branch, implementasse o código e abrisse um pull request para revisão.

Esse modelo exige uma mudança de postura. Em agentes locais, é comum interagir bastante, ajustar instruções, conversar, revisar cada passo. Em agentes assíncronos, o ideal é passar uma tarefa específica, bem descrita, e deixar o agente trabalhar. Quanto mais objetiva e delimitada for a tarefa, melhor será o resultado.

Depois que a tarefa começa, o Jules sobe uma VM, clona o repositório, prepara o ambiente e monta um plano. Ele descreve que vai criar o DTO, implementar a API de eventos, criar o componente de cadastro no front-end, aplicar o design system, atualizar navegação, executar testes e abrir o PR. Esse plano é importante porque mostra o que o agente pretende fazer antes de finalizar a entrega.

Durante a execução, ele altera arquivos, roda testes e registra evidências. Uma parte interessante é que ele consegue abrir o navegador dentro do próprio ambiente e gerar evidências visuais da interface, inclusive imagens ou pequenos registros do teste. Isso facilita bastante a revisão, porque eu consigo ver não apenas os arquivos alterados, mas também o comportamento da aplicação.

Outro ponto interessante é que o agente faz uma espécie de revisão do próprio trabalho. Ele aponta observações, registra logs e mostra se os testes passaram. Claro que isso não substitui code review humano. Eu ainda preciso revisar o PR, testar localmente e conferir se a implementação faz sentido. Mas essa camada ajuda a acelerar muito a triagem.

Depois que o PR foi criado, fizemos o fluxo normal de revisão. Baixamos a branch localmente, executamos a aplicação com Nx, abrimos o sistema e testamos o cadastro de eventos. A tela apareceu, o formulário funcionou e o fluxo básico foi validado. Também observamos que alguns detalhes visuais poderiam ser melhorados, como o menu. Esse é

exatamente o tipo de feedback que eu poderia deixar no próprio pull request para o Jules corrigir em uma nova iteração.

Esse é um ponto muito importante: o agente assíncrono deve ser tratado como outro desenvolvedor. Ele abre PR, eu reviso, comento, peço ajustes ou aprovo. Não devo dar permissões irrestritas para ele alterar produção, mexer direto na main ou acessar ambientes sensíveis. O fluxo seguro continua sendo branch, pull request, revisão e merge.

Ao final, aprovamos o PR e fizemos o merge da feature. Com isso, o sistema passou a ter cadastro de eventos, atualização de navegação e integração com a estrutura existente.

O aprendizado principal desta aula é que não existe uma única forma de usar IA no desenvolvimento. Podemos usar ferramentas conversacionais para refinar requisitos, agentes locais para pair programming, múltiplos agentes com WorkTree para paralelizar tarefas e agentes assíncronos na nuvem para delegar atividades específicas. A produtividade aparece quando combinamos essas estratégias com critério, sempre mantendo boas práticas de engenharia, revisão humana e controle do ambiente.

Módulo 04 - Automação E2E e QA AI-Native

Capítulo 1 - Arquitetura de QA no Nx Cypress Tradicional e Geração com OpenSpec

Até este momento do módulo, nós utilizamos inteligência artificial para acelerar praticamente todas as etapas do desenvolvimento: refinamento de requisitos, geração de componentes, integração front-end e back-end, criação de design systems, prototipação, execução paralela de agentes e até desenvolvimento assíncrono na nuvem com ferramentas como Google Jules.

Agora começamos uma nova etapa extremamente importante dentro da engenharia de software moderna: qualidade e testes automatizados.

E aqui existe um ponto fundamental que precisa ficar muito claro.

IA não elimina a necessidade de testes.

Na verdade, quanto mais rápido passamos a gerar código utilizando agentes, maior se torna a importância de garantir segurança, regressão e validação automatizada da aplicação.

Esse é um dos pontos mais importantes da maturidade no desenvolvimento assistido por IA.

Porque agora conseguimos produzir software muito mais rápido.

Então também precisamos validar software muito mais rápido.

É justamente aqui que entram os testes funcionais, também conhecidos como testes end-to-end.

Enquanto testes unitários validam pequenas partes isoladas do sistema e testes de integração verificam comunicação entre componentes, os testes end-to-end trabalham do ponto de vista do usuário.

Esse detalhe é extremamente importante.

No teste funcional, eu não estou apenas chamando métodos internos ou verificando retorno de funções.

Eu estou automatizando a experiência real de uso da aplicação.

Ou seja:

- abrir navegador
- acessar rota
- preencher formulário
- clicar botão
- validar mensagens
- verificar comportamento visual

Tudo isso simulando um usuário real interagindo com o sistema.

Esse tipo de teste possui enorme valor porque ajuda a detectar problemas que muitas vezes não aparecem em testes unitários.

Por exemplo:

- erro de integração entre front-end e back-end
- falha de navegação
- problema de validação visual
- quebra de formulário
- inconsistência de UX
- regressão de fluxo crítico

Além disso, testes funcionais são extremamente importantes para regressão.

Quando adicionamos novas funcionalidades, refatoramos código ou alteramos arquitetura, conseguimos rodar os testes end-to-end para verificar se os fluxos antigos continuam funcionando corretamente.

Isso reduz muito o risco de quebrar funcionalidades existentes sem perceber.

Outro ponto importante é entender o papel do QA moderno nesse cenário.

Tradicionalmente, muitas equipes possuem profissionais especializados em qualidade que conhecem profundamente:

- regras de negócio
- comportamento do sistema
- cenários críticos
- casos de erro
- fluxos funcionais

Esses profissionais frequentemente escrevem e executam testes manualmente.

E isso continua sendo extremamente valioso.

Mas agora conseguimos usar IA para acelerar enormemente a automação desses testes.

Ou seja, o profissional de QA passa a dedicar menos tempo à execução repetitiva e mais tempo à criação de cenários inteligentes de validação.

Esse é um padrão que apareceu diversas vezes ao longo do módulo:

IA reduz trabalho operacional e amplia capacidade estratégica do profissional.

Nesta aula escolhemos utilizar o Cypress.

O Cypress é um framework extremamente popular e maduro para testes end-to-end no ecossistema JavaScript.

Ele é amplamente utilizado em aplicações Angular, React, Vue e diversos outros frameworks web modernos.

O funcionamento dele é relativamente simples de entender.

O Cypress sobe a aplicação, abre um navegador real ou headless e executa interações simulando um usuário.

Esse detalhe é importante.

Não estamos testando apenas funções internas.

Estamos testando a experiência real da interface.

Outro ponto interessante é que a sintaxe do Cypress lembra bastante frameworks de testes unitários.

Temos:

- preparação
- execução
- assertions

Ou seja:

- preparar estado
- executar ação
- validar resultado

Esse padrão praticamente aparece em qualquer tipo de teste automatizado.

Como estamos utilizando Nx Monorepo, a configuração do Cypress fica ainda mais interessante.

Se estivéssemos trabalhando com projetos separados, normalmente precisaríamos:

- subir back-end
- subir front-end
- iniciar Cypress
- executar testes

Tudo manualmente.

Mas o Nx simplifica bastante isso.

Ao instalar o plugin do Cypress no workspace Nx, ele já entende:

- aplicações existentes
- scripts de execução
- dependências
- ambiente integrado

Isso facilita muito o fluxo de execução.

Outro detalhe importante é perceber novamente como o monorepo melhora contexto para os agentes.

Como o front-end, back-end e contratos estão todos no mesmo repositório, o agente possui visão completa do sistema.

Isso melhora muito assertividade da geração dos testes.

Depois de instalar o plugin do Cypress no Nx, partimos novamente para o OpenSpec.

E aqui aparece algo extremamente interessante.

Nós não utilizamos o Spec-Driven Development apenas para gerar funcionalidades.

Também podemos utilizá-lo para gerar testes.

Esse detalhe é muito importante.

O OpenSpec é extremamente versátil.

Ele funciona como mecanismo estruturado de especificação.

E testes também são requisitos.

Então criamos uma nova spec chamada `createEventTest`.

O objetivo dessa spec era gerar testes funcionais para a tela de cadastro de eventos.

Mais uma vez seguimos o padrão de especificação:

- objetivo
- cenários
- regras
- restrições

E aqui definimos dois cenários principais:

- cenário de sucesso
- cenário de erro

No cenário de sucesso, o teste deveria:

- navegar até `event-new`
- preencher formulário
- clicar em `submit`
- validar mensagem de sucesso

No cenário de erro, deveria:

- acessar rota
- enviar formulário vazio
- validar mensagens de erro

Outro detalhe extremamente importante foi a restrição explícita sobre o framework.

Pedimos para utilizar exclusivamente a sintaxe tradicional do Cypress:

- `get`
- `contains`
- `should`

E proibimos bibliotecas externas nesse momento.

Esse tipo de restrição é extremamente importante no trabalho com agentes.

Quanto mais explícitos formos sobre o que queremos ou não queremos, menor a chance de comportamento inesperado.

Depois da geração da spec, o OpenSpec criou:

- proposal
- design
- tasks

Mais uma vez perceberemos como o fluxo de desenvolvimento guiado por especificação se adapta facilmente para QA.

A especificação agora descreve comportamento esperado dos testes.

Outro detalhe extremamente interessante foi observar o agente utilizando BDD implicitamente.

A spec acabou sendo estruturada utilizando lógica muito parecida com:

- dado
- quando
- então

Esse padrão é extremamente comum em equipes de QA e produto.

Ele ajuda bastante na descrição clara dos cenários funcionais.

Depois disso, utilizamos o OpenSpec Apply para implementar a spec.

E aqui apareceu um detalhe muito interessante.

O agente detectou automaticamente que o projeto já possuía Playwright configurado originalmente.

Então ele propôs coexistência entre Cypress e Playwright dentro do workspace.

Esse tipo de análise contextual é extremamente poderoso.

O agente não apenas escreve código.

Ele também interpreta arquitetura existente.

Depois da aprovação, ele:

- configurou Cypress no Nx
- ajustou ESLint
- criou projeto E2E
- gerou os testes
- executou os testes automaticamente

Outro ponto importante foi observar o Cypress funcionando em modo headless e também visual.

Quando executamos os testes normalmente pelo Nx, o navegador roda sem interface gráfica.

Mas também conseguimos abrir o Cypress visualmente para assistir os testes acontecendo em tempo real.

Esse recurso é extremamente útil para debugging e entendimento do comportamento dos testes.

Ao executar visualmente, conseguimos observar:

- abertura da tela
- preenchimento dos campos
- clique no botão
- validação das mensagens

Tudo automatizado.

Depois analisamos o código gerado.

E aqui aparece um aprendizado extremamente importante sobre testes end-to-end.

A estrutura do teste é relativamente simples:

- beforeEach
- preparação
- ação
- assertion

No beforeEach, acessamos a rota da aplicação.

Depois, no cenário de sucesso:

- localizamos campos
- digitamos valores
- clicamos no botão
- verificamos mensagem de sucesso

No cenário de erro:

- submetemos vazio
- verificamos mensagens de validação

Outro ponto extremamente interessante é que o agente conseguiu capturar corretamente as mensagens reais do sistema.

Isso aconteceu porque o monorepo forneceu contexto completo.

O agente tinha acesso ao:

- front-end
- back-end
- validações
- mensagens
- componentes

Esse é um dos grandes benefícios do desenvolvimento assistido por IA em monorepo.

Quanto mais contexto estruturado fornecemos ao agente, melhor tende a ser a qualidade da geração.

Mas aqui aparece também um ponto extremamente importante sobre fragilidade de testes funcionais.

No exemplo gerado, os seletores estavam utilizando classes CSS e mensagens textuais diretamente.

Isso funciona.

Mas pode gerar o que chamamos de flaky tests.

Ou seja, testes frágeis que quebram facilmente por pequenas mudanças não funcionais.

Por exemplo:

- alteração de texto
- mudança de classe CSS
- pequeno ajuste visual

Nesse caso, o fluxo continua funcionando corretamente para o usuário, mas o teste quebra mesmo assim.

Esse problema é extremamente comum em testes E2E.

Por isso, existe uma prática muito importante chamada instrumentalização do front-end.

A ideia é adicionar atributos específicos para automação, como:

- data-cy
- data-testid
- identificadores semânticos

Assim, os testes deixam de depender de detalhes visuais frágeis.

Esse ponto é muito importante porque novamente conecta QA com arquitetura front-end.

Testabilidade também faz parte do design da aplicação.

Outro aprendizado importante da aula é perceber como IA pode ajudar profissionais de QA mesmo que eles não sejam especialistas profundos em automação.

Muitas vezes o QA conhece perfeitamente:

- regras de negócio
- comportamento esperado
- cenários críticos

Mas não domina completamente frameworks de automação.

Agora conseguimos usar agentes para transformar cenários funcionais em testes automatizados muito mais rapidamente.

Claro que revisão continua sendo importante.

Mas a barreira de entrada para automação diminui bastante.

E isso talvez seja um dos maiores impactos da IA dentro da qualidade de software.

Ela democratiza muito mais a capacidade de gerar automação.

Ao final da aula, tínhamos:

- Cypress configurado no Nx
- coexistência com Playwright
- spec de testes criada
- cenários automatizados
- execução headless funcionando
- execução visual funcionando
- testes de sucesso e erro implementados

Tudo isso utilizando:

- OpenSpec
- Nx

- Cypress
- agentes de IA
- Spec-Driven Development

E talvez o maior aprendizado deste capítulo seja perceber que testes também podem fazer parte do fluxo moderno de desenvolvimento assistido por IA.

Não usamos IA apenas para gerar funcionalidades.

Também usamos IA para gerar segurança, regressão e validação automatizada da experiência do usuário.

Capítulo 2 - O Fim da Fragilidade Asserções Semânticas com Cypress

Na aula anterior, nós configuramos o Cypress dentro do nosso monorepo Nx, utilizamos OpenSpec para gerar testes funcionais automaticamente e começamos a estruturar uma arquitetura moderna de QA utilizando agentes de IA.

Criamos cenários de sucesso e erro, automatizamos execução no navegador e entendemos como testes end-to-end funcionam do ponto de vista do usuário.

Mas agora chegamos em um problema extremamente importante dentro do universo de testes funcionais: fragilidade.

Esse talvez seja um dos maiores problemas históricos dos testes E2E.

Muitas equipes começam extremamente motivadas criando automação, mas depois de alguns meses os testes começam a quebrar constantemente.

E aí nasce uma situação muito comum:

- o time perde confiança nos testes
- os testes começam a ser ignorados
- pipelines falham frequentemente
- surgem falsos negativos
- manutenção vira um pesadelo

Esse tipo de teste costuma receber o nome de flaky test.

Ou seja, um teste instável, quebradiço e sensível demais a pequenas mudanças.

E aqui existe um detalhe extremamente importante.

Muitas vezes o sistema continua funcionando perfeitamente para o usuário, mas o teste quebra mesmo assim.

Por exemplo:

- uma classe CSS muda
- um botão troca de ID
- uma mensagem recebe pontuação diferente
- um elemento muda de posição

Do ponto de vista funcional, nada realmente importante mudou.

Mas o teste automatizado quebra.

Esse problema sempre existiu no universo de automação.

E agora começamos a explorar formas modernas de reduzir essa fragilidade utilizando IA.

Nesta aula utilizamos um recurso extremamente interessante do ecossistema Cypress chamado `cy.prompt`.

Esse detalhe é importante.

O recurso não pertence exatamente ao framework puro local do Cypress.

Ele depende da integração com os serviços de nuvem do Cypress.

Por isso precisamos estar autenticados na plataforma.

O que torna esse recurso tão interessante é justamente a mudança de paradigma.

Na aula anterior, nós utilizamos IA para gerar o código do teste.

Agora fazemos algo diferente.

Passamos a descrever o comportamento do teste semanticamente e deixamos o próprio Cypress gerar e adaptar a implementação durante a execução.

Esse detalhe é extremamente importante.

A diferença parece pequena, mas muda bastante o comportamento do sistema.

Na abordagem anterior, utilizávamos:

- OpenSpec
- agentes
- geração de código Cypress
- execução tradicional

Agora fazemos algo muito mais próximo de prompt engineering aplicado diretamente ao framework de testes.

Em vez de escrever:

- `cy.get`
- `cy.contains`
- `cy.should`

manual ou automaticamente, passamos a escrever instruções semânticas.

Por exemplo:

- digite auditório Oracle no campo nome
- preencha endereço
- coloque capacidade 500
- clique no botão de salvar
- verifique mensagem de sucesso

Ou seja, descrevemos intenção e não implementação.

Esse conceito é extremamente poderoso.

E ele conecta diretamente com uma tendência muito forte da engenharia moderna: abstração semântica.

Ao longo da evolução do desenvolvimento de software, sempre buscamos subir o nível de abstração.

Primeiro escrevíamos assembly.

Depois linguagens de alto nível.

Depois frameworks.

Depois ORMs.

Depois low-code.

Agora começamos a trabalhar também com automação baseada em intenção.

E isso aparece claramente aqui.

No exemplo da aula, criamos um novo arquivo de teste chamado `event-registration-ai.ts`.

O objetivo era validar exatamente o mesmo fluxo de cadastro de eventos que já havíamos testado anteriormente.

Mas agora usando `cy.prompt`.

A estrutura geral do teste continua parecida.

Ainda temos:

- describe
- cenário
- assertions

Mas o corpo da automação muda bastante.

Em vez de seleccionar elementos diretamente com CSS ou IDs, usamos prompts semânticos.

Isso faz com que o Cypress utilize internamente uma LLM para interpretar intenção e gerar o código correspondente.

Outro detalhe extremamente importante é perceber que não existe “mágica”.

Esse é um padrão muito importante ao longo de todo o módulo.

Sempre que analisamos ferramentas modernas de IA mais profundamente, percebemos que normalmente elas funcionam através de:

- prompts
- contexto
- geração intermediária
- execução automatizada

E aqui acontece exatamente isso.

O Cypress não executa diretamente o prompt como se fosse um agente mágico manipulando o navegador de maneira abstrata.

Na prática, ele usa o prompt para gerar código Cypress tradicional.

Isso fica muito claro quando abrimos a aba Code dentro do Cypress.

Lá conseguimos visualizar exatamente o código gerado.

E o mais interessante é perceber que ele gera praticamente a mesma estrutura que criamos anteriormente:

- visit
- get
- type
- click
- contains

Ou seja, por baixo dos panos, continua sendo Cypress tradicional.

A diferença é que agora existe uma camada semântica intermediária responsável por criar e adaptar a implementação.

Esse detalhe é extremamente importante porque ajuda a entender limites e capacidades reais da ferramenta.

Depois da primeira execução, percebemos outro comportamento interessante: cache.

A primeira vez demora mais porque o modelo precisa interpretar os prompts e gerar o código.

Mas nas execuções seguintes o Cypress reutiliza o código já criado.

Isso reduz:

- custo computacional
- consumo de tokens
- tempo de execução

Esse detalhe é importante porque mostra novamente como ferramentas modernas de IA começam a incorporar otimizações práticas para uso real em escala.

Depois disso, começamos a explorar o principal benefício da abordagem semântica: self-healing.

Self-healing significa autocorreção.

Esse talvez seja um dos recursos mais interessantes da automação moderna baseada em IA.

Para demonstrar isso, fizemos uma alteração proposital no front-end.

Mudamos a classe CSS do botão de submit.

No teste tradicional da aula anterior, isso imediatamente quebrou a automação.

Porque o teste dependia diretamente da classe antiga.

Então o Cypress não conseguia mais localizar o botão.

Esse é exatamente o comportamento clássico dos flaky tests.

Uma pequena alteração estrutural quebra toda automação.

Depois executamos o teste baseado em prompt.

E aqui aconteceu algo extremamente interessante.

O teste continuou funcionando.

O Cypress detectou que o seletor anterior não existia mais e utilizou o contexto semântico para reinterpretar a interface e gerar um novo caminho válido.

Ou seja, ele se autocorrigiu.

Esse é o conceito de self-healing.

Em vez de depender rigidamente de seletores fixos, o sistema tenta compreender intenção funcional.

Isso reduz drasticamente fragilidade da automação.

E talvez esse seja um dos pontos mais importantes desta aula.

A IA começa a mudar não apenas velocidade de geração de testes, mas também resiliência dos testes.

Esse detalhe é extremamente poderoso.

Porque manutenção de automação sempre foi um dos maiores custos da engenharia de QA.

Boa parte do esforço histórico em testes E2E não estava na criação inicial, mas sim na manutenção contínua.

Quanto mais o sistema evolui, mais os testes quebram.

Então recursos de self-healing podem reduzir enormemente esse custo operacional.

Mas aqui aparece um ponto extremamente importante de arquitetura e estratégia técnica.

Toda solução possui trade-offs.

Ao usar cy.prompt, ganhamos:

- automação semântica
- self-healing
- menor fragilidade
- menos dependência de seletores rígidos

Mas também criamos dependência da plataforma Cypress Cloud.

Esse é um lock-in tecnológico.

Esse detalhe é extremamente importante.

O teste agora depende:

- da nuvem do Cypress
- do modelo utilizado pela plataforma
- da integração proprietária

Então precisamos analisar cuidadosamente vantagens e desvantagens.

Esse é exatamente o tipo de decisão que arquitetos e líderes técnicos precisam tomar.

Outro ponto importante é perceber que existem alternativas para reduzir fragilidade sem depender necessariamente de IA proprietária.

Uma prática extremamente comum em automação moderna é instrumentar o front-end com atributos específicos para testes.

Por exemplo:

- data-cy
- data-testid
- data-qa

Esses atributos funcionam como identificadores estáveis exclusivos para automação.

Assim, o teste não depende:

- de CSS
- de layout
- de classes visuais
- de IDs frágeis

Isso melhora muito estabilidade da automação.

Inclusive, esse tipo de instrumentação pode ser facilmente automatizado utilizando agentes de IA.

Podemos pedir para um agente:

- analisar componentes
- adicionar data-cy automaticamente
- padronizar identificadores
- refatorar testes existentes

Esse detalhe mostra novamente como IA pode atuar não apenas criando testes, mas também melhorando arquitetura de testabilidade da aplicação.

Outro aprendizado extremamente importante da aula é perceber que profissionais de QA agora começam a trabalhar em um nível muito mais semântico.

Em vez de precisar dominar profundamente detalhes técnicos do framework, eles podem focar mais em:

- regras de negócio
- cenários
- comportamento esperado
- fluxos críticos

E utilizar IA para traduzir isso em automação executável.

Claro que conhecimento técnico continua extremamente importante.

Mas a barreira operacional diminui bastante.

Ao final da aula, entendemos três conceitos extremamente importantes:

Primeiro: testes tradicionais podem ser frágeis quando dependem excessivamente de detalhes estruturais.

Segundo: automação semântica baseada em IA pode reduzir bastante essa fragilidade através de self-healing.

Terceiro: toda solução possui trade-offs arquiteturais que precisam ser analisados cuidadosamente.

E talvez esse seja o maior aprendizado deste capítulo.

A IA não está apenas acelerando criação de software.

Ela também está começando a transformar profundamente a forma como pensamos qualidade, automação e resiliência de testes dentro da engenharia de software moderna.

Capítulo 3 - O Agente de QA Autônomo com Playwright MCP

Nas últimas aulas, nós exploramos diferentes estratégias para criação de testes end-to-end utilizando inteligência artificial. Primeiro utilizamos OpenSpec junto com Cypress tradicional para gerar código de automação. Depois vimos o Cypress Prompt, usando prompts semânticos e self-healing para reduzir fragilidade dos testes.

Agora damos mais um passo importante dentro da evolução do QA assistido por IA.

Nesta aula, deixamos de usar IA apenas para gerar código de teste e passamos a usar IA para executar diretamente os testes como um agente autônomo.

Esse detalhe é extremamente importante.

Até agora, o fluxo era:

- IA gera código
- framework executa código

Agora o fluxo muda bastante:

- IA interpreta o cenário
- IA manipula o navegador
- IA executa o fluxo
- IA valida comportamento

Ou seja, o próprio agente passa a atuar quase como um usuário real navegando na aplicação.

E aqui entra uma ferramenta extremamente importante: Playwright MCP.

O Playwright já é um framework extremamente consolidado no universo de testes E2E modernos. Ele possui capacidades muito parecidas com Cypress:

- automação de navegador
- testes funcionais
- execução headless
- assertions
- integração CI/CD

Inclusive, quando criamos um projeto Angular moderno, normalmente o Playwright já aparece como uma das opções oficiais para testes end-to-end.

Mas nas versões mais recentes surgiu algo extremamente interessante: integração do Playwright com MCP.

E isso muda bastante a forma como agentes conseguem interagir com browsers.

O MCP do Playwright não serve apenas para ensinar melhores práticas ao agente.

Ele também entrega ferramentas reais de manipulação do navegador.

Esse detalhe é extremamente poderoso.

Quando configuramos o MCP do Playwright dentro da nossa IDE agêntica, o agente passa a ganhar capacidades como:

- abrir páginas
- clicar elementos
- preencher formulários

- capturar screenshots
- executar JavaScript
- ler console
- interagir com popups
- navegar pela interface

Ou seja, o agente deixa de apenas escrever testes e passa a controlar efetivamente um browser.

Isso aproxima muito IA de automação real de usuário.

Outro detalhe extremamente importante é que esse modelo não serve apenas para QA.

Ao longo da aula mostramos algo muito interessante: Playwright MCP também pode funcionar como ferramenta de RPA.

RPA significa Robotic Process Automation.

Basicamente:

- automação de tarefas repetitivas
- manipulação de sistemas
- navegação automatizada
- preenchimento de fluxos operacionais

Isso abre possibilidades extremamente grandes.

Por exemplo:

- automatizar sistemas legados
- manipular ferramentas sem API
- preencher portais internos
- integrar plataformas antigas
- executar rotinas administrativas

E tudo isso usando linguagem natural orientando um agente.

Esse ponto é extremamente importante.

O browser passa a funcionar como camada universal de integração.

Mesmo sistemas antigos sem API ainda possuem interface web.

Então conseguimos automatizar comportamento diretamente pela interface.

Na aula, começamos verificando se o Playwright já estava instalado corretamente no projeto.

Como nosso workspace Nx já havia sido criado anteriormente com Playwright habilitado, praticamente não tivemos trabalho adicional.

Esse detalhe mostra uma vantagem importante do Nx Monorepo.

A arquitetura centralizada facilita bastante integração entre:

- front-end
- back-end
- testes
- agentes
- ferramentas de automação

Depois disso, configuramos o MCP do Playwright no AntigraVity.

Mais uma vez seguimos exatamente o mesmo padrão já utilizado anteriormente com:

- Angular MCP
- Nx MCP
- outros servidores MCP

Esse padrão começa a aparecer repetidamente ao longo do módulo inteiro.

Quanto melhor configuramos contexto e ferramentas do agente, melhor tende a ser a qualidade da execução.

E aqui aparece um ponto extremamente importante do futuro do desenvolvimento assistido por IA:

Configuração do ambiente se torna uma habilidade crítica.

Porque agora não basta apenas escrever código.

Precisamos preparar agentes com:

- ferramentas corretas
- documentação correta
- permissões corretas
- contexto correto

Depois da configuração, conseguimos visualizar todas as tools fornecidas pelo MCP do Playwright.

E aqui percebemos claramente como o agente ganha “superpoderes” sobre o navegador.

Ele consegue:

- navegar páginas

- analisar DOM
- executar ações
- interagir com elementos
- capturar estados da interface

Tudo isso programaticamente.

Outro detalhe extremamente interessante é que agora não criamos nenhum código de teste tradicional.

Esse ponto muda completamente o paradigma da automação.

Não escrevemos:

- describe
- it
- beforeEach
- cy.get
- assertions

Nada disso.

Em vez disso, criamos apenas um prompt operacional.

E esse prompt funciona quase como instrução para um operador humano.

No exemplo da aula, pedimos:

- subir ambiente Nx
- iniciar front-end e API
- acessar aplicação
- encontrar formulário
- preencher dados realistas
- submeter formulário
- validar mensagem de sucesso

Perceba como o prompt é extremamente próximo de linguagem natural.

Esse talvez seja um dos pontos mais interessantes dessa abordagem.

O foco deixa de ser implementação técnica do teste e passa a ser intenção funcional.

Outro detalhe extremamente importante é que o próprio agente utilizou MCPs adicionais durante a execução.

Ele primeiro consultou o Nx MCP para entender como subir o ambiente.

Depois utilizou o Playwright MCP para controlar o browser.

Ou seja, múltiplos MCPs começaram a trabalhar juntos dentro do mesmo fluxo.

Esse detalhe mostra uma direção extremamente importante do futuro dos agentes:

Orquestração de ferramentas especializadas.

Cada MCP funciona como uma espécie de “extensão cognitiva” para o agente.

O agente sozinho possui raciocínio geral.

Os MCPs entregam capacidades específicas.

Na execução prática, conseguimos assistir o agente navegando na aplicação exatamente como um usuário humano:

- abriu menu
- navegou até cadastro
- encontrou formulário
- preencheu dados
- clicou botão
- validou resultado

Tudo isso sem código explícito de automação.

Outro ponto extremamente interessante foi observar os artifacts gerados durante a execução.

O Playwright registrou:

- ações
- snapshots
- logs
- sequência de navegação
- interações executadas

Isso melhora bastante rastreabilidade e debugging.

Também percebemos outro detalhe importante: o agente acabou criando uma espécie de “log procedural” do que executou.

Isso pode ser reutilizado posteriormente.

E aqui surge uma possibilidade extremamente interessante.

No futuro, agentes poderão:

- aprender fluxos recorrentes
- otimizar navegação

- reutilizar caminhos
- criar automações híbridas

Esse tipo de comportamento aproxima bastante agentes de sistemas semi-autônomos reais.

Outro ponto importante da aula foi a comparação entre as três estratégias de testes que trabalhamos até agora.

Primeira abordagem:

IA gera código Cypress tradicional.

Vantagens:

- barato para executar
- roda normalmente em CI/CD
- sem dependência contínua de IA
- testes rápidos

Desvantagens:

- fragilidade
- flaky tests
- dependência de seletores

Segunda abordagem:

Cypress Prompt com self-healing.

Vantagens:

- automação semântica
- autocorreção
- menos fragilidade

Desvantagens:

- lock-in na nuvem Cypress
- dependência da plataforma
- possível custo adicional

Terceira abordagem:

Playwright MCP com agente autônomo.

Vantagens:

- extrema flexibilidade
- automação semântica
- navegação inteligente
- uso para QA e RPA
- independência de código de teste

Desvantagens:

- consumo contínuo de tokens
- custo computacional maior
- execução potencialmente mais lenta

Esse comparativo é extremamente importante porque mostra algo fundamental da engenharia moderna:

Não existe solução mágica universal.

Toda escolha possui trade-offs.

E talvez esse seja um dos maiores aprendizados de todo o módulo.

Profissionais seniores não escolhem ferramentas apenas porque estão “na moda”.

Eles analisam:

- custo
- benefício
- contexto
- manutenção
- escala
- arquitetura
- flexibilidade

Outro detalhe extremamente importante foi perceber que essas abordagens não são mutuamente exclusivas.

Podemos perfeitamente combinar estratégias diferentes no mesmo ecossistema.

Por exemplo:

- Cypress tradicional para smoke tests rápidos
- Cypress Prompt para fluxos frágeis
- Playwright MCP para RPAs e sistemas legados

Esse tipo de combinação híbrida provavelmente será cada vez mais comum.

Ao final da aula, o principal aprendizado ficou extremamente claro:

IA não está apenas ajudando a gerar testes.

Ela está começando a transformar completamente a própria forma como pensamos automação, QA e interação com sistemas.

Os agentes deixam de apenas escrever scripts e passam a operar interfaces de maneira cada vez mais próxima de um usuário humano real.

E isso abre possibilidades extremamente grandes não apenas para testes, mas para automação operacional de sistemas inteiros.

Módulo 05 - Integração de Lógica de IA no Cliente e no Servidor

Capítulo 1 - O que é o Genkit e Configuração Inicial do Projeto

Neste último módulo da disciplina, começamos um novo projeto completamente diferente dos anteriores. Até aqui, nós utilizamos inteligência artificial principalmente como apoio ao processo de engenharia: refinamos requisitos, criamos prompts estruturados, geramos componentes, automatizamos testes, utilizamos agentes e organizamos arquitetura enterprise.

Agora mudamos completamente o foco.

A IA deixa de ser apenas uma ferramenta que auxilia o desenvolvimento e passa a fazer parte da própria arquitetura da aplicação.

Esse é um ponto extremamente importante.

Nas aulas anteriores, o Gemini, o Cypress Prompt, os agentes e os MCPs eram ferramentas utilizadas por nós, desenvolvedores, para acelerar trabalho técnico.

Agora a LLM passa a ser consumida diretamente pela aplicação.

Ou seja:

- a aplicação conversa com a IA
- a IA retorna dados
- o backend processa essas respostas
- o frontend consome essas respostas

Isso muda bastante o tipo de arquitetura que precisamos construir.

O projeto que vamos desenvolver neste módulo se chama BragBot.

A ideia dele é extremamente interessante e também bastante próxima da realidade profissional.

No mercado, principalmente para pessoas desenvolvedoras mais experientes, é muito comum manter o que chamamos de Brag Doc.

O Brag Doc é basicamente um diário de conquistas profissionais.

Ao longo do ano, nós:

- entregamos projetos
- resolvemos incidentes
- lideramos iniciativas
- melhoramos sistemas
- fazemos apresentações
- orientamos pessoas
- criamos soluções difíceis

Mas, quando chega o momento de uma avaliação de desempenho, PDI ou promoção, muitas vezes esquecemos tudo o que fizemos.

Então, manter um registro estruturado das conquistas ajuda bastante no crescimento profissional.

O BragBot será justamente uma aplicação voltada para isso.

Mas, na prática, o objetivo principal do módulo não é apenas criar esse sistema.

O verdadeiro foco é aprender como integrar LLMs profissionalmente dentro de uma aplicação real.

E aqui entra um conceito extremamente importante: frameworks de orquestração de IA.

Atualmente, praticamente todos os modelos possuem APIs próprias.

Cada fornecedor oferece:

- endpoints
- SDKs
- autenticação
- formatos de chamada
- formas de retorno

Então, se eu consumir diretamente cada API individualmente, minha aplicação fica extremamente acoplada ao fornecedor da IA.

Esse é um problema importante.

Porque trocar de modelo depois pode se tornar extremamente caro.

É justamente aqui que entra o GenKit.

O GenKit é um framework criado pelo Google para ajudar aplicações Node.js a trabalharem com modelos de IA.

Mas existe um detalhe extremamente importante:

Apesar de ser criado pelo Google, o GenKit é agnóstico de modelo.

Esse ponto é fundamental.

Ele não existe apenas para Gemini.

Ele funciona através de plugins.

Ou seja, conseguimos conectar:

- Gemini
- OpenAI
- outros fornecedores
- modelos open source
- provedores compatíveis

Esse detalhe é extremamente poderoso.

Porque começamos a desacoplar nossa aplicação do fornecedor específico da LLM.

E isso se conecta diretamente com um conceito clássico de engenharia de software: abstração.

Ao longo da história do desenvolvimento, nós sempre criamos camadas de abstração para reduzir acoplamento.

Por exemplo:

- ORMs abstraem banco de dados
- frameworks web abstraem HTTP
- SDKs abstraem APIs
- GenKit abstrai interação com LLMs

Esse é exatamente o papel dele.

Outro ponto extremamente importante é que GenKit não é apenas um “wrapper” simples de API.

Ele também ajuda em vários outros problemas comuns quando trabalhamos com IA.

Por exemplo:

- construção de prompts
- validação de saída
- tratamento de erros
- debug
- observabilidade
- troca de modelos
- padronização de chamadas

E aqui aparece um dos pontos mais importantes da aula: validação estrutural da resposta da LLM.

Quando usamos IA dentro da aplicação, não podemos tratar a resposta apenas como texto solto.

A aplicação precisa trabalhar com estruturas previsíveis.

Por exemplo:

- JSON
- objetos tipados
- contratos
- schemas

Esse detalhe é extremamente importante.

Se a IA retornar algo inválido, o backend precisa detectar isso corretamente.

O GenKit ajuda bastante nesse processo.

Ele não apenas monta prompts mais adequados para o modelo.

Ele também consegue validar se a saída retornada respeita a estrutura esperada.

Caso contrário, ele lança exceções para que o sistema trate adequadamente.

Esse detalhe aproxima bastante IA do fluxo tradicional de engenharia backend.

Porque agora começamos a trabalhar com:

- contratos
- validação
- schemas
- tratamento de erro
- integração tipada

Outro detalhe importante é perceber que o GenKit funciona como biblioteca e não como framework completo de aplicação.

Ou seja, ele pode ser usado dentro de:

- Angular SSR
- Express
- NestJS
- aplicações Node
- microserviços
- APIs tradicionais

No nosso caso, escolhemos utilizar Angular com SSR.

SSR significa Server Side Rendering.

O Angular moderno possui um servidor próprio baseado em Node.

E aqui tomamos uma decisão arquitetural importante.

Em vez de criar:

- frontend Angular
- backend separado

vamos utilizar o próprio servidor SSR do Angular também como backend simplificado.

Esse detalhe foi escolhido para reduzir complexidade didática do projeto.

O foco principal agora é o GenKit.

Então queremos evitar excesso de infraestrutura paralela.

Mas é importante entender:

Essa não é necessariamente a arquitetura ideal para produção enterprise.

Em um cenário real, poderíamos perfeitamente ter:

- Angular no frontend
- NestJS no backend
- microserviço separado de IA
- workers especializados

Aqui estamos simplificando apenas para facilitar aprendizado do fluxo.

Depois disso, criamos o projeto Angular utilizando SSR habilitado.

Esse detalhe é extremamente importante porque o servidor SSR será responsável tanto por renderizar quanto por processar chamadas da LLM.

Durante a criação do projeto, também aparece algo interessante sobre o estado atual das ferramentas modernas.

O Angular já sugere automaticamente criação de arquivos voltados para agentes de IA.

Isso mostra como o ecossistema inteiro está começando a assumir IA como parte natural do desenvolvimento moderno.

Outro ponto extremamente importante da aula foi a criação da API Key do Gemini.

E aqui começamos a entrar em uma área extremamente séria da engenharia moderna: segurança.

Quando trabalhamos com IA em produção, precisamos proteger credenciais exatamente da mesma forma que protegemos:

- senhas
- tokens
- chaves de banco
- segredos de APIs

A API Key da LLM é extremamente sensível.

Porque ela representa custo financeiro e acesso à infraestrutura do modelo.

Se alguém obtiver essa chave, poderá:

- consumir sua cota
- gerar custos
- abusar da API
- comprometer segurança

Por isso nunca devemos colocar credenciais diretamente no código.

Esse é um erro extremamente grave.

Então criamos um arquivo `.env`.

O `.env` funciona como armazenamento local de variáveis de ambiente.

Depois adicionamos o `.env` no `.gitignore`.

Esse detalhe é extremamente importante.

Porque impede que o arquivo seja enviado para o GitHub acidentalmente.

Esse talvez seja um dos maiores erros cometidos por iniciantes:

Subir API Keys em repositórios públicos.

Inclusive existem bots automatizados na internet procurando exatamente esse tipo de vazamento.

Outro detalhe importante é perceber como frameworks modernos trabalham com convenção sobre configuração.

O GenKit já reconhece automaticamente certas variáveis padrão.

Por exemplo:

- `GOOGLE_API_KEY`

Isso reduz bastante configuração manual.

Depois disso instalamos:

- GenKit CLI globalmente
- GenKit no projeto
- plugin do Google AI

O GenKit CLI também é extremamente importante porque fornece ferramentas de:

- desenvolvimento
- debug
- observabilidade
- inspeção

Outro ponto extremamente interessante é que o próprio GenKit possui MCP.

Esse detalhe mostra novamente como MCPs estão se tornando peça central no ecossistema moderno de IA aplicada ao desenvolvimento.

Assim como fizemos anteriormente com:

- Angular MCP
- Nx MCP
- Playwright MCP

agora configuramos também o GenKit MCP.

Isso melhora bastante qualidade da interação do agente com o framework.

Depois de configurar o MCP, realizamos uma validação simples perguntando ao agente quais modelos estavam disponíveis através do servidor MCP do GenKit.

E aqui percebemos que a integração funcionou corretamente.

O agente conseguiu listar modelos disponíveis e validar a comunicação.

Esse tipo de verificação é extremamente importante quando trabalhamos com ambientes baseados em múltiplos MCPs.

Ao final da aula, construímos toda a fundação do novo projeto:

- Angular SSR
- arquitetura simplificada
- GenKit instalado
- plugin Gemini configurado
- API Key protegida
- `.env` estruturado
- Gitignore ajustado
- MCP do GenKit funcionando

E talvez o maior aprendizado deste capítulo seja perceber uma mudança extremamente importante na forma como usamos IA.

Até agora, a IA era principalmente uma ferramenta para auxiliar engenharia.

Agora ela começa a se transformar em componente arquitetural real da aplicação.

Isso muda completamente a forma como pensamos:

- backend
- contratos
- segurança
- validação
- integração
- observabilidade
- arquitetura de software

E é justamente esse novo cenário que começaremos a explorar nas próximas aulas.

Capítulo 2 - Design System UNIPDS e Interface Mockada

Nesta aula, começamos a construir a primeira interface do BragBot. Até aqui, nós criamos o projeto Angular com SSR, instalamos o GenKit, configuramos a chave de API com segurança e conectamos o MCP do GenKit ao nosso ambiente de desenvolvimento.

Agora, antes de integrar a LLM de fato na aplicação, vamos criar uma interface funcional com dados mockados.

Esse é um passo importante porque não precisamos começar conectando tudo ao modelo imediatamente. Em desenvolvimento front-end, muitas vezes é melhor primeiro montar o fluxo visual, validar a navegação, testar o look and feel e só depois plugar a camada real de backend ou inteligência artificial. Assim, conseguimos separar melhor as responsabilidades e evitar confundir problemas de interface com problemas de integração.

Neste projeto, queremos simular uma identidade visual próxima da página da pós-graduação da UniPDS. A ideia é criar uma experiência visual coerente, usando as cores, o estilo e a sensação estética da instituição. Para isso, usamos um prompt no Antigravity pedindo que o agente acesse a página da UniPDS, extraia a identidade visual e aplique esse padrão na aplicação Angular. Como sempre, o ponto principal não é a ferramenta específica, mas a técnica: usar IA para acelerar a construção de uma interface baseada em referência visual e textual.

O prompt definiu algumas regras bem objetivas. O projeto está em Angular 21 com Tailwind CSS, o GenKit e os MCPs já estão instalados, e a aplicação precisa ter duas telas principais: dashboard e detail. A rota padrão deve apontar para o dashboard, enquanto a rota de detalhe deve receber um identificador pela URL. Esse tipo de organização já cria uma estrutura mínima de produto, com uma tela de listagem e uma tela de leitura detalhada.

Também pedimos a criação de um serviço Angular para centralizar o estado da aplicação. Esse ponto é muito importante. Em Angular, quando temos regras de negócio, acesso a dados ou qualquer tipo de estado compartilhado, o mais correto é não colocar tudo diretamente no componente. O componente deve cuidar da tela, da interação e da composição visual. Já o serviço deve concentrar a lógica de dados.

Nesse caso, criamos um serviço usando Signals. Os Signals são um recurso moderno do Angular para trabalhar com estado reativo de forma simples e expressiva. O serviço possui um Signal para armazenar a lista de conquistas e outro Signal para controlar o estado de carregamento. Esse segundo estado é essencial para a experiência do usuário, porque permite mostrar um spinner, alterar o texto do botão e indicar que alguma operação está em andamento.

Por enquanto, ainda não chamamos o GenKit nem o Gemini. Criamos um método mockado para simular uma chamada de rede. Esse método ativa o loading, aguarda um tempo usando uma simulação assíncrona e depois adiciona uma conquista gerada na lista. Essa conquista possui título, contexto, impacto, tecnologias e métrica. Ou seja, já modelamos o formato aproximado que futuramente será retornado pela LLM.

Essa abordagem é muito útil porque permite validar a interface antes da integração real. Quando conectarmos o GenKit, o contrato visual já estará definido. O botão já existe. O

loading já funciona. Os cards já aparecem. A tela de detalhe já sabe renderizar as informações. Assim, a troca do mock pela chamada real será muito mais segura.

No dashboard, a interface possui um cabeçalho, uma área de formulário com um campo de texto para a conquista bruta e um botão chamado “destilar conquista”. A ideia é que o usuário escreva uma conquista de forma livre, como faria em um diário profissional, e depois a aplicação transforme aquilo em uma versão mais estruturada, clara e útil para avaliação de desempenho.

Enquanto o loading estiver ativo, o botão muda de estado e exibe um comportamento visual de processamento. Esse detalhe parece simples, mas melhora muito a experiência. Usuários precisam de feedback quando uma operação está sendo executada. Sem isso, a aplicação parece travada ou inconsistente.

Abaixo do formulário, usamos o novo control flow do Angular, com `aroba-for`, para renderizar os cards das conquistas geradas. Cada card é clicável e leva para a tela de detalhe. Essa tela de detalhe apresenta as informações de forma mais organizada, com contexto, impacto, métrica e tecnologias usadas.

Um ponto interessante da geração feita pelo agente foi a escolha de colocar o template dentro do próprio arquivo TypeScript do componente. Hoje isso não é necessariamente uma má prática na comunidade Angular, especialmente em componentes pequenos. Porém, pessoalmente, eu prefiro separar template, estilo e lógica em arquivos distintos quando a tela começa a crescer. Esse é um exemplo de decisão que precisamos revisar depois que a IA gera código. O agente acelera, mas a preferência arquitetural do time continua sendo importante.

O mais interessante dessa etapa é perceber como o serviço simplifica o componente. O componente não precisa controlar manualmente toda a lógica de loading, atualização da lista e propagação de estado. Ele apenas chama o método do serviço, e os Signals cuidam da reatividade. Quando o loading muda, o template reage. Quando uma nova conquista é adicionada, a lista atualiza. Isso deixa o código mais limpo e mais fácil de evoluir.

Ao executar a aplicação, já conseguimos ver a primeira versão funcional do BragBot. Digitamos uma conquista, clicamos em destilar, a aplicação simula o processamento e renderiza um card com a conquista estruturada. Ao clicar no card, navegamos para a tela de detalhe e visualizamos as informações completas.

Essa aula mostra uma prática muito importante: antes de plugar a inteligência artificial na aplicação, vale a pena construir a interface com dados mockados. Isso permite validar fluxo, estado, navegação, identidade visual e experiência do usuário de maneira isolada. Na próxima etapa, vamos substituir essa simulação pela integração real com o GenKit e começar a usar a LLM como componente da arquitetura da aplicação.

Capítulo 3 - O Cérebro do Genkit: Flows, Zod e Google AI

Na aula anterior, nós estruturamos o front-end do BragBot utilizando Angular com SSR, criamos uma interface inspirada no design system da UniPDS, organizamos o estado da aplicação com Signals e simulamos a geração de conquistas utilizando dados mockados.

Agora chegamos em uma das partes mais importantes de todo o projeto: a integração real com a LLM utilizando GenKit.

E aqui começamos a entrar em um ponto extremamente importante da engenharia moderna com IA.

Até agora, nós utilizamos IA principalmente como ferramenta de apoio ao desenvolvimento:

- geração de componentes
- criação de prompts
- automação
- QA
- especificações
- arquitetura

Agora a IA deixa de ser apenas uma ferramenta para o desenvolvedor e passa a fazer parte da própria aplicação.

Ou seja:

- o usuário interage com a aplicação
- a aplicação conversa com a LLM
- a LLM responde
- o backend processa
- o frontend renderiza

Isso muda bastante o tipo de arquitetura que precisamos construir.

E é justamente aqui que o GenKit começa a mostrar seu valor.

O GenKit funciona como uma camada de abstração entre nossa aplicação e os modelos de IA.

Esse detalhe é extremamente importante.

Porque, no mundo atual, cada fornecedor de LLM possui:

- SDK próprio

- API própria
- autenticação própria
- formato próprio
- convenções próprias

Se consumirmos diretamente cada API, criamos um acoplamento muito forte com o fornecedor.

Então, trocar de modelo depois se torna caro e trabalhoso.

O GenKit resolve justamente esse problema.

Ele abstrai a interação com os modelos.

Na prática, conseguimos trocar entre:

- Gemini
- OpenAI
- Anthropic
- modelos locais
- modelos open source

alterando basicamente apenas a configuração do plugin e do modelo.

E isso é extremamente poderoso do ponto de vista arquitetural.

Outro ponto importante é que o GenKit não é apenas um wrapper simples para APIs.

Ele fornece várias capacidades extremamente úteis:

- definição de flows
- validação estrutural
- observabilidade
- tracing
- debug
- controle de output
- abstração de modelos
- integração tipada

Nesta aula começamos justamente criando o coração da nossa aplicação: o flow responsável por transformar um texto informal do usuário em um Brag Document profissional.

E aqui aparece novamente algo extremamente importante no trabalho com IA moderna:

bons resultados dependem de bons prompts.

Mais uma vez seguimos uma abordagem extremamente estruturada.

Criamos um prompt detalhado orientando o agente a:

- utilizar MCP do GenKit
- seguir melhores práticas
- criar schemas
- configurar plugins
- estruturar o flow
- definir output validado

Esse ponto é importante porque, mesmo usando GenKit, continuamos trabalhando fortemente com engenharia de prompt.

A diferença é que agora o prompt começa a ficar encapsulado dentro da arquitetura da aplicação.

Outro detalhe extremamente importante da aula foi o uso do MCP do GenKit.

Mais uma vez aparece um padrão que atravessou praticamente todo o módulo inteiro:

quanto melhor configuramos contexto e ferramentas do agente, melhor tende a ser a qualidade da implementação.

O MCP fornece:

- documentação atualizada
- exemplos corretos
- melhores práticas
- modelos suportados
- sintaxe correta

Isso reduz muito:

- tentativa e erro
- alucinação
- código desatualizado
- incompatibilidades

Depois disso, começamos a analisar o código gerado.

E aqui aparece um dos conceitos mais importantes do GenKit: flows.

No GenKit, praticamente toda interação com uma LLM acontece através de um flow.

Um flow é basicamente uma função assíncrona estruturada.

Mas ele representa algo maior do que apenas uma função comum.

Ele encapsula:

- prompt
- modelo
- schemas
- validação
- observabilidade
- tracing
- configuração de execução

Ou seja, ele funciona como unidade operacional de IA dentro da aplicação.

No nosso caso, criamos um flow chamado `bragGenerateFlow`.

O objetivo dele é receber um rascunho informal do usuário e devolver um documento estruturado contendo:

- título
- contexto
- ação tomada
- impacto
- métricas
- tecnologias

Esse tipo de transformação é extremamente interessante porque representa muito bem um caso de uso real de IA aplicada a produtividade profissional.

Outro ponto extremamente importante foi a configuração do GenKit.

Criamos uma instância principal configurando:

- plugin Google AI
- modelo Gemini
- temperatura
- comportamento padrão

E aqui aparece um detalhe extremamente importante sobre abstração arquitetural.

A partir do momento em que usamos GenKit, praticamente todo o restante da aplicação fica agnóstico de modelo.

Ou seja:

- o flow continua igual
- os schemas continuam iguais
- a lógica continua igual

O que muda normalmente é apenas:

- plugin
- nome do modelo
- eventualmente alguma configuração específica

Esse desacoplamento é extremamente valioso em aplicações reais.

Porque o mercado de IA muda muito rapidamente.

Hoje um modelo pode ser melhor.

Amanhã outro modelo pode assumir vantagem.

Então não queremos que nossa aplicação fique presa profundamente a um fornecedor específico.

Outro detalhe importante da aula foi o conceito de temperatura.

Temperatura controla nível de criatividade da resposta da LLM.

Quanto maior a temperatura:

- mais criatividade
- mais diversidade
- mais imprevisibilidade

Quanto menor:

- mais consistência
- mais previsibilidade
- menos variação

No nosso caso, utilizamos uma temperatura intermediária para permitir alguma criatividade na transformação do Brag Document sem perder profissionalismo.

Outro ponto extremamente importante da aula foi o uso do Zod.

E aqui começamos a entrar em uma das partes mais poderosas do GenKit.

O Zod é uma biblioteca extremamente popular no ecossistema TypeScript para validação de schemas.

Normalmente o TypeScript valida tipos apenas em tempo de compilação ou transpilação.

Mas isso não resolve completamente o problema de APIs.

Porque quando recebemos dados externos:

- usuários
- requests
- serviços
- LLMs

não temos garantia de que aquilo realmente respeita nossa tipagem.

É justamente aqui que o Zod entra.

Ele permite validar contratos em tempo de execução.

E isso é extremamente importante quando trabalhamos com IA.

Porque LLMs não são determinísticas.

Elas podem:

- alucinar
- mudar estrutura
- esquecer propriedades
- retornar formatos inválidos

Então precisamos validar rigorosamente o retorno.

E aqui aparece um detalhe extremamente inteligente do GenKit.

O GenKit utiliza o Zod não apenas para validação estrutural.

Ele também usa os describes do schema para construir o próprio prompt enviado para a LLM.

Esse detalhe é extremamente poderoso.

Porque os describes funcionam quase como documentação semântica da estrutura esperada.

Por exemplo:

- title representa ação principal
- context descreve situação original
- actionTaken explica estratégia aplicada
- impact descreve impacto de negócio

Tudo isso ajuda o modelo a entender exatamente o que esperamos.

Esse é um ponto extremamente importante.

Os schemas deixam de ser apenas validação técnica.

Eles passam a participar ativamente da engenharia de prompt.

Isso aproxima muito IA do fluxo tradicional de engenharia backend baseada em contratos.

Outro ponto importante foi o próprio System Prompt.

Mais uma vez aplicamos praticamente todas as boas práticas trabalhadas ao longo do módulo:

- persona
- objetivo
- regras
- formato esperado
- restrições

No prompt definimos que a LLM deve atuar como redatora profissional especializada em transformar conquistas técnicas em documentação executiva.

Também pedimos:

- tom profissional
- linguagem objetiva
- ausência de exageros emocionais
- preservação da língua original do input

Esse último detalhe é extremamente interessante.

Com apenas uma instrução simples, deixamos a aplicação multilíngue.

Se o usuário escrever em português, o retorno vem em português.

Se escrever em inglês, retorna em inglês.

Se escrever em espanhol, retorna em espanhol.

Esse é um dos grandes poderes das LLMs modernas.

Outro detalhe importante foi a configuração do output schema.

O GenKit consegue instruir a LLM a retornar diretamente no formato esperado.

Dependendo do modelo utilizado, isso pode acontecer de formas diferentes.

No caso do Gemini, o schema pode ser enviado separadamente como parte estruturada da requisição.

Em outros modelos, o GenKit pode incorporar isso diretamente no prompt textual.

E esse é exatamente um dos benefícios da abstração.

A aplicação não precisa se preocupar com as particularidades de cada fornecedor.

O GenKit adapta isso internamente.

Outro ponto extremamente importante da aula foi o uso da GenKit Dev UI.

Essa ferramenta é extremamente poderosa.

Em vez de precisar:

- criar API
- subir frontend
- executar integração completa
- debugar requests

conseguimos testar diretamente os flows isoladamente.

A Dev UI permite:

- executar flows
- visualizar traces
- analisar prompts
- verificar output
- inspecionar schemas
- debugar erros

Isso acelera enormemente desenvolvimento.

Durante a aula, inclusive, apareceu um problema extremamente importante relacionado à configuração do modelo Gemini.

Inicialmente o modelo estava configurado incorretamente.

E aqui aconteceu algo extremamente valioso pedagogicamente.

Em vez de simplesmente ignorar o problema, usamos a própria ferramenta para:

- diagnosticar
- investigar
- corrigir
- validar

Esse fluxo mostra muito bem uma realidade da engenharia moderna com IA:

o ambiente muda rapidamente.

Versões mudam.
Modelos mudam.
SDKs mudam.
Nomes mudam.

Então precisamos desenvolver capacidade de observabilidade e debugging.

A Dev UI ajudou justamente nisso.

Conseguimos:

- visualizar erro
- identificar modelo inválido
- consultar modelos disponíveis
- ajustar configuração
- reexecutar flow

Depois da correção, o flow passou a funcionar corretamente.

E aqui apareceu algo extremamente interessante.

A Dev UI permitiu visualizar:

- prompt enviado
- schema gerado
- output retornado
- parsing do JSON
- trace completo da execução

Esse nível de observabilidade é extremamente importante em aplicações com IA.

Porque debugging de LLM é diferente de debugging tradicional.

Não estamos apenas analisando código determinístico.

Precisamos entender:

- contexto enviado
- instruções
- temperatura
- schema
- comportamento do modelo

Outro ponto importante foi perceber que o próprio GenKit transforma a resposta textual da LLM em JSON validado.

Ou seja:

- a LLM responde
- o GenKit processa
- o schema valida
- o flow retorna objeto tipado

Isso reduz bastante trabalho manual de parsing e validação.

Ao final da aula, tínhamos:

- GenKit configurado
- plugin Google AI funcionando
- Gemini integrado
- flow criado
- schemas definidos
- validação com Zod
- prompt estruturado
- Dev UI funcionando
- tracing operacional

E talvez o maior aprendizado deste capítulo seja perceber que frameworks como GenKit transformam IA em algo muito mais próximo da engenharia backend tradicional.

Começamos a trabalhar com:

- contratos
- schemas
- observabilidade
- validação
- abstração
- tracing
- debugging
- arquitetura desacoplada

Ou seja, a IA deixa de ser apenas “um chat” e passa a funcionar como componente estruturado e observável dentro da arquitetura real da aplicação.

Capítulo 4 - O Micro-BFF, Integração Full-Stack

Nas aulas anteriores, nós construímos praticamente toda a base da nossa aplicação BragBot. Criamos o projeto Angular com SSR, configuramos o GenKit, integramos o Gemini, estruturamos o flow responsável pela transformação das conquistas profissionais e criamos uma interface mockada utilizando Signals e serviços Angular.

Agora chegamos em um ponto extremamente importante da arquitetura: conectar tudo isso de ponta a ponta.

Até este momento, o front-end ainda estava trabalhando apenas com dados simulados.

Ou seja:

- a tela existia
- os formulários funcionavam
- o loading funcionava
- os cards eram renderizados

Mas a geração das conquistas ainda era fake.

Agora vamos transformar a aplicação em um fluxo full-stack real.

E aqui aparece um conceito extremamente importante dentro da arquitetura moderna: o BFF.

BFF significa Backend For Frontend.

Esse padrão arquitetural surgiu para resolver um problema muito comum em aplicações modernas.

Normalmente temos:

- frontend
- backend
- banco de dados
- serviços externos
- regras de negócio

Mas muitas vezes o frontend não deveria conversar diretamente com todos esses serviços.

Então criamos uma camada intermediária especializada em servir exatamente as necessidades da interface.

Essa camada é o BFF.

O papel dela é:

- expor APIs adequadas para o frontend
- encapsular regras de negócio
- adaptar payloads
- proteger credenciais
- centralizar integrações

Esse padrão ficou extremamente comum em arquiteturas modernas com:

- microserviços
- SPAs
- mobile
- SSR
- APIs externas

No nosso caso, criamos uma versão simplificada disso: um micro-BFF.

E aqui é importante entender uma decisão arquitetural importante da aula.

Nós escolhemos usar o próprio servidor SSR do Angular como backend simplificado.

Ou seja:

- frontend Angular
- servidor SSR
- API Express
- GenKit

tudo rodando no mesmo processo Node.js.

Esse detalhe é extremamente importante.

Porque essa não é necessariamente a arquitetura enterprise ideal para produção em larga escala.

Em um cenário enterprise mais robusto, poderíamos perfeitamente ter:

- Angular separado
- NestJS separado
- microserviço de IA
- autenticação dedicada
- gateway
- filas
- workers

Aqui simplificamos intencionalmente.

O foco principal deste módulo é aprender GenKit e integração de IA.

Então reduzimos complexidade de infraestrutura para concentrar atenção na arquitetura da LLM.

Esse é um padrão muito importante em aprendizado técnico:

simplificar infraestrutura para focar no conceito principal.

Depois disso, começamos a construir nossa API.

Mais uma vez utilizamos um prompt extremamente estruturado orientando o agente sobre:

- uso do MCP do GenKit
- modificação do server.ts
- criação da rota
- integração com o flow
- tratamento de erro
- atualização do serviço Angular
- validação do build

Outro detalhe extremamente importante foi a orientação sobre o posicionamento da rota Express dentro do Angular SSR.

Isso é muito importante.

No Angular SSR existe uma rota catch-all responsável pela renderização da aplicação.

Se colocarmos nossa API depois dessa rota, o Angular pode interceptar as chamadas e quebrar o comportamento esperado.

Esse tipo de detalhe mostra algo extremamente importante sobre desenvolvimento assistido por IA:

o agente acelera implementação, mas conhecimento arquitetural continua sendo fundamental.

Precisamos entender:

- ciclo do framework
- ordem das rotas
- comportamento do servidor
- interceptação
- SSR

Depois disso, o agente começou a modificar o servidor.

E aqui aparece um detalhe extremamente interessante.

Como estamos usando SSR do Angular, o próprio servidor Node já utiliza Express internamente.

Então conseguimos simplesmente adicionar endpoints REST diretamente nele.

Para isso, adicionamos:

- express.json
- rota POST

- integração com o flow

A API criada acabou ficando extremamente simples.

E isso mostra algo muito importante sobre GenKit.

O flow funciona praticamente como uma função assíncrona tradicional.

Então a rota:

- recebe request
- extrai definition
- chama flow
- retorna JSON

Arquiteturalmente, é extremamente parecido com qualquer outro serviço backend moderno.

Esse detalhe é muito importante porque ajuda a desmistificar IA dentro da arquitetura.

No fim das contas, uma aplicação com LLM continua sendo:

- entrada
- processamento
- validação
- resposta

A diferença é que agora parte do processamento acontece através de modelos generativos.

Outro detalhe extremamente importante foi o tratamento de erro.

Como LLMs podem:

- falhar
- exceder limites
- retornar inválido
- lançar exceções

precisamos tratar erros corretamente.

Então a rota captura exceções e retorna HTTP 500.

Esse detalhe parece simples, mas é extremamente importante em aplicações reais.

Integrações com IA precisam ser tratadas exatamente como qualquer integração externa crítica.

Depois disso, o agente atualizou o frontend.

E aqui aparece outro ponto importante da arquitetura Angular moderna.

Como estamos utilizando componentes standalone, não trabalhamos mais com módulos tradicionais da forma clássica.

Então precisamos registrar providers explicitamente para serviços como:

- HttpClient

Esse detalhe é importante porque a arquitetura Angular mudou bastante nas versões mais recentes.

Outro ponto importante foi a atualização do serviço.

Anteriormente, nosso serviço apenas simulava chamadas usando:

- setTimeout
- mock
- arrays locais

Agora ele passou a consumir a API real utilizando HttpClient.

Mas aqui aparece algo extremamente interessante sobre boa arquitetura frontend.

A interface praticamente não precisou mudar.

Esse detalhe mostra a importância de separar corretamente:

- estado
- serviço
- interface
- integração

Como toda a lógica já estava encapsulada no serviço, a troca do mock pela API foi extremamente simples.

Esse é um dos maiores benefícios da boa separação de responsabilidades.

Outro detalhe extremamente importante foi a validação automática de build.

O agente executou:

- build
- ajustes de configuração
- validação SSR
- correção de integração

Isso mostra novamente como agentes modernos não apenas escrevem código.

Eles também:

- validam
- corrigem
- ajustam ambiente
- executam pipeline

Depois disso, subimos a aplicação e fizemos o primeiro teste real da integração completa.

E aqui aconteceu algo extremamente importante.

Pela primeira vez, tínhamos um fluxo full-stack completo:

- frontend Angular
- API Express
- GenKit
- Gemini
- resposta estruturada
- renderização dinâmica

O usuário escreve uma conquista informal.

A aplicação envia para a API.

A API chama o flow do GenKit.

O GenKit conversa com o Gemini.

O Gemini gera a resposta estruturada.

O frontend renderiza o resultado.

Esse é um momento extremamente importante da arquitetura do projeto.

Porque agora a IA deixou definitivamente de ser apenas ferramenta de desenvolvimento e passou a ser parte operacional da aplicação.

Outro ponto extremamente interessante foi a tentativa de multilanguage.

No prompt do flow, definimos que a resposta deveria respeitar a linguagem original do input.

Teoricamente:

- português gera português
- inglês gera inglês

- espanhol gera espanhol

Mas durante o teste percebemos um comportamento inconsistente.

O input foi enviado em inglês, mas parte da resposta ainda veio em português.

E esse ponto é extremamente importante pedagogicamente.

Porque ele mostra uma característica fundamental das LLMs:

elas não são determinísticas.

Mesmo com prompts corretos, ainda podem existir ambiguidades.

Isso reforça novamente algo extremamente importante de toda engenharia de IA:

prompt engineering é um processo iterativo.

Precisamos:

- testar
- observar
- refinar
- ajustar
- validar

E aqui aparece novamente o valor da observabilidade do GenKit.

Com a Dev UI conseguimos posteriormente:

- revisar prompt
- entender comportamento
- analisar contexto
- ajustar instruções

Esse tipo de iteração faz parte natural do desenvolvimento com IA.

Outro detalhe extremamente importante da aula foi perceber como o GenKit simplifica bastante a integração arquitetural.

Sem ele, precisaríamos manualmente lidar com:

- SDK da LLM
- autenticação
- parsing
- validação
- schemas
- retries

- tracing

O GenKit centraliza tudo isso.

E talvez esse seja um dos maiores aprendizados deste capítulo.

Frameworks de IA começam a ocupar um papel muito parecido com frameworks backend tradicionais.

Eles abstraem complexidade operacional para que possamos focar na regra de negócio.

Ao final da aula, tínhamos:

- frontend Angular funcional
- API Express integrada
- Angular SSR servindo backend
- GenKit conectado
- Gemini operacional
- flow executando
- integração full-stack funcionando
- geração dinâmica de Brag Documents

Tudo isso utilizando:

- Angular SSR
- Express
- GenKit
- Gemini
- Signals
- HttpClient
- MCP
- flows estruturados

E talvez o maior aprendizado deste capítulo seja perceber que aplicações modernas com IA começam a se parecer muito mais com sistemas backend completos do que apenas interfaces conectadas a um chat.

Agora precisamos pensar em:

- arquitetura
- contratos
- APIs
- validação
- observabilidade
- segurança
- desacoplamento

- integração full-stack

Porque a IA finalmente passou a fazer parte da própria lógica operacional da aplicação.

Capítulo 5 - O Engenheiro AI-Native (Revisão da Jornada e Encerramento)

Ao longo desta unidade, nós percorremos uma jornada extremamente ampla sobre o uso de inteligência artificial dentro da engenharia de software moderna. E talvez o ponto mais importante de todo o conteúdo não seja uma ferramenta específica, um framework específico ou um modelo específico.

O ponto central foi aprender uma nova forma de pensar desenvolvimento de software.

Essa talvez seja a principal transformação que estamos vivendo atualmente na indústria.

Durante muitos anos, o desenvolvimento foi centrado principalmente em:

- escrever código manualmente
- implementar lógica operacional
- construir tudo de forma artesanal

Agora começamos a entrar em uma nova fase.

Uma fase onde o desenvolvedor passa a atuar muito mais como:

- arquiteto
- orquestrador
- integrador
- revisor
- estrategista técnico

Esse é o conceito do engenheiro AI-Native.

Ser AI-Native não significa depender cegamente de IA.

Também não significa deixar a IA “fazer tudo”.

Na verdade, significa aprender como integrar inteligência artificial corretamente dentro do fluxo real de engenharia.

E foi exatamente isso que trabalhamos ao longo de toda a unidade.

No módulo 1, começamos estudando os fundamentos da engenharia de IA.

Esse ponto foi extremamente importante porque mostrou algo que muita gente ainda não percebeu corretamente:

IA não serve apenas para gerar código.

Ela pode apoiar praticamente todas as etapas do ciclo de desenvolvimento.

Nós vimos como utilizar ferramentas de IA para:

- refinamento de requisitos
- estimativa de tarefas
- detalhamento técnico
- identificação de edge cases
- UX Writing
- documentação
- design de interface
- arquitetura

Esse talvez tenha sido um dos primeiros grandes aprendizados do curso.

A IA não substitui o time.

Ela potencializa o time.

Quando usada corretamente, ela ajuda desenvolvedores, arquitetos, designers, QAs e product owners a trabalharem de maneira mais eficiente e estratégica.

Depois disso, nos módulos 2 e 3, começamos a explorar desenvolvimento utilizando agentes autônomos.

E aqui demos um salto extremamente importante.

Saímos do modelo simples de “chat para gerar código” e começamos a trabalhar com:

- agentes
- MCPs
- orquestração
- monorepo
- isolamento
- automação
- desenvolvimento paralelo

Utilizamos ferramentas como:

- Antigravity

- Jules
- OpenSpec
- Nx
- WorkTree

Mas novamente, o mais importante não era decorar as ferramentas.

O foco sempre foi entender os princípios.

Aprendemos sobre Spec-Driven Development e como especificações voltam a ganhar enorme importância no desenvolvimento moderno.

Esse ponto é extremamente importante.

Durante muitos anos, parte da indústria passou a minimizar documentação e especificação.

Mas agora, com agentes de IA, especificação volta a se tornar fundamental.

Porque agora não estamos apenas alinhando pessoas.

Estamos orientando sistemas generativos.

E sistemas generativos dependem fortemente de contexto bem estruturado.

Por isso trabalhamos com:

- proposal
- design
- tasks
- restrições
- arquitetura explícita
- contratos claros

Também vimos como múltiplos agentes podem trabalhar simultaneamente utilizando:

- branches isoladas
- WorkTrees
- contexto separado
- tarefas específicas

Esse ponto mostrou uma mudança muito importante no papel do desenvolvedor.

Começamos a atuar muito mais como coordenadores de sistemas inteligentes.

Depois, no módulo 4, avançamos para qualidade e automação.

E aqui apareceu outro aprendizado extremamente importante:

IA não elimina testes.

Na verdade, quanto mais rápido conseguimos gerar software, mais importante se torna validar software.

Trabalhamos com:

- Cypress
- Playwright
- QA automatizado
- automação semântica
- self-healing
- Playwright MCP

Vimos diferentes abordagens para testes funcionais:

- geração tradicional de testes
- prompts semânticos
- agentes autônomos operando browsers

E isso mostrou algo extremamente interessante.

A IA não está apenas acelerando desenvolvimento.

Ela também está transformando profundamente a maneira como pensamos automação e qualidade de software.

Os testes começam a migrar de uma abordagem extremamente rígida e baseada em seletores para uma abordagem mais semântica e resiliente.

Outro ponto importante foi perceber que QA também se transforma.

Profissionais de qualidade passam a conseguir automatizar cenários muito mais rapidamente utilizando IA como acelerador operacional.

Depois disso, chegamos ao módulo final com o BragBot.

E aqui demos talvez o passo mais importante de toda a unidade.

Até então, a IA era principalmente uma ferramenta usada pelo desenvolvedor.

Agora ela passou a fazer parte da própria arquitetura da aplicação.

Esse é um ponto extremamente importante.

Nós utilizamos:

- GenKit

- Gemini
- flows
- Zod
- SSR
- integração full-stack

E começamos a trabalhar com IA como componente arquitetural real.

Isso muda bastante a forma como pensamos aplicações modernas.

Agora precisamos considerar:

- observabilidade
- tracing
- validação
- schemas
- contratos
- segurança
- desacoplamento
- gerenciamento de modelos

Também aprendemos algo extremamente importante sobre frameworks de IA.

O GenKit mostrou como frameworks modernos começam a abstrair complexidade de LLMs da mesma forma que:

- ORMs abstraem banco
- frameworks abstraem HTTP
- SDKs abstraem APIs

Isso será cada vez mais comum no mercado.

Outro ponto extremamente importante do BragBot foi perceber que engenharia de prompt não desaparece quando usamos frameworks.

Ela continua extremamente importante.

Mas agora começa a ficar encapsulada dentro da arquitetura da aplicação.

Também vimos a importância de:

- schemas com Zod
- validação estrutural
- observabilidade
- debugging
- Dev UI
- flows reutilizáveis

E talvez um dos maiores aprendizados de toda a unidade seja justamente este:

IA profissional não é apenas “mandar prompts”.

IA profissional envolve:

- arquitetura
- contexto
- validação
- observabilidade
- integração
- engenharia de software

Outro ponto extremamente importante foi perceber que nenhuma ferramenta é definitiva.

Ao longo da unidade usamos:

- Gemini
- GenKit
- Angular
- Nx
- Cypress
- Playwright
- Antigravity
- Jules
- OpenSpec

Mas todas essas ferramentas continuarão evoluindo rapidamente.

Novas ferramentas surgirão.

Outras desaparecerão.

Frameworks mudarão.

Modelos mudarão.

Então o mais importante não é decorar tecnologia específica.

O mais importante é entender os fundamentos.

Porque os fundamentos permanecem.

E talvez esse seja o principal objetivo da formação de um engenheiro AI-Native:

Aprender a adaptar-se rapidamente a um ecossistema tecnológico que evolui constantemente.

Outro aprendizado extremamente importante de toda a unidade foi perceber que IA não elimina necessidade de pensamento crítico.

Na verdade, ela aumenta essa necessidade.

Porque agora conseguimos produzir software muito mais rápido.

Então precisamos revisar, validar e arquitetar com muito mais responsabilidade.

Ao longo das aulas vimos várias situações onde:

- prompts precisavam ser refinados
- agentes precisavam de supervisão
- outputs continham inconsistências
- testes precisavam de validação
- flows precisavam de correção
- arquitetura precisava de revisão

Isso é extremamente importante.

A IA acelera execução.

Mas responsabilidade técnica continua humana.

Também vimos como prática é fundamental.

Não basta apenas assistir conteúdo ou copiar exemplos.

O verdadeiro aprendizado acontece quando começamos a aplicar os conceitos no dia a dia.

Por isso, o mais importante após esta unidade é continuar praticando.

Experimentar:

- outros frameworks
- outras stacks
- outros modelos
- outras arquiteturas
- novos fluxos
- novos agentes

Porque o mercado inteiro ainda está aprendendo coletivamente como trabalhar com IA dentro da engenharia de software.

E isso cria uma oportunidade extremamente interessante para profissionais que desenvolvem fundamentos sólidos desde agora.

Ao final desta jornada, talvez o ponto mais importante fique extremamente claro:

O futuro da engenharia de software não será substituição completa por IA.

Também não será desenvolvimento tradicional sem IA.

O futuro provavelmente será colaboração contínua entre humanos e sistemas inteligentes.

E os profissionais mais valiosos serão justamente aqueles que souberem integrar:

- engenharia clássica
- arquitetura
- qualidade
- pensamento crítico
- automação
- IA generativa

dentro de um fluxo moderno, produtivo e sustentável de desenvolvimento de software.

